

Sintaxis

Wednesday, April 2, 2025 1:56 PM



01_sintaxis

Lenguajes y Compiladores Introducción

Miguel Pagano
12 de marzo de 2025

¿Qué hace un compilador?

De un texto a una estructura de datos

Un programa en Python

```
def factorial(n):  
    if n <= 1:  
        return 1  
    else:  
        return factorial(n-1) * n
```

¿Qué ve el intérprete cuando usamos esa función?

```
>>> texto = """def factorial(n):  
...     if n <= 1:  
...         ...  
...     """  
>>> function = ast.parse(texto).body[0]
```

1

El output

```
>>> print(ast.dump(function.body[0],indent=2))  
If(  
  test=Compare(  
    left=Name(id='n', ctx=Load()),  
    ops=[  
      LtE(),  
    ],  
    comparators=[  
      Constant(value=1)],  
  ),  
  body=[  
    Return(  
      value=Constant(value=1)],  
  ),  
  orelse=[  
    Return(  
      value=BinOp(  
        left=Call(  
          func=Name(id='factorial', ctx=Load()),  
          args=[  
            BinOp(  
              left=Name(id='n', ctx=Load()),  
              op=Sub(),  
              right=Constant(value=1)],  
            keywords=[],  
            op=Mult(),  
            right=Name(id='n', ctx=Load())))]  
        )  
      )  
    ]  
  )
```

2

Fases de un compilador

Lexer

Reconoce los elementos sintácticos, *tokens*, válidos:

```
>>> g = tokenize.tokenize(BytesIO(texto.encode('utf-8')).readline)
>>> for i in g:
...     print(i)
TokenInfo(type=1 (NAME), string='def', start=(1, 0), end=(1, 3), line='
TokenInfo(type=1 (NAME), string='factorial', start=(1, 4), end=(1, 13),
TokenInfo(type=54 (OP), string='(', start=(1, 13), end=(1, 14), line='d
```

Los tokens válidos se describen con expresiones regulares.

3

Fases de un compilador

El parser

```
function_def:
    'def' NAME &&'(' [params] ')' && ':' block
```

A medida que se consume el input se genera un *árbol de sintaxis abstracta*.

3

Fases de un compilador

La gramática abstracta

se concentra en la información relevante:

```
<stmt> ::= 'FunctionDef' <identifier> <arguments> <stmt>* <expr>* <expr>?  
          <string>?  
          | ...
```

Sólo tenemos un **constructor** 'FunctionDef', un **no-terminal** <identifier> para el **nombre**, los **argumentos** (con su propia estructura), el **cuerpo** (que es una lista de 'stmt') y un par de **argumentos más** (opcionales).

3

Fases de un compilador

Análisis estático

Una vez que tenemos el árbol de **sintaxis abstracta** (AST) podemos analizarlo para detectar errores antes de ejecutar (por ejemplo, **chequeo de tipos**) o para optimizarlo (por ejemplo, ¿hay código inútil?).

Generación de código ó Interpretación

Finalmente, se genera código de bajo nivel a partir de un AST.

3

¿Cómo le damos sentido a un lenguaje?

1. Conociendo la **sintaxis abstracta** del lenguaje.
2. Explicando el **sentido** de cada construcción del lenguaje.

Descripciones de un lenguaje

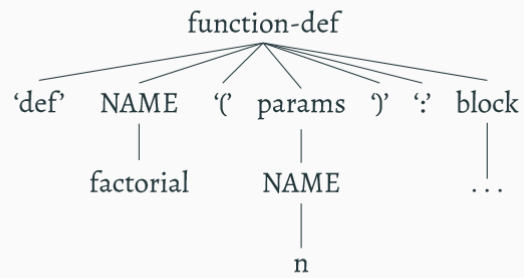
1. **Informal**: “The unary $-$ (minus) operator yields the negation of its numeric argument; the operation can be overridden with the `--neg__()` special method.”.
2. **Axiomática**: Si $P \Rightarrow Q[x := E]$, entonces $\{P\} x := E \{Q\}$.
3. **Operacional**: Si $e \rightsquigarrow [k]$, entonces $-e \rightsquigarrow [-k]$.
4. **Denotacional**: lo que veremos a continuación.

4

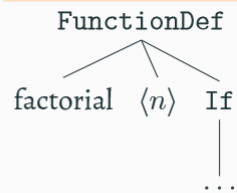
Sintaxis Abstracta

Dos gramáticas, dos árboles

Concreta: demasiada información irrelevante



Abstracta: sólo lo necesario para analizar la frase



5

¿Por qué Sintaxis Abstracta?

Queremos una notación matemática concisa que nos permita dar sentido a las construcciones del lenguaje sin preocuparnos de cómo se escriben.

6

¿Por qué Sintaxis Abstracta?

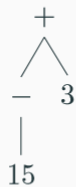
$$\begin{aligned}\langle exp \rangle ::= & \circ \mid 1 \mid 2 \mid \dots \\ & \mid - \langle exp \rangle \\ & \mid \langle exp \rangle * \langle exp \rangle \\ & \mid \langle exp \rangle + \langle exp \rangle\end{aligned}$$

Consideremos la expresión: $-15 + 3$.

6

¿Por qué Sintaxis Abstracta?

1. Diremos que $\langle exp \rangle$ es una *categoría sintáctica* de nuestro lenguaje.
2. Si bien lo escribimos linealmente, pensamos en árboles.
3. Como lo escribimos linealmente, puede haber ambigüedad.
4. Resolvemos la ambigüedad mediante reglas de asociatividad, precedencia, y usando paréntesis (sólo en caso de mucha necesidad).



Los árboles no son árboles de derivación: sólo ponemos los constructores (terminales) y los sub-árboles de sus argumentos.

6

¿Por qué Sintaxis Abstracta?

Nada nuevo bajo el sol

Esto es lo que venimos haciendo desde primer año, con un poco de problematización en IntroLog (y volviendo para atrás en Lógica).

6

Sintaxis abstracta

Ahora más matemáticamente

En IntroLog teníamos fórmulas, *la sintaxis* que la presentábamos como un conjunto definido inductivamente).

Si volvemos a nuestro ejemplo $\langle exp \rangle$, podemos pensar que la gramática abstracta establece que un conjunto E es un conjunto de frases abstractas si al tomar un elemento $e \in E$

1. podemos reconocer que e es 0 ó 1 ó 2, etc, ó
2. podemos reconocer que es $-e'$, con $e' \in E$,
3. podemos reconocer a $e_1 + e_2$, con $e_1, e_2 \in E$.

Es decir, 0, 1, ... son constantes en E y tanto $-_: E \rightarrow E$ como $+_: E \times E \rightarrow E$ son funciones inyectivas.

Ejercicio: dar ejemplos concretos de conjuntos de frases abstractas.

7

Semántica

Sentido

Repaso

En IntroLog hacíamos una distinción muy tajante entre la “sintaxis” y la “semántica”; la sintaxis eran las fórmulas (pura forma) a las que les dábamos **sentido** en el conjunto $\{0, 1\}$ mediante la función

$\llbracket _ \rrbracket : Prop \rightarrow \{0, 1\}$.

Al dar semántica, fórmulas distintas como $\top$ y $\perp \Rightarrow \perp$ quedaban igualadas.

¿Cómo definíamos $\llbracket _ \rrbracket$ para asegurarnos que estaba bien definida?

Sentido

Ecuaciones dirigidas por la sintaxis

Sin saberlo lo que hacíamos era dar una definición *dirigida por la sintaxis*:

1. Para cada constructor había una ecuación y
2. el lado derecho estaba dado en función de la semántica de sus sub-frases inmediatas.

$$\begin{aligned} \llbracket _ \rrbracket &: \langle \text{exp} \rangle \rightarrow \mathbb{Z} \\ \llbracket 0 \rrbracket &= 0 \\ \llbracket 1 \rrbracket &= 1 \\ &\vdots \\ \llbracket -e \rrbracket &= -\llbracket e \rrbracket \\ \llbracket e + e' \rrbracket &= \llbracket e \rrbracket + \llbracket e' \rrbracket \\ \llbracket e * e' \rrbracket &= \llbracket e \rrbracket * \llbracket e' \rrbracket \end{aligned}$$

8

Sentido

Observaciones

1. En $\llbracket 0 \rrbracket = 0$ las dos ocurrencias de 0 tienen tipos distintos; a esto le llamamos *meta-circularidad*: definimos el sentido de un símbolo por una entidad matemática (el cero) a la cual escribimos con ese mismo símbolo.
2. En $\llbracket -e \rrbracket = -\llbracket e \rrbracket$ estamos usando la *meta-variable* e , algo que pertenece al *meta-lenguaje*, para referirnos al argumento de $-$.
3. Decimos que $\langle \text{exp} \rangle$ es el *lenguaje objeto* y usamos toda la matemática como meta-lenguaje, es decir como lenguaje para describirlo y estudiarlo.
4. La dirección por sintaxis asegura la *composicionalidad de la semántica*; es decir, si en una frase e cambio una subfrase e_1 por otra e_2 obteniendo e' : si $\llbracket e_1 \rrbracket = \llbracket e_2 \rrbracket$, entonces $\llbracket e \rrbracket = \llbracket e' \rrbracket$.

8

Un poco más abstracto

En lenguaje de Lógica

Damos una interpretación para la signatura de nuestro lenguaje de términos: elegimos un conjunto A e interpretamos cada terminal, es un símbolo de función, como una función de acuerdo a la aridad del símbolo.

En lenguaje de Haskell

Podemos pensar que la gramática abstracta está dada por un tipo de datos (ejercicio) y que la semántica no es otra cosa que el “fold” asociado a ese tipo.

Sintaxis como semántica

asociado a ese tipo.

Sintaxis como semántica

La interpretación de cada constructor como sí mismo nos da un modelo. De hecho, hay una única función desde el modelo sintáctico a cualquier otro modelo.

9

La necesidad de un estado

Consideremos este nuevo lenguaje:

```

$$\begin{aligned} \langle \text{intexp} \rangle &::= \langle \text{var} \rangle \mid 0 \mid 1 \mid \dots \\ &\mid - \langle \text{intexp} \rangle \\ &\mid \langle \text{intexp} \rangle * \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle / \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle \% \langle \text{intexp} \rangle \\ &\mid \langle \text{intexp} \rangle + \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle - \langle \text{intexp} \rangle \\ \langle \text{assert} \rangle &::= \text{true} \mid \text{false} \\ &\mid \langle \text{intexp} \rangle = \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle < \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle \leq \langle \text{intexp} \rangle \\ &\mid \langle \text{intexp} \rangle \neq \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle > \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle \geq \langle \text{intexp} \rangle \\ &\mid \neg \langle \text{assert} \rangle \\ &\mid \langle \text{assert} \rangle \vee \langle \text{assert} \rangle \mid \langle \text{assert} \rangle \wedge \langle \text{assert} \rangle \\ &\mid \langle \text{assert} \rangle \Rightarrow \langle \text{assert} \rangle \mid \langle \text{assert} \rangle \equiv \langle \text{assert} \rangle \\ &\mid \exists \langle \text{var} \rangle. \langle \text{assert} \rangle \mid \forall \langle \text{var} \rangle. \langle \text{assert} \rangle \end{aligned}$$

```

$\langle \text{var} \rangle$ es un no-terminal sin constructores: es un conjunto numerable de “variables”.

10

La necesidad de un estado

Preguntas

- ¿Qué categorías sintácticas tiene el lenguaje?
- ¿Cuál es el dominio semántico “natural”?
- ¿Cuál es la diferencia con los símbolos de función de Lógica?
- ¿Las ecuaciones semánticas?

10

Variables

Wednesday, April 2, 2025 2:05 PM



sustitucion

Lenguajes y Compiladores Introducción

Miguel Pagano
15 de marzo de 2023

Repaso de nuestro lenguaje de fórmulas

La necesidad de un estado

Recordemos nuestro lenguaje:

$$\begin{aligned}
 \langle \text{intexp} \rangle &::= \langle \text{var} \rangle \mid 0 \mid 1 \mid \dots \\
 &\mid - \langle \text{intexp} \rangle \\
 &\mid \langle \text{intexp} \rangle * \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle / \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle \% \langle \text{intexp} \rangle \\
 &\mid \langle \text{intexp} \rangle + \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle - \langle \text{intexp} \rangle \\
 \langle \text{assert} \rangle &::= \text{true} \mid \text{false} \\
 &\mid \langle \text{intexp} \rangle = \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle < \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle \leq \langle \text{intexp} \rangle \\
 &\mid \langle \text{intexp} \rangle \neq \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle > \langle \text{intexp} \rangle \mid \langle \text{intexp} \rangle \geq \langle \text{intexp} \rangle \\
 &\mid \neg \langle \text{assert} \rangle \\
 &\mid \langle \text{assert} \rangle \vee \langle \text{assert} \rangle \mid \langle \text{assert} \rangle \wedge \langle \text{assert} \rangle \\
 &\mid \langle \text{assert} \rangle \Rightarrow \langle \text{assert} \rangle \mid \langle \text{assert} \rangle \Leftrightarrow \langle \text{assert} \rangle \\
 &\mid \exists \langle \text{var} \rangle . \langle \text{assert} \rangle \mid \forall \langle \text{var} \rangle . \langle \text{assert} \rangle
 \end{aligned}$$

$\langle \text{var} \rangle$ es un no-terminal sin constructores: es un conjunto numerable de "variables".

1

Ecuaciones semánticas

Para expresiones enteras

Σ es un estado
 $\sigma \in \Sigma$

Recordemos que $\Sigma = \langle \text{var} \rangle \rightarrow \mathbb{Z}$.

$$\begin{aligned}
 \llbracket _ \rrbracket_{\text{intexp}} &: \langle \text{intexp} \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z}) \\
 \llbracket v \rrbracket_{\text{intexp}} \sigma &= \sigma v \\
 \llbracket [k] \rrbracket_{\text{intexp}} \sigma &= k \\
 \llbracket -e \rrbracket_{\text{intexp}} \sigma &= -(\llbracket e \rrbracket_{\text{intexp}} \sigma) \\
 \llbracket e \oplus e' \rrbracket_{\text{intexp}} \sigma &= \llbracket e \rrbracket_{\text{intexp}} \sigma \oplus \llbracket e' \rrbracket_{\text{intexp}} \sigma
 \end{aligned}$$

2

Ecuaciones semánticas

Para expresiones booleanas

$$\begin{aligned}
 \llbracket _ \rrbracket_{\text{assert}} &: \langle \text{assert} \rangle \rightarrow (\Sigma \rightarrow \{0, 1\}) \\
 \llbracket \text{true} \rrbracket_{\text{assert}} \sigma &= 1 \\
 \llbracket \text{false} \rrbracket_{\text{assert}} \sigma &= 0 \\
 \llbracket \neg p \rrbracket_{\text{assert}} \sigma &= \neg(\llbracket p \rrbracket_{\text{assert}} \sigma) \\
 \llbracket e \otimes e' \rrbracket_{\text{assert}} \sigma &= \llbracket e \rrbracket_{\text{interp}} \sigma \otimes \llbracket e' \rrbracket_{\text{interp}} \sigma \\
 \llbracket p \odot p' \rrbracket_{\text{assert}} \sigma &= \llbracket p \rrbracket_{\text{assert}} \sigma \odot \llbracket p' \rrbracket_{\text{assert}} \sigma \\
 \llbracket \forall v. p \rrbracket_{\text{assert}} \sigma &= \bigcap_{n \in \mathbb{Z}} \llbracket p \rrbracket_{\text{assert}} [\sigma \mid v : n] \\
 \llbracket \exists v. p \rrbracket_{\text{assert}} \sigma &= \bigcup_{n \in \mathbb{Z}} \llbracket p \rrbracket_{\text{assert}} [\sigma \mid v : n]
 \end{aligned}$$

2

Ecuaciones semánticas

Observaciones

1. La notación $[\sigma \mid v : n]$ define la función

$$\sigma' w = \begin{cases} \sigma w & \text{si } w \neq v \\ n & \text{si } w = v \end{cases}$$

2. Podemos iterar la definición $[[\sigma \mid v : n] \mid w : m]$ que escribimos como $[\sigma \mid v : n \mid w : m]$.
3. A partir de ahora no pondremos los sub-índices en las ecuaciones semánticas (polimorfismo ad-hoc).
4. En la ecuación semántica de los cuantificadores queda claro que no importa el valor de σv .

¿Por qué no importa? ¿Podemos decir algo más general?

2

Variables, libres y ligadas

Exploremos algunas frases

Consideremos las frases:

1. $\exists v, w . (2 \leq v < z) \wedge (2 \leq w < z) \wedge v * w = z$
2. $\exists u, x . (2 \leq u < z) \wedge (2 \leq x < z) \wedge u * x = z$
3. $\exists v . (2 \leq v < z) \wedge (2 \leq w < z) \wedge v * w = z$

Diremos que dos frases p y p' son *equivalentes* si su semántica es la misma, es decir si $\llbracket p \rrbracket = \llbracket p' \rrbracket$.

¿Cuáles de esas tres frases son equivalentes entre sí?

Exploreemos algunas frases

Las veamos en color:

1. $\exists v, w. (2 \leq v < z) \wedge (2 \leq w < z) \wedge v * w = z$
2. $\exists u, x. (2 \leq u < z) \wedge (2 \leq x < z) \wedge u * x = z$
3. $\exists v. (2 \leq v < z) \wedge (2 \leq w < z) \wedge v * w = z$

Observación

Una frase *habla* de sus variables libres: las dos primeras expresiones son predicados sobre z mientras que la segunda es sobre z y w .

4

Exploreemos algunas frases

Definiciones

Ocurrencia Ligadora La ocurrencia de una variable se dice *ligadora* (binding) si está pegada a un cuantificador (binder); el *alcance* es el cuerpo de la cuantificación.

Ocurrencia Ligada Una ocurrencia de una variable v se dice *ligada* si está en el alcance de una ocurrencia ligadora de v .

Ocurrencia Libre Una ocurrencia de una variable v se dice *libre* si NO está en el alcance de una ocurrencia ligadora de v .

Variable Libre Una variable se dice *libre* si hay alguna ocurrencia libre de ella.

Expresión Cerrada Una expresión sin variables libres.

4

Exploremos algunas frases

Ejemplos

Repasemos esos conceptos con estos ejemplos.

1. $\exists v. (2 \leq v < z) \wedge (2 \leq w < z) \wedge v * w = z$
2. $\exists v. (2 \leq v < z) \wedge (2 \leq w < z) \wedge (\exists v. v > 5)$
3. $(\exists v. (2 \leq v < z) \wedge (2 \leq w < z)) \wedge (v > 5)$

4

Exploremos algunas frases

Renombres

- Sea p la expresión

$$\exists v, w. (2 \leq v < z) \wedge (2 \leq w < z) \wedge v * w = z$$

- Sea q la expresión

$$\exists u, x. (2 \leq u < z) \wedge (2 \leq x < z) \wedge u * x = z$$

p y q son equivalentes porque una es un renombre de la otra:
cambiamos la variable ligada v por la variable u y (al mismo tiempo)
la variable w por x .

4

Exploremos algunas frases

Coincidencia (o no)

- Sea p la expresión

$$\exists v, w. (2 \leq v < z) \wedge (2 \leq w < z) \wedge v * w = z$$

Ahora nos centramos en una única frase y recordemos la ecuación semántica para los cuantificadores. Supongamos que tenemos dos estados $\sigma, \sigma' \in \Sigma$, tales que $\sigma z = \sigma' z$.

¿Será que $\llbracket p \rrbracket \sigma = \llbracket p \rrbracket \sigma'$?

4

Variables Libres Formalmente

Para expresiones enteras

FV : free Variables

$$FV(_) : \langle intexp \rangle \rightarrow \mathcal{P}(\langle var \rangle)$$

$$FV(v) = \{v\}$$

$$FV([k]) = \emptyset$$

$$FV(-e) = FV(e)$$

$$FV(e \oplus e') = FV(e) \cup FV(e')$$

5

Para expresiones booleanas

$$FV(_) : \langle \text{assert} \rangle \rightarrow \mathcal{P}(\langle \text{var} \rangle)$$

$$FV(\mathbf{true}) = \emptyset$$

$$FV(\mathbf{false}) = \emptyset$$

$$FV(\neg p) = FV(p)$$

$$FV(e \otimes e') = FV(e) \cup FV(e')$$

$$FV(p \otimes p') = FV(p) \cup FV(p')$$

$$FV(Qv.p) = FV(p) \setminus \{v\}$$

5

Sustitución

Sustitución

Como vimos en el ejemplo de los renombres, necesitamos una operación sobre las frases que nos permita cambiar variables por otras variables y, más en general, por expresiones enteras.

Una **sustitución** es un mapeo de variables a expresiones enteras.

Usaremos $\Delta = \langle var \rangle \rightarrow \langle intexp \rangle$. Una sustitución es similar a un estado, en tanto que asigna posibles “valores” a las variables.

Podemos definir un operador de sustitución que deja intacta la estructura de la frase **instanciando** las variables libres con lo que indica la sustitución.

6

Sustitución

Sin embargo, hay que tener cuidado para no cambiar el sentido de la frase: Sabemos que

$$\forall x. (\exists y. y > x)$$

es cierto en $\mathbb{Z}$; entonces podríamos concluir que es válido para un valor concreto de x , por ejemplo mapeando $x \mapsto y + 1$ lo que daría lugar a:

$$\exists y. y > y + 1$$

El problema es que al hacer el reemplazo sintáctico estamos capturando a y .

6

Sustitución

Definición

Sea $\delta \in \Delta = \langle var \rangle \rightarrow \langle intexp \rangle$.

$$_/_ : \theta \times \Delta \rightarrow \theta$$

$$v/\delta = \delta v$$

$$[k]/\delta = [k]$$

$$(\neg p)/\delta = \neg(p/\delta)$$

$$(p \otimes p')/\delta = p/\delta \otimes p'/\delta$$

$$(Qv.p)/\delta = Q_{v_{new}}.p/[\delta \mid v : v_{new}]$$

$[k] \rightarrow \text{symbolo}$

$$\begin{aligned}
 (p \otimes p')/\delta &= p/\delta \otimes p'/\delta \\
 (Qv.p)/\delta &= Q^{v_{new}.p}/[\delta \mid v : v_{new}] \\
 &\text{donde } v_{new} \notin \bigcup_{w \in FV(p) \setminus \{v\}} FV(\delta w)
 \end{aligned}$$

6

Sustitución

Observaciones

1. La definición evita la captura pidiendo que v_{new} no vaya a capturar ninguna de las variables que introducirá la sustitución en el cuerpo del cuantificador.
2. ¿Quién es v_{new} ? Si v no está en el conjunto de nuevas variables libres, entonces es v . Si no, será la primera variable que no ocurra en ese conjunto.
3. Es decir, asumimos que hay algún orden (lo cual es gratis porque asumíamos que $\langle var \rangle$ era numerable).
4. En el Reynolds aparece c_{var} que es la función que inyecta variables como expresiones; es una especie de identidad de las sustituciones.

6

Teoremas

Coincidencia

Si dos estados σ y σ' coinciden en las variables libres de p , entonces $\llbracket p \rrbracket \sigma = \llbracket p \rrbracket \sigma'$.

Teorema

Para todo tipo θ , para toda frase p de tipo θ . Si para toda $v \in FV(p)$, $\sigma v = \sigma' v$; entonces $\llbracket p \rrbracket \sigma = \llbracket p \rrbracket \sigma'$.

Demostración.

Primero lo probamos para $\langle intexp \rangle$ y luego para $\langle assert \rangle$. En cada caso por inducción en p . $\square$

7

Sustitución

Si un estado σ' hace el trabajo combinado de un estado σ y una sustitución δ , restringidas a las variables libres de p ; entonces $\llbracket p/\delta \rrbracket \sigma = \llbracket p \rrbracket \sigma'$.

Teorema

Para todo tipo θ , para toda frase p de tipo θ . Para todas $\sigma, \sigma' \in \Sigma$ y $\delta \in \langle var \rangle \rightarrow \langle intexp \rangle$. Si para toda $v \in FV(p)$, $\llbracket \delta v \rrbracket \sigma = \sigma' v$; entonces $\llbracket p/\delta \rrbracket \sigma = \llbracket p \rrbracket \sigma'$.

Demostración.

Primero lo probamos para $\langle intexp \rangle$ y luego para $\langle assert \rangle$. En cada caso por inducción en p . $\square$

8

Sustitución finita

Sustituir algunas variables por expresiones y calcular la semántica es lo mismo que calcular la semántica en el estado donde esas variables vales la semántica de las expresiones.

Teorema

$$\llbracket p / [v_0 : e_0 \mid \dots \mid v_{n-1} : e_{n-1}] \rrbracket \sigma = \\ \llbracket p \rrbracket [\sigma \mid v_0 : \llbracket e_0 \rrbracket \sigma \mid \dots \mid v_{n-1} : \llbracket e_{n-1} \rrbracket \sigma]$$

Demostración.

Sale aplicando el teorema de sustitución. □

9

Renombre

Si hago un renombre de una variable ligada, el significado no cambia.

Teorema

$$\llbracket \forall u. p / [v : u] \rrbracket = \llbracket \forall v. p \rrbracket.$$

Demostración.

Sale aplicando el teorema de sustitución. □

10



recursion

Lenguajes y Compiladores

Miguel Pagano

22 de marzo de 2023

Ôtra vez sopa,
donde la Ô es por “Ôtra vez sopa”

¿Qué función es?

```
f :: Int -> Int
f n = if n == 0
      then 0
      else if n == 1
            then 1
            else f (n-2)
```

1

¿Qué función es?

```
f :: Int -> Int
f n = if n == 0
      then 0
      else if n == 1
            then 1
            else f (n-2)

ghci> import Control.Arrow ((&&&))
ghci> map (id &&& f) [0..9]
[(0,0),(1,1),(2,0),(3,1),(4,0),(5,1),(6,0),(7,1),(8,0),(9,1)]
ghci> f 9
1
ghci> f (-1)
```

1

¿Definición o Ecuación?

Lo escribamos en matemática

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

2

¿Definición o Ecuación?

Lo escribamos en matemática

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Interrogantes

1. ¿Se corresponde esta “definición” con la de Haskell?
2. ¿Define esta ecuación una función? ¿Qué dominio y qué rango tiene?
3. ¿Define una única función?
4. ¿Por qué las ecuaciones $\llbracket _ \rrbracket$ no tenían problemas?

2

¿Definición o Ecuación?

Lo escribamos en matemática

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Encontrás dependiendo de dónde buscás

1. ¿Qué sucede si asumimos $f: \mathbb{N} \rightarrow \mathbb{N}$?
2. ¿Por qué sería distinto si quisiéramos $f: \mathbb{Z} \rightarrow \mathbb{Z}$?
3. ¿Qué sucede si nos conformamos con $f: \mathbb{Z} \rightarrow \mathbb{Z}$?
4. ¿Cómo podríamos elegir una u otra solución en cada uno de esos conjuntos?

2

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Esta ecuación **recursiva** la podemos ver como una especificación.

Entonces dada una función $g: \mathbb{Z} \rightarrow \mathbb{Z}$ podemos ver si g satisface la ecuación probando:

1. $g\ 0 = 0$,
2. $g\ 1 = 1$ y
3. para todo $n \notin \{0, 1\}$, $g\ n = g\ (n - 2)$.

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$g_0(n) = n \bmod 2$$

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$g_0(n) = n \bmod 2$$

$$g_1(n) = \begin{cases} n \bmod 2 & \text{si } n \geq 0 \\ 23 & \text{si } n < 0 \end{cases}$$

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$g_0\ n = n \bmod 2$$

$$g_1\ n = \begin{cases} n \bmod 2 & \text{si } n \geq 0 \\ 23 & \text{si } n < 0 \end{cases}$$

1. $g_0(-1) = 1 \neq 23 = g_1(-1)$

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$g_0\ n = n \bmod 2$$

$$g_1\ n = \begin{cases} n \bmod 2 & \text{si } n \geq 0 \\ 23 & \text{si } n < 0 \end{cases}$$

1. $g_0(-1) = 1 \neq 23 = g_1(-1)$

2. Es decir son incomparables.

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$g_0\ n = n \bmod 2$$

$$g_1\ n = \begin{cases} n \bmod 2 & \text{si } n \geq 0 \\ 23 & \text{si } n < 0 \end{cases}$$

$$g_1 n = \begin{cases} n \bmod 2 & \text{si } n \geq 0 \\ 23 & \text{si } n < 0 \end{cases}$$

1. $g_0(-1) = 1 \neq 23 = g_1(-1)$
2. Es decir son incomparables.
3. Elegir una u otra es arbitrario.

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f n = \begin{cases} n0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

4

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f n = \begin{cases} n0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$h_0 n = n \bmod 2 \text{ si } n \geq 0$$

4

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$h_0(n) = n \bmod 2 \text{ si } n \geq 0$$

$$h_1(n) = n \bmod 2$$

1. ¿Cómo comparamos funciones parciales?

4

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$h_0(n) = n \bmod 2 \text{ si } n \geq 0$$

$$h_1(n) = n \bmod 2$$

1. ¿Cómo comparamos funciones parciales?
2. ¿Cuál preferimos?

4

La ecuación

$$f\ n = \begin{cases} n0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$h_0\ n = n \bmod 2 \text{ si } n \geq 0$$

$$h_1\ n = n \bmod 2$$

1. ¿Cómo comparamos funciones parciales?
2. ¿Cuál preferimos?
3. ¿Cuál representa mejor a la f de Haskell?

4

¡El bottom¹ del orden!

¹En inglés culo se dice bottom

Introduciendo parcialidad explícitamente

De $\mathbb{Z} \rightarrow \mathbb{Z}$ a $\mathbb{Z} \rightarrow \mathbb{Z}_\perp$

Lo que vamos a hacer es **totalizar las funciones parciales** agregando un **nuevo elemento, $\perp$** que llamamos **bottom**, al codominio.

Repaso de órdenes parciales

Un conjunto X junto con un orden parcial $\leq$ sobre X es un **poset**. Un orden parcial es una **relación reflexiva, anti-simétrica y transitiva**.

5

Introduciendo parcialidad explícitamente

Repaso de órdenes parciales

Un conjunto X junto con un orden parcial $\leq$ sobre X es un **poset**. Un orden parcial es una relación reflexiva, anti-simétrica y transitiva.

Ejemplos

1. $(\mathbb{Z}, \leq)$.
2. $(\mathcal{P}(X), \subseteq)$.
3. $(X, =)$ es el poset **discreto**.

5

Introduciendo parcialidad explícitamente

Repaso de órdenes parciales

Un conjunto X junto con un **orden parcial** $\leq$ sobre X es un **poset**. Un **orden parcial** es una relación **reflexiva, anti-simétrica y transitiva**.

Poset de Funciones Parciales

Como $X \rightarrow Y \subseteq \mathcal{P}(X \times Y)$, podemos ordenar las funciones parciales usando la contención de conjuntos.

Explícitamente tenemos que $f \leq g$ si para todo $x \in X$,

$f(\{x\}) \subseteq g(\{x\})$, que es equivalente a

si $f(x)$ está definido entonces $g(x)$ también lo está y $f(x) = g(x)$.

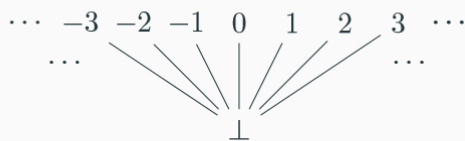
¿Cuál es el menor elemento de $X \rightarrow Y$?

5

Construcciones sobre Posets

Lifting

- Sea X un conjunto y $\perp \notin X$, entonces $X_\perp = X \cup \{\perp\}$.
- Si $(X, \preceq)$ es un poset, entonces $(X_\perp, \preceq_\perp)$ también lo es:
 $x \preceq_\perp x'$ si y sólo si $x = \perp$ o $x \preceq x'$.
- El poset $(X_\perp, \preceq_\perp)$ se llama el **poset llano**.



- Está claro que $\perp$ es el mínimo del poset $(X_\perp, \preceq_\perp)$.
- A partir de ahora usaremos $\preceq$ para referirnos a el poset que tenga sentido. Si hay posibilidad de confusión lo decoramos.
- Otras veces nos referimos al poset sólo mediante su conjunto.

6

Construcciones sobre Posets

Topping

- Sea X un conjunto y $\top \notin X$, entonces $X^\top = X \cup \{\top\}$.
- Si $(X, \preceq)$ es un poset, entonces $(X^\top, \preceq^\top)$ también lo es:
 $x \preceq^\top x'$ si y sólo si $x' = \top$ o $x \preceq x'$.
- Ejemplo: $\mathbb{N}_\infty$ que deberíamos escribirlo como $\mathbb{N}^\infty$.

Producto de posets

Si $(X, \preceq_X)$ y $(Y, \preceq_Y)$ son posets, entonces $(X \times Y, \preceq_{X \times Y})$ también lo es:

$\langle x, y \rangle \preceq_{X \times Y} \langle x', y' \rangle$ si y sólo si $x \preceq_X x'$ e $y \preceq_Y y'$.

6

Construcciones sobre Posets

Espacio de funciones

Si $(Y, \preceq)$ es un poset, entonces $(X \rightarrow Y, \preceq^\rightarrow)$ también lo es
 $f \preceq^\rightarrow g$ sii para todo $x \in X$, $f x \preceq g x$.

Observaciones

1. Notemos que acá tenemos un único poset $(Y, \preceq)$; X es sólo un conjunto.
2. Ejercicio: $(X \rightarrow Y, \subseteq)$ es isomorfo a $(X \rightarrow Y_\perp, \preceq^\rightarrow)$.

6

Predominios y Dominios

Más repaso

Supremo

Dado un poset $(X, \preceq)$ y $Q \subseteq X$, decimos que

- $b \in X$ es una **cota superior para Q** si $x \preceq b$ para todo $x \in Q$.
- $a \in X$ es el **supremo de Q** si es la menor de las cotas superiores.

Observaciones

1. El **supremo de Q** no necesariamente pertenece a Q .
2. Si tomamos $(\mathbb{N}, \leq)$ y Q como los números pares, tenemos que Q no tiene supremo.
3. ¿Cambia la respuesta si nos pasamos a $\mathbb{N}^\infty$?
4. Supongamos que X es finito, ¿todo subconjunto de X tiene supremo?

Predominio

Cadena

Dado un poset $(X, \preceq)$ decimos que una secuencia $x_0, x_1, \dots, x_n, \dots$ es una **cadena** si $x_i \preceq x_{i+1}$, para todo $i \in \mathbb{N}$.

Decimos que una cadena es **interesante** si tiene infinitos elementos distintos.

Ejemplos en $\mathbb{N}$

1. $23 \leq 23 \leq \dots \leq 23 \leq 23$,
2. $i * 2$ para $i \in \mathbb{N}$ (los pares vistos como cadena),

Ejemplos en $\mathbb{N}_\perp$

1. $23 \leq 23 \leq \dots \leq 23 \leq 23$,
2. En $\perp \preceq \perp \preceq \dots \preceq \perp \preceq 4 \preceq x \preceq \dots \preceq y \preceq \dots$,
qué pueden ser x e y ?

8

Predominio

Cadena

Dado un poset $(X, \preceq)$ decimos que una secuencia $x_0, x_1, \dots, x_n, \dots$ es una **cadena** si $x_i \preceq x_{i+1}$, para todo $i \in \mathbb{N}$.

Decimos que una cadena es **interesante** si tiene infinitos elementos distintos.

Definición

Un poset $(P, \preceq)$ es un **predominio** si toda cadena tiene supremo.

Observaciones

1. Si P es finito, ¿toda cadena tiene supremo?
2. Más en general, una cadena no interesante tiene supremo?
3. En $X_\perp$, hay cadenas interesantes?

8

Predominio

Definición

Un poset $(P, \preceq)$ es un **predominio** si toda cadena tiene supremo.

Caracterización

P es un dominio si todas las cadenas interesantes de P tienen supremo.

Corolario: $X_{\perp}$ es un dominio siempre.

Ejemplo y contra-ejemplo

1. $(\mathbb{Z}, \leq)$ no es un dominio (de hecho $\mathbb{N}$ tampoco).
2. $(\mathbb{Z}^{\infty}, \preceq)$ es un dominio.
3. $(\mathcal{P}(X), \subseteq)$ es un dominio, cuál es el supremo de Q_i ?

8

Construcciones sobre dominios

1. Ya vimos que $X_{\perp}$ es un dominio.
2. También vimos que $\mathbb{N}^{\infty}$ es dominio;
3. más en general si P es un dominio, entonces $P^{\top}$ también lo es, pero
4. por ejemplo $(\mathbb{R} \setminus \{2\})^{\infty}$ no es un dominio porque $2 - (1/i)$ es una cadena interesante sin supremo.
5. ¿Qué pasa con $P \times Q$? Si ambos son dominios, lo es $P \times Q$?

9

Construcciones sobre predomnios

Espacio de funciones

Sea Y es un predominio y X un conjunto. ¿Cómo es una cadena de funciones en $X \rightarrow Y$?

Sea f_i una cadena de funciones, entonces podemos definir $\sup_{X \rightarrow Y}(f_i) x = \sup_Y(\{f_i x | i \in \mathbb{N}\})$.

Ejercicio: probar que está bien esa definición.

9

Dominios

Un predominio P es un dominio si tiene mínimo.²

²Volviendo al chiste del principio, un dominio es un predominio con culo.

10

rew 2

Monday, September 22, 2025

5:20 PM



recursion-2

Lenguajes y Compiladores

Miguel Pagano

29 de marzo de 2023

Ótra vez sopa,
donde la Ó es por “Ótra vez sopa”

¿Qué función es?

```
f :: Int -> Int
f n = if n == 0
      then 0
      else if n == 1
            then 1
            else f (n-2)
```

1

¿Qué función es?

```
f :: Int -> Int
f n = if n == 0
      then 0
      else if n == 1
            then 1
            else f (n-2)

ghci> import Control.Arrow ((&&&))
ghci> map (id &&& f) [0..9]
[(0,0),(1,1),(2,0),(3,1),(4,0),(5,1),(6,0),(7,1),(8,0),(9,1)]
ghci> f 9
1
ghci> f (-1)
```

1

¿Definición o Ecuación?

Lo escribamos en matemática

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

2

¿Definición o Ecuación?

Lo escribamos en matemática

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Interrogantes

1. ¿Se corresponde esta “definición” con la de Haskell?
2. ¿Define esta ecuación una función? ¿Qué dominio y qué rango tiene?
3. ¿Define una única función?
4. ¿Por qué las ecuaciones $\llbracket _ \rrbracket$ no tenían problemas?

2

¿Definición o Ecuación?

Lo escribamos en matemática

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Encontrás dependiendo de dónde buscás

1. ¿Qué sucede si asumimos $f: \mathbb{N} \rightarrow \mathbb{N}$?
2. ¿Por qué sería distinto si quisiéramos $f: \mathbb{Z} \rightarrow \mathbb{Z}$?
3. ¿Qué sucede si nos conformamos con $f: \mathbb{Z} \rightarrow \mathbb{Z}$?
4. ¿Cómo podríamos elegir una u otra solución en cada uno de esos conjuntos?

2

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Esta ecuación **recursiva** la podemos ver como una especificación. Entonces dada una función $g: \mathbb{Z} \rightarrow \mathbb{Z}$ podemos ver si g satisface la ecuación probando:

1. $g\ 0 = 0$,
2. $g\ 1 = 1$ y
3. para todo $n \notin \{0, 1\}$, $g\ n = g\ (n - 2)$.

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$g_0\ n = n \bmod 2$$

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$g_0\ n = n \bmod 2$$

$$g_1\ n = \begin{cases} n \bmod 2 & \text{si } n \geq 0 \\ 23 & \text{si } n < 0 \end{cases}$$

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$g_0 n = n \bmod 2$$

$$g_1 n = \begin{cases} n \bmod 2 & \text{si } n \geq 0 \\ 23 & \text{si } n < 0 \end{cases}$$

1. $g_0(-1) = 1 \neq 23 = g_1(-1)$

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$g_0 n = n \bmod 2$$

$$g_1 n = \begin{cases} n \bmod 2 & \text{si } n \geq 0 \\ 23 & \text{si } n < 0 \end{cases}$$

1. $g_0(-1) = 1 \neq 23 = g_1(-1)$

2. Es decir son incomparables.

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$g_0\ n = n \bmod 2$$

$$g_1\ n = \begin{cases} n \bmod 2 & \text{si } n \geq 0 \\ 23 & \text{si } n < 0 \end{cases}$$

1. $g_0(-1) = 1 \neq 23 = g_1(-1)$
2. Es decir son incomparables.
3. Elegir una u otra es arbitrario.

3

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} n0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

4

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$h_0(n) = n \bmod 2 \text{ si } n \geq 0$$

4

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$h_0(n) = n \bmod 2 \text{ si } n \geq 0$$

$$h_1(n) = n \bmod 2$$

1. ¿Cómo comparamos funciones parciales?

4

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} n0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$h_0\ n = n \bmod 2 \text{ si } n \geq 0$$

$$h_1\ n = n \bmod 2$$

1. ¿Cómo comparamos funciones parciales?
2. ¿Cuál preferimos?

4

Soluciones en $\mathbb{Z} \rightarrow \mathbb{Z}$

La ecuación

$$f\ n = \begin{cases} n0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f\ (n - 2) & \text{si } n \notin \{0, 1\} \end{cases}$$

Soluciones

$$h_0\ n = n \bmod 2 \text{ si } n \geq 0$$

$$h_1\ n = n \bmod 2$$

1. ¿Cómo comparamos funciones parciales?
2. ¿Cuál preferimos?
3. ¿Cuál representa mejor a la f de Haskell?

4

¡El bottom del orden!

Introduciendo parcialidad explícitamente

De $\mathbb{Z} \rightarrow \mathbb{Z}$ a $\mathbb{Z} \rightarrow \mathbb{Z}_\perp$

Lo que vamos a hacer es totalizar las funciones parciales agregando un nuevo elemento, $\perp$ que llamamos **bottom**, al codominio.

Repaso de órdenes parciales

Un conjunto X junto con un orden parcial $\leq$ sobre X es un **poset**. Un orden parcial es una relación reflexiva, anti-simétrica y transitiva.

Introduciendo parcialidad explícitamente

Repaso de órdenes parciales

Un conjunto X junto con un orden parcial $\leq$ sobre X es un **poset**. Un orden parcial es una relación reflexiva, anti-simétrica y transitiva.

Ejemplos

1. $(\mathbb{Z}, \leq)$.
2. $(\mathcal{P}(X), \subseteq)$.
3. $(X, =)$ es el poset **discreto**.

5

Introduciendo parcialidad explícitamente

Repaso de órdenes parciales

Un conjunto X junto con un orden parcial $\leq$ sobre X es un **poset**. Un orden parcial es una relación reflexiva, anti-simétrica y transitiva.

Poset de Funciones Parciales

Como $X \rightarrow Y \subseteq \mathcal{P}(X \times Y)$, podemos ordenar las funciones parciales usando la contención de conjuntos.

Explícitamente tenemos que $f \leq g$ si para todo $x \in X$,

$f(\{x\}) \subseteq g(\{x\})$, que es equivalente a

si $f(x)$ está definido entonces $g(x)$ también lo está y $f(x) = g(x)$.

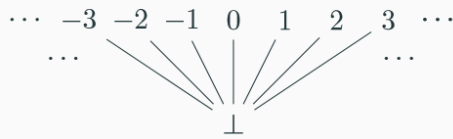
¿Cuál es el menor elemento de $X \rightarrow Y$?

5

Construcciones sobre Posets

Lifting

- Sea X un conjunto y $\perp \notin X$, entonces $X_{\perp} = X \cup \{\perp\}$.
- Si $(X, \preceq)$ es un poset, entonces $(X_{\perp}, \preceq_{\perp})$ también lo es:
 $x \preceq_{\perp} x'$ si y sólo si $x = \perp$ o $x \preceq x'$.
- El poset $(X_{\perp}, =_{\perp})$ se llama el poset **llano**.



- Está claro que $\perp$ es el mínimo del poset $(X_{\perp}, \preceq_{\perp})$.
- A partir de ahora usaremos $\preceq$ para referirnos a el poset que tenga sentido. Si hay posibilidad de confusión lo decoramos.
- Otras veces nos referimos al poset sólo mediante su conjunto.

6

Construcciones sobre Posets

Topping

- Sea X un conjunto y $\top \notin X$, entonces $X^{\top} = X \cup \{\top\}$.
- Si $(X, \preceq)$ es un poset, entonces $(X^{\top}, \preceq^{\top})$ también lo es:
 $x \preceq^{\top} x'$ si y sólo si $x' = \top$ o $x \preceq x'$.
- Ejemplo: $\mathbb{N}_{\infty}$ que deberíamos escribirlo como $\mathbb{N}^{\infty}$.

Producto de posets

Si $(X, \preceq_X)$ y $(Y, \preceq_Y)$ son posets, entonces $(X \times Y, \preceq_{X \times Y})$ también lo es:

$\langle x, y \rangle \preceq_{X \times Y} \langle x', y' \rangle$ si y sólo si $x \preceq_X x'$ e $y \preceq_Y y'$.

6

Espacio de funciones

Si $(Y, \preceq)$ es un poset, entonces $(X \rightarrow Y, \preceq^{\rightarrow})$ también lo es
 $f \preceq^{\rightarrow} g$ sii para todo $x \in X, f x \preceq g x$.

Observaciones

1. Notemos que acá tenemos un único poset $(Y, \preceq)$; X es sólo un conjunto.
2. Ejercicio: $(X \rightarrow Y, \subseteq)$ es isomorfo a $(X \rightarrow Y_{\perp}, \preceq^{\rightarrow})$.

6

Predominios y Dominios

Más repaso

Supremo

Dado un poset $(X, \preceq)$ y $Q \subseteq X$, decimos que

- $b \in X$ es una **cota superior para** Q si $x \preceq b$ para todo $x \in Q$.
- $a \in X$ es el **supremo de** Q si es la menor de las cotas superiores.

Observaciones

1. El supremo de Q no necesariamente pertenece a Q .
2. Si tomamos $(\mathbb{N}, \leq)$ y Q como los números pares, tenemos que Q no tiene supremo.
3. ¿Cambia la respuesta si nos pasamos a $\mathbb{N}^\infty$?
4. Supongamos que X es finito, ¿todo subconjunto de X tiene supremo?

7

Predominio

Cadena

Dado un poset $(X, \preceq)$ decimos que una secuencia $x_0, x_1, \dots, x_n, \dots$ es una **cadena** si $x_i \preceq x_{i+1}$, para todo $i \in \mathbb{N}$.

Decimos que una cadena es **interesante** si tiene infinitos elementos distintos.

Ejemplos en $\mathbb{N}$

1. $23 \leq 23 \leq \dots \leq 23 \leq 23 \leq \dots$,
2. $i * 2$ para $i \in \mathbb{N}$ (los pares vistos como cadena),

Ejemplos en $\mathbb{N}_\perp$

1. $23 \leq 23 \leq \dots \leq 23 \leq 23 \leq \dots$,
2. En $\perp \preceq \perp \preceq \dots \preceq \perp \preceq 4 \preceq x \preceq \dots \preceq y \preceq \dots$,
qué pueden ser x e y ?

8

Predominio

Cadena

Dado un poset $(X, \preceq)$ decimos que una secuencia $x_0, x_1, \dots, x_n, \dots$ es una **cadena** si $x_i \preceq x_{i+1}$, para todo $i \in \mathbb{N}$.

Decimos que una cadena es **interesante** si tiene infinitos elementos distintos.

Definición

Un poset $(P, \preceq)$ es un **predominio** si toda cadena tiene supremo.

Observaciones

1. Si P es finito, ¿toda cadena tiene supremo?
2. Más en general, una cadena no interesante tiene supremo?
3. En $X_\perp$, hay cadenas interesantes?

8

Predominio

Definición

Un poset $(P, \preceq)$ es un **predominio** si toda cadena tiene supremo.

Caracterización

P es un dominio si **todas las cadenas interesantes de P tienen supremo.**

Corolario: $X_\perp$ es un dominio siempre.

Ejemplo y contra-ejemplo

1. $(\mathbb{Z}, \leq)$ no es un dominio (de hecho $\mathbb{N}$ tampoco).
2. $(\mathbb{Z}^\infty, \preceq)$ es un dominio.
3. $(\mathcal{P}(X), \subseteq)$ es un dominio, cuál es el supremo de Q_i ?

8

Construcciones sobre predomnios

1. Ya vimos que $X_{\perp}$ es un predominio.
2. También vimos que $\mathbb{N}^{\infty}$ es dominio;
3. más en general si P es un predominio, entonces $P^{\top}$ también lo es, pero
4. por ejemplo $(\mathbb{R} \setminus \{2\})^{\infty}$ no es un predominio porque $2 - (1/i)$ es una cadena interesante sin supremo.
5. ¿Qué pasa con $P \times Q$? Si ambos son predomnios, lo es $P \times Q$?

9

Construcciones sobre predomnios

Espacio de funciones

Sea Y es un predominio y X un conjunto. ¿Cómo es una cadena de funciones en $X \rightarrow Y$?

Sea f_i una cadena de funciones, entonces podemos definir

$$\bigsqcup_{X \rightarrow Y} (f_i) x = \bigsqcup_Y (\{f_i x \mid i \in \mathbb{N}\}).$$

Ejercicio: probar que está bien esa definición.

9

Un predominio P es un dominio si tiene mínimo. Vamos a empacar la definición en una tupla de cuatro cosas: $D = (D, \sqsubseteq, \sqcup, \perp)$

Lema

Si D es un dominio, entonces $X \rightarrow D$ también lo es:

1. $\perp_{X \rightarrow D}$ está definido como:

$$\perp_{X \rightarrow D} : X \rightarrow D$$

$$\perp_{X \rightarrow D} x = \perp_D$$

esto es la función mínimo.

2. El supremo de una cadena de funciones ya lo definimos.

10

Funciones entre posets, predominios, dominios

Definiciones

Funciones monótonas

Dados dos posets $(P, \leq)$, $(Q, \sqsubseteq)$ y una función $f: P \rightarrow Q$, decimos que f es **monótona** si $x \leq y$ implica $f x \sqsubseteq f y$.

Observaciones

1. Es una función que respeta la estructura de los posets.
2. Si $f: P \rightarrow P'$ es una **función entre predomnios**, tenemos $\bigsqcup' (f x_i) \sqsubseteq' f (\bigsqcup x_i)$ para cualquier cadena $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \sqsubseteq \dots$ de P . $\bigsqcup$ supremo
3. (Recordatorio) $x \sqsubseteq y$ define una cadena: $x \sqsubseteq y \sqsubseteq y \sqsubseteq \dots$
4. (Recordatorio) $x \sqsubseteq y$ si y sólo si $x \sqcup y = y$.

11

Definiciones

Funciones continuas

Dados dos predomnios $(P, \sqsubseteq, \bigsqcup)$, $(P', \sqsubseteq', \bigsqcup')$ y $f: P \rightarrow P'$, decimos que f es **continua** si, para toda cadena $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \sqsubseteq \dots$, se da $f (\bigsqcup x_i) = \bigsqcup' (f x_i)$.

Observaciones

1. Es una función que respeta la estructura de los **predomnios (supremos)**.
2. Por los dos recordatorios, continua implica monótona.
3. Por la segunda propiedad, es equivalente a pedir que sea monótona y $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \sqsubseteq \dots$, se da $f (\bigsqcup x_i) \sqsubseteq \bigsqcup' (f x_i)$.
4. Para **romper continuidad de una función monótona** $f: P \rightarrow P'$ debemos proponer una **cadena concreta** de P tal que $\bigsqcup' (f x_i) \sqsubset f (\bigsqcup x_i)$.

11

monotonas
continuas
estrictas

Definiciones

Funciones estrictas

Dados dos dominios $(D, \sqsubseteq, \sqcup, \perp)$, $(P', \sqsubseteq', \sqcup') \perp'$ y $f: P \rightarrow P'$, decimos que f es **estricta** si se da $f \perp = \perp'$.

Observaciones

1. Es una función que respeta parte de la estructura de los dominios (supremos).
2. Para respetar todo, además tiene que ser continua.

11

Espacio de funciones

La clase pasada vimos que podíamos definir varias construcciones sobre Posets, Predominios y Dominios.

En general cuando trabajamos con estructuras queremos que el **espacio de funciones sea el conjunto de las funciones que respetan la estructura**. Es decir,

- En **Posets**, $P \rightarrow Q = \{f: P \rightarrow Q \mid f \text{ es monótona}\}$.
- En **Predominios**, $P \rightarrow P' = \{f: P \rightarrow P' \mid f \text{ es continua}\}$.
- En **Dominios**,
 $D \rightarrow D = \{f: D \rightarrow D' \mid f \text{ es continua y estricta}\}$.
- En **Dominios** también,
 $D \rightarrow D' = \{f: D \rightarrow D' \mid f \text{ es continua}\}$.

Entonces deberíamos probar que $\bigsqcup f_i: P \rightarrow P'$ es continua.

12

Categorías (no es importante)

En cada una de esas **categorías**, posets con funciones monótonas, predominios con funciones continuas, dominios con funciones continuas (y estrictas), tenemos:

1. $id_P: P \rightarrow P$ es **monótona, continua, continua y estricta**.
2. Si $f: P \rightarrow P'$ y $g: P' \rightarrow P''$, entonces $g \circ f: P \rightarrow P''$.

Esto nos dice que tenemos un categoría de Posets y funciones monótonas, de Predominios y funciones continuas, de dominios y funciones continuas y estrictas. También una de dominios y funciones continuas (pero no necesariamente estrictas).

Las clausuras por productos y espacio de funciones nos hablan de

funciones continuas (pero no necesariamente estrictas).

Las clausuras por productos y espacio de funciones nos hablan de categorías cartesianas cerradas.

13

Teorema del menor punto fijo

Sea $f: A \rightarrow A$, decimos que $a \in A$ es un **punto fijo de f** si $f a = a$.

14

Teorema del menor punto fijo

Sea $f: A \rightarrow A$, decimos que $a \in A$ es un **punto fijo de f** si $f a = a$.

Sea $F: D \rightarrow D$ una **función continua**. Entonces existe el **menor punto fijo de F** .

14

Teorema del menor punto fijo

Sea $f: A \rightarrow A$, decimos que $a \in A$ es un **punto fijo de f** si $f a = a$.

Sea $F: D \rightarrow D$ una función **continua**. Entonces existe el menor punto fijo de F .

Consideremos la cadena $\perp \sqsubseteq F \perp \sqsubseteq F^2 \perp \sqsubseteq F^3 \perp \sqsubseteq \dots$.

14

Una posible demostración

1. Lema: Si x_i es una ω -cadena, entonces x_{i+1} también lo es.
2. Lema: Las cadenas x_i y x_{i+1} tienen el mismo supremo.
3. Lema: Si $x_i \sqsubseteq y$, para todo $i \in \mathbb{N}$, entonces $\bigsqcup_{i \in \mathbb{N}} x_i \sqsubseteq y$.

15

Imperativo simple

Monday, September 22, 2025

5:20 PM



05_imperativo

Lenguajes y Compiladores

Mati Steinberg y Miguel Pagano
5 de abril de 2024

Repaso

Predominio

Cadena

Dado un poset $(X, \leq)$ decimos que una secuencia $x_0, x_1, \dots, x_n, \dots$ es una **cadena** si $x_i \leq x_{i+1}$, para todo $i \in \mathbb{N}$.

Decimos que una cadena es **interesante** si tiene infinitos elementos distintos.

1

Predominio

Cadena

Dado un poset $(X, \leq)$ decimos que una secuencia $x_0, x_1, \dots, x_n, \dots$ es una **cadena** si $x_i \leq x_{i+1}$, para todo $i \in \mathbb{N}$.

Decimos que una cadena es **interesante** si tiene infinitos elementos distintos.

Definición

Un poset $(P, \leq)$ es un **predominio** si toda cadena tiene supremo.

Caracterización

P es un **predominio** si todas las cadenas interesantes de P tienen supremo.

Corolario: $X_\perp$ es un dominio siempre.

1

Construcciones sobre predomnios

1. Ya vimos que $X_{\perp}$ es un predominio.
2. Si P es un predominio, entonces $P_{\perp}$ y $P^{\top}$ también lo son.
3. Si tanto P como Q son predomnios, también lo es $P \times Q$.
4. Si P es un predominio, entonces $A \rightarrow P$ también.

2

Construcciones sobre predomnios

Lema P es pro $\rightarrow (A \rightarrow P)$ es pred $\forall x \in A$
 $f \in g$ $f x \in g x$

Espacio de funciones

Sea P es un predominio y A un conjunto. ¿Cómo es una cadena de funciones en $A \rightarrow P$?

Sea f_i una cadena de funciones, entonces podemos definir

$$\sqcup_{A \rightarrow P}(f_i) x = \sqcup_P(\{f_i x \mid i \in \mathbb{N}\}).$$

Ejercicio: probar que está bien esa definición.

2

Dominios

Un predominio P es un **dominio** si tiene mínimo. Vamos a empacar la definición en una tupla de cuatro cosas: $D = (D, \sqsubseteq, \sqcup, \perp)$

Lema

Si D es un dominio, entonces $X \rightarrow D$ también lo es:

1. $\perp_{X \rightarrow D}$ está definido como:

$$\perp_{X \rightarrow D} : X \rightarrow D$$

$$\perp_{X \rightarrow D} x = \perp_D$$

2. El supremo de una cadena de funciones ya lo definimos.

3

Funciones

Funciones monótonas

Dados dos posets $(P, \leq)$, $(Q, \sqsubseteq)$ y una función $f: P \rightarrow Q$, decimos que f es **monótona** si $x \leq y$ implica $f x \sqsubseteq f y$.

4

Funciones

Funciones monótonas

Dados dos posets $(P, \leq)$, $(Q, \sqsubseteq)$ y una función $f: P \rightarrow Q$, decimos que f es **monótona** si $x \leq y$ implica $f x \sqsubseteq f y$.

Funciones continuas

Dados dos predomnios $(P, \sqsubseteq, \sqcup)$, $(P', \sqsubseteq', \sqcup')$ y $f: P \rightarrow P'$, decimos que f es **continua** si, para toda cadena $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \sqsubseteq \dots$, se da $f(\sqcup x_i) = \sqcup' (f x_i)$.

4

Funciones

Funciones monótonas

Dados dos posets $(P, \leq)$, $(Q, \sqsubseteq)$ y una función $f: P \rightarrow Q$, decimos que f es **monótona** si $x \leq y$ implica $f x \sqsubseteq f y$.

Funciones continuas

Dados dos predomnios $(P, \sqsubseteq, \sqcup)$, $(P', \sqsubseteq', \sqcup')$ y $f: P \rightarrow P'$, decimos que f es **continua** si, para toda cadena $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \sqsubseteq \dots$, se da $f(\sqcup x_i) = \sqcup' (f x_i)$.

Funciones estrictas

Dados dos dominios $(D, \sqsubseteq, \sqcup, \perp)$, $(P', \sqsubseteq', \sqcup') \perp'$ y $f: P \rightarrow P'$, decimos que f es **estricta** si se da $f \perp = \perp'$.

4

Funciones

Extensión estrictas

Si $f: A \rightarrow B$, entonces definimos su **extensión estricta**, $f_{\perp}: A_{\perp} \rightarrow B_{\perp}$

$$f_{\perp} x = \begin{cases} \perp & \text{si } x = \perp \\ f a & \text{si } x = a \end{cases}$$

5

Funciones

Extensión estrictas

Si $f: A \rightarrow B$, entonces definimos su **extensión estricta**, $f_{\perp}: A_{\perp} \rightarrow B_{\perp}$

$$f_{\perp} x = \begin{cases} \perp & \text{si } x = \perp \\ f a & \text{si } x = a \end{cases}$$

Extensiones estrictas

Si $f: P \rightarrow D$, entonces definimos su **extensión estricta**, $f_{\perp}: P_{\perp} \rightarrow D$

$$f_{\perp} x = \begin{cases} \perp & \text{si } x = \perp \\ f a & \text{si } x = a \end{cases}$$

5

Lenguaje de Programación

Un Lenguaje Imperativo Simple

Comandos

```
 $\langle comm \rangle ::= skip$   
|  $\langle var \rangle := \langle intexp \rangle$   
|  $\langle comm \rangle ; \langle comm \rangle$   
| if  $\langle boolexp \rangle$  then  $\langle comm \rangle$  else  $\langle comm \rangle$   
| newvar  $\langle var \rangle := \langle intexp \rangle$  in  $\langle comm \rangle$   
| while  $\langle boolexp \rangle$  do  $\langle comm \rangle$ 
```

Un Lenguaje Imperativo Simple

Expresiones

$\langle \text{natconst} \rangle ::= 0 \mid 1 \mid 2 \mid \dots$

$\langle \text{boolconst} \rangle ::= \text{true} \mid \text{false}$

$\langle \text{intexp} \rangle ::= \langle \text{natconst} \rangle$

$\langle \text{boolexp} \rangle ::= \langle \text{boolconst} \rangle$

| $\langle \text{var} \rangle$

| $\neg \langle \text{boolexp} \rangle$

| $- \langle \text{intexp} \rangle$

| $\langle \text{intexp} \rangle \oplus \langle \text{intexp} \rangle$

| $\langle \text{intexp} \rangle \oplus \langle \text{intexp} \rangle$

| $\langle \text{boolexp} \rangle \oslash \langle \text{boolexp} \rangle$

$\oplus \in \{+, -, *, /, \%, \text{rem}\}$

$\oslash \in \{<, \leq, =, \neq, \geq, >\}$

$\oslash \in \{\wedge, \vee, \Rightarrow, \Leftrightarrow\}$

6

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas

7

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas
- Todas las funciones primitivas (incluida la división) son funciones totales.

7

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas
- Todas las funciones primitivas (incluida la división) son funciones totales.
- Como en la lógica de predicados, sólo hay variables enteras:

$$\Sigma = \langle var \rangle \rightarrow \mathbb{Z}$$

7

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas
- Todas las funciones primitivas (incluida la división) son funciones totales.
- Como en la lógica de predicados, sólo hay variables enteras:

$$\Sigma = \langle var \rangle \rightarrow \mathbb{Z}$$

- El significado de una expresión entera (booleana), fijado un estado, es un entero (booleano).

7

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas
- Todas las funciones primitivas (incluida la división) son funciones totales.
- Como en la lógica de predicados, sólo hay variables enteras:

$$\Sigma = \langle var \rangle \rightarrow \mathbb{Z}$$

- El significado de una expresión entera (booleana), fijado un estado, es un entero (booleano).
- El significado de un comando *que termina*, fijado un estado, es un estado.

7

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas
- Todas las funciones primitivas (incluida la división) son funciones totales.
- Como en la lógica de predicados, sólo hay variables enteras:

$$\Sigma = \langle var \rangle \rightarrow \mathbb{Z}$$

- El significado de una expresión entera (booleana), fijado un estado, es un entero (booleano).
- El significado de un comando *que termina*, fijado un estado, es un estado.
- ¿Y el significado de un comando que **NO** termina?

7

Significado de los comandos de LIS

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

*dados un estado
devuelve un $\mathbb{Z}$*

8

Significado de los comandos de LIS

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

$$\llbracket _ \rrbracket^{boolexp} \in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\})$$

8

Significado de los comandos de LIS

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

$$\llbracket _ \rrbracket^{boolexp} \in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\})$$

$$\llbracket _ \rrbracket^{comm} \in \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

8

Significado de los comandos de LIS

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

$$\llbracket _ \rrbracket^{boolexp} \in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\})$$

$$\llbracket _ \rrbracket^{comm} \in \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

Observación: en el codominio de $\llbracket _ \rrbracket^{comm}$ aparece $\Sigma_{\perp}$ para dar cuenta de la no terminación.

8

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

9

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

En nuestro caso, es equivalente a decir que para todo $\sigma \in \Sigma$,

$$\llbracket e \rrbracket \sigma = \llbracket e' \rrbracket \sigma$$

9

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

En nuestro caso, es equivalente a decir que para todo $\sigma \in \Sigma$,

$$\llbracket e \rrbracket \sigma = \llbracket e' \rrbracket \sigma$$

Por ejemplo:

9

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

En nuestro caso, es equivalente a decir que para todo $\sigma \in \Sigma$,

$$\llbracket e \rrbracket \sigma = \llbracket e' \rrbracket \sigma$$

Por ejemplo:

$$\mathbf{true} \equiv x = x$$

9

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

En nuestro caso, es equivalente a decir que para todo $\sigma \in \Sigma$,

$$\llbracket e \rrbracket \sigma = \llbracket e' \rrbracket \sigma$$

Por ejemplo:

$$\mathbf{true} \equiv x = x$$

$$2 * x \equiv x + x$$

9

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

En nuestro caso, es equivalente a decir que para todo $\sigma \in \Sigma$,

$$\llbracket e \rrbracket \sigma = \llbracket e' \rrbracket \sigma$$

Por ejemplo:

$$\text{true} \equiv x = x$$

$$2 * x \equiv x + x$$

$$x := 0; y := 1 \equiv y := 1; x := 0$$

9

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\llbracket \text{skip} \rrbracket \sigma = \sigma$$

10

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma]\end{aligned}$$

10

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases}\end{aligned}$$

10

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\llbracket \text{skip} \rrbracket \sigma = \sigma$$

$$\llbracket v := e \rrbracket \sigma = [\sigma \mid v : \llbracket e \rrbracket \sigma]$$

$$\llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma = \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases}$$

Claramente **if** ... es un caso recursivo, pero no problemático.

10

Ecuaciones Semánticas de Comandos

Composición

La ecuación obvia

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket (\llbracket c_0 \rrbracket \sigma)$$

no tiene en cuenta que c_0 se puede colgar.

10

Ecuaciones Semánticas de Comandos

Composición

La ecuación obvia

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket (\llbracket c_0 \rrbracket \sigma)$$

no tiene en cuenta que c_0 se puede colgar.

De hecho, nuestro dominio semántico está bien pensado porque hay un error de tipos.

10

Ecuaciones Semánticas de Comandos

Composición

La ecuación obvia

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket (\llbracket c_0 \rrbracket \sigma)$$

no tiene en cuenta que c_0 se puede colgar.

De hecho, nuestro dominio semántico está bien pensado porque hay un error de tipos.

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket \perp (\llbracket c_0 \rrbracket \sigma)$$

10

Ecuaciones Semánticas de Comandos

Bloque local

Después de ejecutar el cuerpo hay que restaurar el valor de la variable.

$$\begin{aligned} rest_{v,\sigma} &: \Sigma \rightarrow \Sigma \\ rest_{v,\sigma}(\sigma') &= [\sigma' \mid v : \sigma v] \end{aligned}$$

Acá σ y v son parámetros fijos; el argumento es σ' .

10

Ecuaciones Semánticas de Comandos

Bloque local

Después de ejecutar el cuerpo hay que restaurar el valor de la variable.

$$\begin{aligned} rest_{v,\sigma} &: \Sigma \rightarrow \Sigma \\ rest_{v,\sigma}(\sigma') &= [\sigma' \mid v : \sigma v] \end{aligned}$$

Acá σ y v son parámetros fijos; el argumento es σ' .

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (rest_{v,\sigma})_{\perp}(\llbracket c \rrbracket [\sigma \mid v : \llbracket e \rrbracket \sigma])$$

10

Ecuaciones Semánticas de Comandos

Bloque local

Después de ejecutar el cuerpo hay que restaurar el valor de la variable.

$$\begin{aligned} rest_{v,\sigma} &: \Sigma \rightarrow \Sigma \\ rest_{v,\sigma}(\sigma') &= [\sigma' \mid v : \sigma v] \end{aligned}$$

Acá σ y v son parámetros fijos; el argumento es σ' .

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (rest_{v,\sigma})_{\perp}(\llbracket c \rrbracket [\sigma \mid v : \llbracket e \rrbracket \sigma])$$

Con notación lambda:

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (\text{rest}_{v,\sigma})_{\perp} (\llbracket c \rrbracket [\sigma|v : \llbracket e \rrbracket \sigma])$$

Con notación lambda:

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (\lambda \sigma' \in \Sigma. [\sigma' | v : \sigma v])_{\perp} (\llbracket c \rrbracket [\sigma|v : \llbracket e \rrbracket \sigma])$$

10

Ecuaciones Semánticas de Comandos

Ciclo

Evalúamos la guarda. Si es falsa, se terminó el programa.

Si es verdadera, ejecutamos el cuerpo y luego el ciclo entero.

10

Ecuaciones Semánticas de Comandos

Ciclo

Evalúamos la guarda. Si es falsa, se terminó el programa.

Si es verdadera, ejecutamos el cuerpo y luego el ciclo entero.

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ \llbracket \text{while } b \text{ do } c \rrbracket_{\perp} (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

10

Ecuaciones Semánticas de Comandos

Ciclo

Evaluamos la guarda. Si es falsa, se terminó el programa.
Si es verdadera, ejecutamos el cuerpo y luego el ciclo entero.

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ \llbracket \text{while } b \text{ do } c \rrbracket \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

Problema: no es dirigida por sintaxis. Solución: definir

$$F_{b,c}: (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

y encontrar su menor punto fijo.

10

Ecuaciones Semánticas de Comandos

Ciclo

Evaluamos la guarda. Si es falsa, se terminó el programa.
Si es verdadera, ejecutamos el cuerpo y luego el ciclo entero.

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ \llbracket \text{while } b \text{ do } c \rrbracket \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

Problema: no es dirigida por sintaxis. Solución: definir

$$F_{b,c}: (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

y encontrar su menor punto fijo.

$$F_{b,c} f \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

10

Ecuación semántica para el ciclo

La ecuación

Sea $b \in \langle \text{boolexp} \rangle$ y $c \in \langle \text{comm} \rangle$.

$$\begin{aligned} F_{b,c} &: (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp}) \\ F_{b,c} f \sigma &= \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \end{cases} \end{aligned}$$

Ecuación semántica para el ciclo

La ecuación

Sea $b \in \langle \text{boolexp} \rangle$ y $c \in \langle \text{comm} \rangle$.

$$F_{b,c} : (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

$$F_{b,c} f \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f_{\perp}(\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \left(\bigsqcup_{i \in \mathbb{N}} F^i \perp \right) \sigma$$

Ecuación semántica para el ciclo

La intuición

El elemento k -ésimo de la cadena $F^i \perp$ representa la idea de *desenrollar* k veces el while:

$$F^0 \perp : p_0 = \text{while true do skip}$$

$$F^{i+1} \perp : p_{i+1} = \text{if } b \text{ then } (c; p_i) \text{ else skip}$$

Ecuación semántica para el ciclo

La intuición

El elemento k -ésimo de la cadena $F^i \perp$ representa la idea de *desenrollar* k veces el while:

$$\begin{aligned} F^0 \perp & : p_0 = \text{while true do skip} \\ F^{i+1} \perp & : p_{i+1} = \text{if } b \text{ then } (c; p_i) \text{ else skip} \end{aligned}$$

Si con esas k veces ya llegamos al resultado, la cadena se mantiene estable.

11

Ecuación semántica para el ciclo

La intuición

El elemento k -ésimo de la cadena $F^i \perp$ representa la idea de *desenrollar* k veces el while:

$$\begin{aligned} F^0 \perp & : p_0 = \text{while true do skip} \\ F^{i+1} \perp & : p_{i+1} = \text{if } b \text{ then } (c; p_i) \text{ else skip} \end{aligned}$$

Si con esas k veces ya llegamos al resultado, la cadena se mantiene estable.

Pero si las k veces no fueron suficientes, tenemos un resultado indefinido todavía.

11



06_impera
tivo-teore...

Lenguajes y Compiladores Lenguaje Imperativo: ejemplo y teoremas

Miguel Pagano
12 de abril de 2023

Repaso

Funciones

Extensión estrictas

Si $f: A \rightarrow B$, entonces definimos su *extensión estricta*, $f_{\perp}: A_{\perp} \rightarrow B_{\perp}$

$$f_{\perp} x = \begin{cases} \perp & \text{si } x = \perp \\ f a & \text{si } x = a \end{cases}$$

Funciones

Extensión estrictas

Si $f: A \rightarrow B$, entonces definimos su *extensión estricta*, $f_{\perp}: A_{\perp} \rightarrow B_{\perp}$

$$f_{\perp} x = \begin{cases} \perp & \text{si } x = \perp \\ f a & \text{si } x = a \end{cases}$$

Extensiones estrictas

Si $f: P \rightarrow D$, entonces definimos su *extensión estricta*, $f_{\perp}: P_{\perp} \rightarrow D$

$$f_{\perp} x = \begin{cases} \perp & \text{si } x = \perp \\ f a & \text{si } x = a \end{cases}$$

1

Lenguaje de Programación

Un Lenguaje Imperativo Simple

Comandos

```
 $\langle comm \rangle ::= \text{skip}$   
|  $\langle var \rangle := \langle intexp \rangle$   
|  $\langle comm \rangle ; \langle comm \rangle$   
| if  $\langle boolexp \rangle$  then  $\langle comm \rangle$  else  $\langle comm \rangle$   
| newvar  $\langle var \rangle := \langle intexp \rangle$  in  $\langle comm \rangle$   
| while  $\langle boolexp \rangle$  do  $\langle comm \rangle$ 
```

2

Un Lenguaje Imperativo Simple

Expresiones

$\langle natconst \rangle ::= 0 \mid 1 \mid 2 \mid \dots$	$\langle boolconst \rangle ::= \text{true} \mid \text{false}$
$\langle intexp \rangle ::= \langle natconst \rangle$	$\langle boolexp \rangle ::= \langle boolconst \rangle$
$\mid \langle var \rangle$	$\mid \neg \langle boolexp \rangle$
$\mid - \langle intexp \rangle$	$\mid \langle intexp \rangle \otimes \langle intexp \rangle$
$\mid \langle intexp \rangle \oplus \langle intexp \rangle$	$\mid \langle boolexp \rangle \oslash \langle boolexp \rangle$
$\oplus \in \{+, -, *, /, \%, \text{rem}\}$	$\otimes \in \{<, \leq, =, \neq, \geq, >\}$
	$\oslash \in \{\wedge, \vee, \Rightarrow, \Leftrightarrow\}$

2

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas

3

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas
- Todas las funciones primitivas (incluida la división) son funciones totales.

3

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas
- Todas las funciones primitivas (incluida la división) son funciones totales.
- Como en la lógica de predicados, sólo hay variables enteras:

$$\Sigma = \langle var \rangle \rightarrow \mathbb{Z}$$

3

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas
- Todas las funciones primitivas (incluida la división) son funciones totales.
- Como en la lógica de predicados, sólo hay variables enteras:

$$\Sigma = \langle var \rangle \rightarrow \mathbb{Z}$$

- El significado de una expresión entera (booleana), fijado un estado, es un entero (booleano).

3

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas
- Todas las funciones primitivas (incluida la división) son funciones totales.
- Como en la lógica de predicados, sólo hay variables enteras:

$$\Sigma = \langle var \rangle \rightarrow \mathbb{Z}$$

- El significado de una expresión entera (booleana), fijado un estado, es un entero (booleano).
- El significado de un comando *que termina*, fijado un estado, es un estado.

3

Características de LIS

- Hay sólo dos tipos de expresiones: enteras y booleanas
- Todas las funciones primitivas (incluida la división) son funciones totales.
- Como en la lógica de predicados, sólo hay variables enteras:

$$\Sigma = \langle var \rangle \rightarrow \mathbb{Z}$$

- El significado de una expresión entera (booleana), fijado un estado, es un entero (booleano).
- El significado de un comando **que termina**, fijado un estado, es un estado.
- ¿Y el significado de un comando que **NO** termina?

3

Significado de los comandos de LIS

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

4

Significado de los comandos de LIS

Funciones semánticas

$$\begin{aligned} \llbracket _ \rrbracket^{intexp} &\in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z}) \\ \llbracket _ \rrbracket^{boolexp} &\in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\}) \end{aligned}$$

4

Significado de los comandos de LIS

Funciones semánticas

$$\begin{aligned}\llbracket _ \rrbracket^{intexp} &\in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z}) \\ \llbracket _ \rrbracket^{boolexp} &\in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\}) \\ \llbracket _ \rrbracket^{comm} &\in \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma_{\perp})\end{aligned}$$

4

Significado de los comandos de LIS

Funciones semánticas

$$\begin{aligned}\llbracket _ \rrbracket^{intexp} &\in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z}) \\ \llbracket _ \rrbracket^{boolexp} &\in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\}) \\ \llbracket _ \rrbracket^{comm} &\in \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma_{\perp})\end{aligned}$$

Observación: en el codominio de $\llbracket _ \rrbracket^{comm}$ aparece $\Sigma_{\perp}$ para dar cuenta de la no terminación.

4

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

5

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

En nuestro caso, es equivalente a decir que para todo $\sigma \in \Sigma$,

$$\llbracket e \rrbracket \sigma = \llbracket e' \rrbracket \sigma$$

5

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

En nuestro caso, es equivalente a decir que para todo $\sigma \in \Sigma$,

$$\llbracket e \rrbracket \sigma = \llbracket e' \rrbracket \sigma$$

Por ejemplo:

5

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

En nuestro caso, es equivalente a decir que para todo $\sigma \in \Sigma$,

$$\llbracket e \rrbracket \sigma = \llbracket e' \rrbracket \sigma$$

Por ejemplo:

$$\text{true} \equiv x = x$$

5

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

En nuestro caso, es equivalente a decir que para todo $\sigma \in \Sigma$,

$$\llbracket e \rrbracket \sigma = \llbracket e' \rrbracket \sigma$$

Por ejemplo:

$$\mathbf{true} \equiv x = x$$

$$2 * x \equiv x + x$$

5

Equivalencia de Expresiones y Comandos

Equivalencia de términos

Dados e, e' en la misma categoría sintáctica, se dice que e es equivalente a e' , que escribimos como $e \equiv e'$, si y solo si

$$\llbracket e \rrbracket = \llbracket e' \rrbracket$$

En nuestro caso, es equivalente a decir que para todo $\sigma \in \Sigma$,

$$\llbracket e \rrbracket \sigma = \llbracket e' \rrbracket \sigma$$

Por ejemplo:

$$\mathbf{true} \equiv x = x$$

$$2 * x \equiv x + x$$

$$x := 0; y := 1 \equiv y := 1; x := 0$$

5

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\llbracket \mathbf{skip} \rrbracket \sigma = \sigma$$

6

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma]\end{aligned}$$

6

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases}\end{aligned}$$

6

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases}\end{aligned}$$

Claramente **if...** es un caso recursivo, pero no problemático.

6

Ecuaciones Semánticas de Comandos

Composición

La ecuación obvia

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket (\llbracket c_0 \rrbracket \sigma)$$

no tiene en cuenta que c_0 se puede colgar.

6

Ecuaciones Semánticas de Comandos

Composición

La ecuación obvia

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket (\llbracket c_0 \rrbracket \sigma)$$

no tiene en cuenta que c_0 se puede colgar.

De hecho, nuestro dominio semántico está bien pensado porque hay un error de tipos.

6

Ecuaciones Semánticas de Comandos

Composición

La ecuación obvia

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket (\llbracket c_0 \rrbracket \sigma)$$

no tiene en cuenta que c_0 se puede colgar.

De hecho, nuestro dominio semántico está bien pensado porque hay un error de tipos.

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket \perp (\llbracket c_0 \rrbracket \sigma)$$

6

Ecuaciones Semánticas de Comandos

Bloque local

Después de ejecutar el cuerpo hay que restaurar el valor de la variable.

$$\begin{aligned} rest_{v,\sigma} &: \Sigma \rightarrow \Sigma \\ rest_{v,\sigma}(\sigma') &= [\sigma' \mid v : \sigma v] \end{aligned}$$

Acá σ y v son parámetros fijos; el argumento es σ' .

6

Ecuaciones Semánticas de Comandos

Bloque local

Después de ejecutar el cuerpo hay que restaurar el valor de la variable.

$$\begin{aligned} rest_{v,\sigma} &: \Sigma \rightarrow \Sigma \\ rest_{v,\sigma}(\sigma') &= [\sigma' \mid v : \sigma v] \end{aligned}$$

Acá σ y v son parámetros fijos; el argumento es σ' .

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (rest_{v,\sigma})_{\perp} (\llbracket c \rrbracket [\sigma \mid v : \llbracket e \rrbracket \sigma])$$

6

Ecuaciones Semánticas de Comandos

Bloque local

Después de ejecutar el cuerpo hay que restaurar el valor de la variable.

$$\begin{aligned} rest_{v,\sigma} &: \Sigma \rightarrow \Sigma \\ rest_{v,\sigma}(\sigma') &= [\sigma' \mid v : \sigma v] \end{aligned}$$

Acá σ y v son parámetros fijos; el argumento es σ' .

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (rest_{v,\sigma})_{\perp} (\llbracket c \rrbracket [\sigma \mid v : \llbracket e \rrbracket \sigma])$$

Con notación lambda:

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (\lambda \sigma' \in \Sigma. [\sigma' \mid v : \sigma v])_{\perp} (\llbracket c \rrbracket [\sigma \mid v : \llbracket e \rrbracket \sigma])$$

6

Ecuaciones Semánticas de Comandos

Ciclo

Evaluamos la guarda. Si es falsa, se terminó el programa.
Si es verdadera, ejecutamos el cuerpo y luego el ciclo entero.

6

Ecuaciones Semánticas de Comandos

Ciclo

Evaluamos la guarda. Si es falsa, se terminó el programa.
Si es verdadera, ejecutamos el cuerpo y luego el ciclo entero.

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ \llbracket \text{while } b \text{ do } c \rrbracket \perp (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

6

Ecuaciones Semánticas de Comandos

Ciclo

Evaluamos la guarda. Si es falsa, se terminó el programa.
Si es verdadera, ejecutamos el cuerpo y luego el ciclo entero.

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ \llbracket \text{while } b \text{ do } c \rrbracket \perp (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

Problema: no es dirigida por sintaxis. Solución: definir

$$F_{b,c}: (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

y encontrar su menor punto fijo.

6

Ecuaciones Semánticas de Comandos

Ciclo

Evaluamos la guarda. Si es falsa, se terminó el programa.
Si es verdadera, ejecutamos el cuerpo y luego el ciclo entero.

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ \llbracket \text{while } b \text{ do } c \rrbracket \perp (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

Problema: no es dirigida por sintaxis. Solución: definir

$$F_{b,c}: (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

y encontrar su menor punto fijo.

$$F_{b,c} f \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f(\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

todo lo mismo que el anterior

y encontrar su menor punto fijo.

$$F_{b,c} f \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f_{\perp}(\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

6



Ejemplo: Exponenciación

Exponenciación: Enunciado

Especificación

Quiero un programa c que calcule x^y , asumiendo que $y \geq 0$.

7

Exponenciación: Enunciado

Especificación

Quiero un programa c que calcule x^y , asumiendo que $y \geq 0$.

Volviendo a primer año:

```
{Y ≥ 0}  
c  
{e = XY}
```

7

Exponenciación: Enunciado

Especificación

Quiero un programa c que calcule x^y , asumiendo que $y \geq 0$.

Volviendo a primer año:

$$\{Y \geq 0\}$$

c

$$\{e = X^Y\}$$

Volviendo a quinto $\llbracket c \rrbracket \sigma e = \sigma x^{\sigma y}$ si $\sigma y \geq 0$.

7

Exponenciación: Programa

Solución obvia

$e := 1; \underbrace{\text{while } y > 0}_{b} \text{ do } \underbrace{e := e * x; y := y - 1}_{body}$

8

Exponenciación: Programa

Solución obvia

$e := 1; \underbrace{\text{while } y > 0}_{b} \text{ do } \underbrace{e := e * x; y := y - 1}_{body}$

Cálculo de semántica

- $\llbracket e := 1 \rrbracket \sigma$.

8

Exponenciación: Programa

Solución obvia

$$e := 1; \underbrace{\text{while } y > 0}_{b} \text{ do } \underbrace{e := e * x; y := y - 1}_{body}$$

Cálculo de semántica

- $\llbracket e := 1 \rrbracket \sigma.$
- $\llbracket body \rrbracket \sigma.$

8

Exponenciación: Programa

Solución obvia

$$e := 1; \underbrace{\text{while } y > 0}_{b} \text{ do } \underbrace{e := e * x; y := y - 1}_{body}$$

Cálculo de semántica

- $\llbracket e := 1 \rrbracket \sigma.$
- $\llbracket body \rrbracket \sigma.$
- $F_{b, body}^i \perp \sigma.$

8

Exponenciación: Programa

Solución obvia

$$e := 1; \underbrace{\text{while } y > 0}_{b} \text{ do } \underbrace{e := e * x; y := y - 1}_{body}$$

Cálculo de semántica

- $\llbracket e := 1 \rrbracket \sigma.$
- $\llbracket body \rrbracket \sigma.$
- $F_{b, body}^i \perp \sigma.$
- $\llbracket \text{while } b \text{ do } body \rrbracket \sigma.$

8

Resumen

Cálculo de semántica de ciclos

1. Hallar la expresión más sencilla posible para $body$.

9

Resumen

Cálculo de semántica de ciclos

1. Hallar la expresión más sencilla posible para $body$.
2. Hallar la expresión más sencilla posible para $F_{b,body} f \sigma$.

9

Resumen

Cálculo de semántica de ciclos

1. Hallar la expresión más sencilla posible para $body$.
2. Hallar la expresión más sencilla posible para $F_{b,body} f \sigma$.
3. Calcular los primeros elementos de la cadena $F^i \perp \sigma$.

9

Recapitulación

Cálculo de semántica de ciclos

1. Hallar la expresión más sencilla posible para $body$.
2. Hallar la expresión más sencilla posible para $F_{b,body} f \sigma$.
3. Calcular los primeros elementos de la cadena $F^i \perp \sigma$.
4. Hallar una expresión general para el supremo.

9

Teorema de Coincidencia

Expresiones y comandos

Para expresiones

Si $\sigma w = \sigma' w$ para toda $w \in FV(e)$, entonces $\llbracket e \rrbracket \sigma = \llbracket e \rrbracket \sigma'$.

Para comandos

¿Si $\sigma w = \sigma' w$ para toda $w \in FV(c)$, entonces $\llbracket c \rrbracket \sigma = \llbracket c \rrbracket \sigma'$?

10

Expresiones y comandos

Para expresiones

Si $\sigma w = \sigma' w$ para toda $w \in FV(e)$, entonces $\llbracket e \rrbracket \sigma = \llbracket e \rrbracket \sigma'$.

Para comandos

¿Si $\sigma w = \sigma' w$ para toda $w \in FV(c)$, entonces $\llbracket c \rrbracket \sigma = \llbracket c \rrbracket \sigma'$?

Sea $c \doteq x := 5 + w$.

Sea $\sigma_0 u = 0$. Ahora consideremos

$\sigma = [\sigma_0 \mid x : 3 \mid w : 4 \mid z : 3]$ y

$\sigma' = [\sigma_0 \mid x : 3 \mid w : 4]$.

10

Variables libres y Variables asignables

Variables libres

$$FV(\text{skip}) = \emptyset$$

11

Variables libres y Variables asignables

Variables libres

$$FV(\text{skip}) = \emptyset$$

$$FV(v := e) = \{v\} \cup FV(e)$$

11

Variables libres y Variables asignables

Variables libres

$$\begin{aligned} FV(\text{skip}) &= \emptyset \\ FV(v := e) &= \{v\} \cup FV(e) \\ FV(c_0; c_1) &= FV(c_0) \cup FV(c_1) \end{aligned}$$

11

Variables libres y Variables asignables

Variables libres

$$\begin{aligned} FV(\text{skip}) &= \emptyset \\ FV(v := e) &= \{v\} \cup FV(e) \\ FV(c_0; c_1) &= FV(c_0) \cup FV(c_1) \\ FV(\text{if } b \text{ then } c_0 \text{ else } c_1) &= FV(b) \cup FV(c_0) \cup FV(c_1) \end{aligned}$$

11

Variables libres y Variables asignables

Variables libres

$$\begin{aligned} FV(\text{skip}) &= \emptyset \\ FV(v := e) &= \{v\} \cup FV(e) \\ FV(c_0; c_1) &= FV(c_0) \cup FV(c_1) \\ FV(\text{if } b \text{ then } c_0 \text{ else } c_1) &= FV(b) \cup FV(c_0) \cup FV(c_1) \\ FV(\text{while } b \text{ do } c) &= FV(b) \cup FV(c) \end{aligned}$$

11

Variables libres y Variables asignables

Variables libres

$$\begin{aligned}FV(\text{skip}) &= \emptyset \\FV(v := e) &= \{v\} \cup FV(e) \\FV(c_0; c_1) &= FV(c_0) \cup FV(c_1) \\FV(\text{if } b \text{ then } c_0 \text{ else } c_1) &= FV(b) \cup FV(c_0) \cup FV(c_1) \\FV(\text{while } b \text{ do } c) &= FV(b) \cup FV(c) \\FV(\text{newvar } v := e \text{ in } c) &= FV(e) \cup (FV(c) - \{v\})\end{aligned}$$

11

Variables libres y Variables asignables

Variables Asignables

$$FA(\text{skip}) = \emptyset$$

11

Variables libres y Variables asignables

Variables Asignables

$$\begin{aligned}FA(\text{skip}) &= \emptyset \\FA(v := e) &= \{v\}\end{aligned}$$

11

Variables libres y Variables asignables

Variables Asignables

$$\begin{aligned}FA(\text{skip}) &= \emptyset \\FA(v := e) &= \{v\} \\FA(c_0; c_1) &= FA(c_0) \cup FA(c_1)\end{aligned}$$

11

Variables libres y Variables asignables

Variables Asignables

$$\begin{aligned}FA(\text{skip}) &= \emptyset \\FA(v := e) &= \{v\} \\FA(c_0; c_1) &= FA(c_0) \cup FA(c_1) \\FA(\text{if } b \text{ then } c_0 \text{ else } c_1) &= FA(c_0) \cup FA(c_1)\end{aligned}$$

11

Variables libres y Variables asignables

Variables Asignables

$$\begin{aligned}FA(\text{skip}) &= \emptyset \\FA(v := e) &= \{v\} \\FA(c_0; c_1) &= FA(c_0) \cup FA(c_1) \\FA(\text{if } b \text{ then } c_0 \text{ else } c_1) &= FA(c_0) \cup FA(c_1) \\FA(\text{while } b \text{ do } c) &= FA(c)\end{aligned}$$

11

Variables libres y Variables asignables

Variables Asignables

$FA(\text{skip})$	$= \emptyset$
$FA(v := e)$	$= \{v\}$
$FA(c_0; c_1)$	$= FA(c_0) \cup FA(c_1)$
$FA(\text{if } b \text{ then } c_0 \text{ else } c_1)$	$= FA(c_0) \cup FA(c_1)$
$FA(\text{while } b \text{ do } c)$	$= FA(c)$
$FA(\text{newvar } v := e \text{ in } c)$	$= FA(c) - \{v\}$

11

Variables libres y Variables asignables

Variables Asignables

$FA(\text{skip})$	$= \emptyset$
$FA(\bar{v} := e)$	$= \{\bar{v}\}$
$FA(c_0; c_1)$	$= FA(c_0) \cup FA(c_1)$
$FA(\text{if } b \text{ then } c_0 \text{ else } c_1)$	$= FA(c_0) \cup FA(c_1)$
$FA(\text{while } b \text{ do } c)$	$= FA(c)$
$FA(\text{newvar } \bar{v} := e \text{ in } c)$	$= FA(c) - \{\bar{v}\}$

11

Enunciado correcto

Lema de intangibilidad

Si $\llbracket c \rrbracket \sigma = \sigma_1 \neq \perp$, entonces $\sigma_1 v = \sigma v$ para toda $v \notin FA(c)$.

12

Enunciado correcto

Lema de intangibilidad

Si $\llbracket c \rrbracket \sigma = \sigma_1 \neq \perp$, entonces $\sigma_1 v = \sigma v$ para toda $v \notin FA(c)$.

Teorema de coincidencia

Si $\sigma w = \sigma' w$ para toda $w \in FV(c)$, entonces:

1. o bien $\llbracket c \rrbracket \sigma = \perp = \llbracket c \rrbracket \sigma'$,
2. o bien $\llbracket c \rrbracket \sigma = \sigma_1 \neq \perp$ y $\llbracket c \rrbracket \sigma' = \sigma_2 \neq \perp$, y para toda $w \in FV(c)$, $\sigma_1 w = \sigma_2 w$.

12

Esquema de prueba

1. Inducción estructural en c .

13

Esquema de prueba

1. Inducción estructural en c .
2. Los casos que no son ciclos son fáciles.

13

Esquema de prueba

1. Inducción estructural en c .
2. Los casos que no son ciclos son fáciles.
3. Para el ciclo **while** b **do** cw enunciamos un lema auxiliar para $F_{b,cw}^i \perp$ que probamos por inducción en i .

13

Esquema de prueba

1. Inducción estructural en c .
2. Los casos que **no son ciclos son fáciles**.
3. Para el ciclo **while** b **do** cw enunciamos un lema auxiliar para $F_{b,cw}^i \perp$ que probamos por inducción en i .
4. Como es una inducción en los naturales dentro de la inducción de comandos podemos usar la hipótesis inductiva para cw y también tenemos la hipótesis inductiva para el natural.

13

Sustituciones

Expresiones y comandos

Para expresiones

Si $\sigma w = \llbracket \delta w \rrbracket \sigma'$ para toda $w \in FV(e)$, entonces $\llbracket e \rrbracket \sigma = \llbracket e/\delta \rrbracket \sigma'$.

Para comandos

¿ Si $\sigma w = \llbracket \delta w \rrbracket \sigma'$ para toda $w \in FV(c)$, entonces $\llbracket c \rrbracket \sigma = \llbracket c/\delta \rrbracket \sigma'$?

14

Expresiones y comandos

Para expresiones

Si $\sigma w = \llbracket \delta w \rrbracket \sigma'$ para toda $w \in FV(e)$, entonces $\llbracket e \rrbracket \sigma = \llbracket e/\delta \rrbracket \sigma'$.

Para comandos

¿ Si $\sigma w = \llbracket \delta w \rrbracket \sigma'$ para toda $w \in FV(c)$, entonces $\llbracket c \rrbracket \sigma = \llbracket c/\delta \rrbracket \sigma'$?

Sea $c \doteq x := 5 + w$.

Consideremos la siguiente sustitución: $\delta w = \begin{cases} y + 1 & \text{si } w = x \\ w & \text{si } w \neq x \end{cases}$

14

Expresiones y comandos

Para expresiones

Si $\sigma w = \llbracket \delta w \rrbracket \sigma'$ para toda $w \in FV(e)$, entonces $\llbracket e \rrbracket \sigma = \llbracket e/\delta \rrbracket \sigma'$.

Para comandos

¿ Si $\sigma w = \llbracket \delta w \rrbracket \sigma'$ para toda $w \in FV(c)$, entonces $\llbracket c \rrbracket \sigma = \llbracket c/\delta \rrbracket \sigma'$?

Sea $c \doteq x := 5 + w$.

Consideremos la siguiente sustitución: $\delta w = \begin{cases} y + 1 & \text{si } w = x \\ w & \text{si } w \neq x \end{cases}$

Entonces $c/\delta = y + 1 := 5 + w$!

14

Sustituciones más razonables

Renombres

Una sustitución $\delta: \langle var \rangle \rightarrow \langle intexp \rangle$ es un **renombre** si para toda $v \in \langle var \rangle$, $\delta v = w$ para alguna $w \in \langle var \rangle$.

Es decir, un renombre es una función $\delta: \langle var \rangle \rightarrow \langle var \rangle$.

Enunciado del teorema

Sean $\delta: \langle var \rangle \rightarrow \langle var \rangle$ y $\sigma, \sigma': \Sigma$. Si para todo $w \in FV(c)$, $\sigma'(\delta w) = \sigma w$, entonces

1. o bien $\llbracket c \rrbracket \sigma = \perp = \llbracket c / \delta \rrbracket \sigma'$.
2. o bien $\llbracket c \rrbracket \sigma = \sigma_1$, $\llbracket c / \delta \rrbracket \sigma' = \sigma_2$ y para toda $w \in FV(c)$, $\sigma_1 w = \sigma_2 \square$.

¿Qué debe ir en el espacio en blanco?

15

Sustituciones más razonables

Renombres

Una sustitución $\delta: \langle var \rangle \rightarrow \langle intexp \rangle$ es un **renombre** si para toda $v \in \langle var \rangle$, $\delta v = w$ para alguna $w \in \langle var \rangle$.

Es decir, un renombre es una función $\delta: \langle var \rangle \rightarrow \langle var \rangle$.

Enunciado del teorema

Sean $\delta: \langle var \rangle \rightarrow \langle var \rangle$ y $\sigma, \sigma': \Sigma$. Si para todo $w \in FV(c)$, $\sigma'(\delta w) = \sigma w$, entonces

1. o bien $\llbracket c \rrbracket \sigma = \perp = \llbracket c / \delta \rrbracket \sigma'$.
2. o bien $\llbracket c \rrbracket \sigma = \sigma_1$, $\llbracket c / \delta \rrbracket \sigma' = \sigma_2$ y para toda $w \in FV(c)$, $\sigma_1 w = \sigma_2(\delta w)$.

15

Sustitución en Comandos

Definición de sustitución

$\text{skip} / \delta = \text{skip}$

16

Sustitución en Comandos

Definición de sustitución

skip / δ	=	skip
$v := e$ / δ	=	$\delta v := e$ / δ

16

Sustitución en Comandos

Definición de sustitución

skip / δ	=	skip
$v := e$ / δ	=	$\delta v := e$ / δ
$(c_0; c_1)$ / δ	=	$c_0 / \delta; c_1 / \delta$

16

Sustitución en Comandos

Definición de sustitución

skip / δ	=	skip
$v := e$ / δ	=	$\delta v := e$ / δ
$(c_0; c_1)$ / δ	=	$c_0 / \delta; c_1 / \delta$
$(\text{if } b \text{ then } c_0 \text{ else } c_1)$ / δ	=	$\text{if } b / \delta \text{ then } c_0 / \delta \text{ else } c_1 / \delta$

16

Sustitución en Comandos

Definición de sustitución

skip / δ	=	skip
$v := e$ / δ	=	$\delta v := e$ / δ
$(c_0; c_1)$ / δ	=	$c_0 / \delta; c_1 / \delta$
(if b then c_0 else c_1) / δ	=	if b / δ then c_0 / δ else c_1 / δ
(while b do c) / δ	=	while (b / δ) do (c_0 / δ)

16

Sustitución en Comandos

Definición de sustitución

skip / δ	=	skip
$v := e$ / δ	=	$\delta v := e$ / δ
$(c_0; c_1)$ / δ	=	$c_0 / \delta; c_1 / \delta$
(if b then c_0 else c_1) / δ	=	if b / δ then c_0 / δ else c_1 / δ
(while b do c) / δ	=	while (b / δ) do (c_0 / δ)
(newvar v := e in c) / δ	=	newvar $v_{new} := e / \delta$ in $c / [\delta v : v_{new}]$

donde $v_{new} \notin \{\delta w \mid w \in FV(c) - \{v\}\}$

16

Problema de alias

Consideremos el programa

$$c \doteq x := x + 1; y := y * 2$$

Y el renombre $\delta x = \delta y = z$. Entonces

$$c / \delta = z := z + 1; z := z * 2$$

17

Problema de alias

Consideremos el programa

$$c \doteq x := x + 1; y := y * 2$$

Y el renombre $\delta x = \delta y = z$. Entonces

$$c / \delta = z := z + 1; z := z * 2$$

Sea σ' un estado tal que $\sigma' z = 2$ y sea $\sigma x = \sigma y = 2$.

17

Problema de alias

Consideremos el programa

$$c \doteq x := x + 1; y := y * 2$$

Y el renombre $\delta x = \delta y = z$. Entonces

$$c / \delta = z := z + 1; z := z * 2$$

Sea σ' un estado tal que $\sigma' z = 2$ y sea $\sigma x = \sigma y = 2$.

Pero ni siquiera vale lo que esperamos que valga.

17

Teorema de Sustitución

Sean $\delta: \langle var \rangle \rightarrow \langle var \rangle$ y $\sigma, \sigma': \Sigma$. Si δ es *inyectiva* y para todo $w \in FV(c)$, $\sigma'(\delta w) = \sigma w$,

1. o bien $\llbracket c \rrbracket \sigma = \perp = \llbracket c / \delta \rrbracket \sigma'$.
2. o bien $\llbracket c \rrbracket \sigma = \sigma_1$, $\llbracket c / \delta \rrbracket \sigma' = \sigma_2$ y
para toda $w \in FV(c)$, $\sigma_1 w = \sigma_2(\delta w)$.

18

Teorema de Sustitución. Demostración.

La misma receta que el teorema de coincidencia.

19

Agregando cosas al lenguaje

Repetición acotada

¿Cómo añadir un comando que nos permita ejecutar un cuerpo una cierta cantidad de veces?

Python

```
v, w = 10, 5
for i in range(v+5, w*4):
    c = c + i
```

Javascript

```
var w, v, i;
[v, w] = [10, 5];
for (i = v+5; i < w*4; i++) {
    c = c + i;
}
```

20

1. Construcción sintáctica: nuevo comando real o syntactic-sugar?
2. Qué versión queremos?
3. Ecuación semántica



imperativo
-fallas

Lenguajes y Compiladores

Matías Steinberg y Miguel Pagano
5 de abril de 2023

Repaso

Un Lenguaje Imperativo Simple

Comandos

$\langle comm \rangle ::= \text{skip}$
| $\langle var \rangle := \langle intexp \rangle$
| $\langle comm \rangle ; \langle comm \rangle$
| **if** $\langle boolexp \rangle$ **then** $\langle comm \rangle$ **else** $\langle comm \rangle$
| **newvar** $\langle var \rangle := \langle intexp \rangle$ **in** $\langle comm \rangle$
| **while** $\langle boolexp \rangle$ **do** $\langle comm \rangle$

1

Un Lenguaje Imperativo Simple

Expresiones

$\langle natconst \rangle ::= 0 \mid 1 \mid 2 \mid \dots$	$\langle boolconst \rangle ::= \text{true} \mid \text{false}$
$\langle intexp \rangle ::= \langle natconst \rangle$	$\langle boolexp \rangle ::= \langle boolconst \rangle$
$\langle var \rangle$	$\neg \langle boolexp \rangle$
$- \langle intexp \rangle$	$\langle boolexp \rangle \otimes \langle boolexp \rangle$
$\langle intexp \rangle \oplus \langle intexp \rangle$	$\langle boolexp \rangle \oslash \langle boolexp \rangle$
$\oplus \in \{+, -, *, /, \%, \text{rem}\}$	
$\otimes \in \{<, \leq, =, \neq, \geq, >\}$	
$\oslash \in \{\wedge, \vee, \Rightarrow, \Leftrightarrow\}$	

1

Significado de los comandos de LIS

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

2

Significado de los comandos de LIS

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

$$\llbracket _ \rrbracket^{boolexp} \in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\})$$

2

Significado de los comandos de LIS

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

$$\llbracket _ \rrbracket^{boolexp} \in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\})$$

$$\llbracket _ \rrbracket^{comm} \in \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

2

Significado de los comandos de LIS

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

$$\llbracket _ \rrbracket^{boolexp} \in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\})$$

$$\llbracket _ \rrbracket^{comm} \in \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

2

Significado de los comandos de LIS

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

$$\llbracket _ \rrbracket^{boolexp} \in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\})$$

$$\llbracket _ \rrbracket^{comm} \in \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

Observación: en el codominio de $\llbracket _ \rrbracket^{comm}$ aparece $\Sigma_{\perp}$ para dar cuenta de la no terminación.

2

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\llbracket \text{skip} \rrbracket \sigma = \sigma$$

3

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma]\end{aligned}$$

3

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases}\end{aligned}$$

3

Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases} \\ \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket \perp (\llbracket c_0 \rrbracket \sigma)\end{aligned}$$

3

Ecuaciones Semánticas de Comandos

Bloque local

Después de ejecutar el cuerpo hay que restaurar el valor de la variable.

4

Ecuaciones Semánticas de Comandos

Bloque local

Después de ejecutar el cuerpo hay que restaurar el valor de la variable.

$$\begin{aligned} rest_{v,\sigma} &: \Sigma \rightarrow \Sigma \\ rest_{v,\sigma}(\sigma') &= [\sigma' \mid v : \sigma v] \end{aligned}$$

Acá σ y v son parámetros fijos; el argumento es σ' .

4

Ecuaciones Semánticas de Comandos

Bloque local

Después de ejecutar el cuerpo hay que restaurar el valor de la variable.

$$\begin{aligned} rest_{v,\sigma} &: \Sigma \rightarrow \Sigma \\ rest_{v,\sigma}(\sigma') &= [\sigma' \mid v : \sigma v] \end{aligned}$$

Acá σ y v son parámetros fijos; el argumento es σ' .

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (rest_{v,\sigma})_{\perp}(\llbracket c \rrbracket [\sigma \mid v : \llbracket e \rrbracket \sigma])$$

4

Ecuaciones Semánticas de Comandos

La ecuación

Sea $b \in \langle \text{boolexp} \rangle$ y $c \in \langle \text{comm} \rangle$.

$$F_{b,c} : (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$
$$F_{b,c} f \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f_{\perp}(\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

5

Ecuaciones Semánticas de Comandos

La ecuación

Sea $b \in \langle \text{boolexp} \rangle$ y $c \in \langle \text{comm} \rangle$.

$$F_{b,c} : (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$
$$F_{b,c} f \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f_{\perp}(\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \left(\bigsqcup_{i \in \mathbb{N}} F^i \perp \right) \sigma$$

5

Fallas

Falla

Extender el lenguaje con fallas (de hecho una única falla).

```
if n == 0:  
    raise Exception("acá no dividimos por 0")  
else:  
    m = e / n
```

- ¿Cómo extendemos el lenguaje?

Falla

Extender el lenguaje con fallas (de hecho una única falla).

```
if n == 0:
    raise Exception("acá no dividimos por 0")
else:
    m = e / n
```

- ¿Cómo extendemos el lenguaje?
- Lo comparemos con **for** $v := e_0$ **to** e_1 **do** c .

6

Falla

Extender el lenguaje con fallas (de hecho una única falla).

```
if n == 0:
    raise Exception("acá no dividimos por 0")
else:
    m = e / n
```

- ¿Cómo extendemos el lenguaje?
- Lo comparemos con **for** $v := e_0$ **to** e_1 **do** c .
- ¿Se necesitan los mismos cambios en la semántica?

6

Falla

Extender el lenguaje con fallas (de hecho una única falla).

```
if n == 0:  
    raise Exception("acá no dividimos por 0")  
else:  
    m = e / n
```

- ¿Cómo extendemos el lenguaje?
- Lo comparemos con **for** $v := e_0$ **to** e_1 **do** c .
- ¿Se necesitan los mismos cambios en la semántica?
- ¿Qué otros cambios necesitaríamos si quisiéramos distintos errores?

6

Extensión de lenguaje y de semántica

Sintaxis

$\langle comm \rangle ::= \dots \mid \text{fail}$

7

Extensión de lenguaje y de semántica

Sintaxis

$\langle comm \rangle ::= \dots \mid \text{fail}$

Semántica

Estados correctos (sin ninguna etiqueta) o estados erróneos.

$$\hat{\Sigma} = \Sigma \cup (\{\text{abort}\} \times \Sigma) \quad \Sigma' = (\hat{\Sigma})_{\perp}$$

7

Extensión de lenguaje y de semántica

Sintaxis

$\langle comm \rangle ::= \dots \mid \text{fail}$

Semántica

Estados correctos (sin ninguna etiqueta) o estados erróneos.

$$\hat{\Sigma} = \Sigma \cup (\{\text{abort}\} \times \Sigma) \quad \Sigma' = (\hat{\Sigma})_{\perp}$$

Nueva semántica de programas:

$$\llbracket _ \rrbracket : \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma')$$

7

Extensión de lenguaje y de semántica

Sintaxis

$\langle comm \rangle ::= \dots \mid \text{fail}$

Semántica

Estados correctos (sin ninguna etiqueta) o estados erróneos.

$$\hat{\Sigma} = \Sigma \cup (\{\text{abort}\} \times \Sigma) \quad \Sigma' = (\hat{\Sigma})_{\perp}$$

Nueva semántica de programas:

$$\llbracket _ \rrbracket : \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma')$$

$$\llbracket \text{fail} \rrbracket \sigma = \langle \text{abort}, \sigma \rangle$$

$\llbracket \text{fail} \rrbracket \sigma = \langle \text{abort}, \sigma \rangle$

7

Revisando las Ecuaciones Semánticas de Comandos

Comandos sencillos

$\llbracket \text{skip} \rrbracket \sigma = \sigma$

8

Revisando las Ecuaciones Semánticas de Comandos

Comandos sencillos

$\llbracket \text{skip} \rrbracket \sigma = \sigma$

$\llbracket v := e \rrbracket \sigma = [\sigma \mid v : \llbracket e \rrbracket \sigma]$

8

Revisando las Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases}\end{aligned}$$

8

Revisando las Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases} \\ \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket \text{.ll}(\llbracket c_0 \rrbracket \sigma)\end{aligned}$$

8

Revisando las Ecuaciones Semánticas de Comandos

Comandos sencillos

$$\begin{aligned}
 \llbracket \text{skip} \rrbracket \sigma &= \sigma \\
 \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\
 \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases} \\
 \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket \llbracket c_0 \rrbracket \sigma \\
 \llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma &= (\text{rest}_{v, \sigma}) \llbracket c \rrbracket [\sigma \mid v : \llbracket e \rrbracket \sigma]
 \end{aligned}$$

problememes

8

Funciones de propagación de errores

Propagación de errores

$$\begin{aligned}
 (_)_* : (\Sigma \rightarrow \Sigma') &\rightarrow (\Sigma' \rightarrow \Sigma') \\
 f_*(\perp) &= \perp \\
 f_*(\langle \text{abort}, \sigma \rangle) &= \langle \text{abort}, \sigma \rangle \\
 f_*(\sigma) &= f(\sigma)
 \end{aligned}$$

9

Funciones de propagación de errores

Propagación de errores

$$\begin{aligned}
 (_)_* : (\Sigma \rightarrow \Sigma') &\rightarrow (\Sigma' \rightarrow \Sigma') \\
 f_*(\perp) &= \perp \\
 f_*(\langle \text{abort}, \sigma \rangle) &= \langle \text{abort}, \sigma \rangle \\
 f_*(\sigma) &= f(\sigma)
 \end{aligned}$$

la versión de $\perp$

Como map para listas

$$\begin{aligned}
 (_)_{\dagger} : (\Sigma \rightarrow \Sigma) &\rightarrow (\Sigma' \rightarrow \Sigma') \\
 f_{\dagger}(\perp) &= \perp \\
 f_{\dagger}(\langle \text{abort}, \sigma \rangle) &= \langle \text{abort}, f(\sigma) \rangle
 \end{aligned}$$

la versión de $\perp$

$$\begin{aligned}
 f_{\dagger}(\perp) &= \perp \\
 f_{\dagger}(\langle \mathbf{abort}, \sigma \rangle) &= \langle \mathbf{abort}, f(\sigma) \rangle \\
 f_{\dagger}(\sigma) &= f(\sigma)
 \end{aligned}$$

9

Revisando las ecuaciones

Comandos sencillos

$$\llbracket \mathbf{skip} \rrbracket \sigma = \sigma$$

10

Revisando las ecuaciones

Comandos sencillos

$$\begin{aligned}
 \llbracket \mathbf{skip} \rrbracket \sigma &= \sigma \\
 \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma]
 \end{aligned}$$

10

Revisando las ecuaciones

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases}\end{aligned}$$

10

Revisando las ecuaciones

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases} \\ \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket \star (\llbracket c_0 \rrbracket \sigma)\end{aligned}$$

10

Revisando las ecuaciones

Comandos sencillos

$$\begin{aligned}
 \llbracket \text{skip} \rrbracket \sigma &= \sigma \\
 \llbracket v := e \rrbracket \sigma &= [\sigma \mid v : \llbracket e \rrbracket \sigma] \\
 \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases} \\
 \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket_* (\llbracket c_0 \rrbracket \sigma) \\
 \llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma &= (\text{rest}_{v, \sigma})_* (\llbracket c \rrbracket [\sigma \mid v : \llbracket e \rrbracket \sigma])
 \end{aligned}$$

$\Downarrow (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma' \rightarrow \Sigma')$
 $\downarrow (\Sigma \rightarrow \Sigma) \rightarrow (\Sigma' \rightarrow \Sigma')$

10

Ecuaciones Semánticas de Comandos

La ecuación

Sea $b \in \langle \text{boolexp} \rangle$ y $c \in \langle \text{comm} \rangle$.

$$\begin{aligned}
 F_{b,c} &: (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma \rightarrow \Sigma') \\
 F_{b,c}(f)(\sigma) &= \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f_* (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}
 \end{aligned}$$

11

Ecuaciones Semánticas de Comandos

La ecuación

Sea $b \in \langle \text{boolexp} \rangle$ y $c \in \langle \text{comm} \rangle$.

$$F_{b,c} : (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma \rightarrow \Sigma')$$
$$F_{b,c}(f)(\sigma) = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f_{\star}(\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \left(\bigsqcup_{i \in \mathbb{N}} F^i \perp \right) \sigma$$

11

Cómo “capturar” fallas

Ejemplo

```
def safediv(n,m):  
    if m == 0:  
        raise Exception("A")  
    else:  
        return (n / m)  
  
try:  
    safediv(10,0)  
except Exception("A"):  
    print("err")
```

12

Capturando fallas

Sintaxis

$\langle comm \rangle ::= \dots \mid \text{fail} \mid \text{catchin } \langle comm \rangle \text{ with } \langle comm \rangle$

13

Capturando fallas

Sintaxis

$\langle comm \rangle ::= \dots \mid \text{fail} \mid \text{catchin } \langle comm \rangle \text{ with } \langle comm \rangle$

Semántica

$\llbracket _ \rrbracket : \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma')$

13

Capturando fallas

Sintaxis

$\langle comm \rangle ::= \dots \mid \text{fail} \mid \text{catchin } \langle comm \rangle \text{ with } \langle comm \rangle$

Semántica

$$\begin{aligned} \llbracket _ \rrbracket &: \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma') \\ \llbracket \text{catchin } c_0 \text{ with } c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket + (\llbracket c_0 \rrbracket \sigma) \end{aligned}$$

13

Capturando fallas

Sintaxis

$\langle comm \rangle ::= \dots \mid \text{fail} \mid \text{catchin } \langle comm \rangle \text{ with } \langle comm \rangle$

Semántica

$$\begin{aligned} \llbracket _ \rrbracket &: \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma') \\ \llbracket \text{catchin } c_0 \text{ with } c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket + (\llbracket c_0 \rrbracket \sigma) \end{aligned}$$

donde $+$ está definido por

$$\begin{aligned} (_)_{+} &: (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma' \rightarrow \Sigma') \\ f_{+}(\perp) &= \perp \\ f_{+}(\langle \text{abort}, \sigma \rangle) &= f(\sigma) \\ f_{+}(\sigma) &= \sigma \end{aligned}$$

13



imperativo-
operacional

Lenguajes y Compiladores

Miguel Pagano
5 de mayo de 2023

Repaso

Un Lenguaje Imperativo Simple. Comandos.

Comandos

$\langle comm \rangle ::= \text{skip}$
| $\langle var \rangle := \langle intexp \rangle$
| $\langle comm \rangle ; \langle comm \rangle$
| **if** $\langle boolexp \rangle$ **then** $\langle comm \rangle$ **else** $\langle comm \rangle$
| **newvar** $\langle var \rangle := \langle intexp \rangle$ **in** $\langle comm \rangle$
| **while** $\langle boolexp \rangle$ **do** $\langle comm \rangle$
| **fail**
| **catchin** $\langle comm \rangle$ **with** $\langle comm \rangle$

1

Un Lenguaje Imperativo Simple. Expresiones.

Expresiones

$\langle natconst \rangle ::= 0 \mid 1 \mid 2 \mid \dots$ $\langle boolconst \rangle ::= \text{true} \mid \text{false}$
 $\langle intexp \rangle ::= \langle natconst \rangle$ $\langle boolexp \rangle ::= \langle boolconst \rangle$
| $\langle var \rangle$ | $\neg \langle boolexp \rangle$
| $-\langle intexp \rangle$ | $\langle boolexp \rangle \otimes \langle boolexp \rangle$
| $\langle intexp \rangle \oplus \langle intexp \rangle$ | $\langle boolexp \rangle \oslash \langle boolexp \rangle$
 $\oplus \in \{+, -, *, /, \%, \text{rem}\}$ $\otimes \in \{<, \leq, =, \neq, \geq, >\}$
 $\oslash \in \{\wedge, \vee, \Rightarrow, \Leftrightarrow\}$

2

Significado de los comandos de LIS

La semántica denotacional daba el significado de los comandos como una función de estados en “comportamientos”:

Funciones semánticas

$$\llbracket _ \rrbracket^{intexp} \in \langle intexp \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z})$$

$$\llbracket _ \rrbracket^{boolexp} \in \langle boolexp \rangle \rightarrow (\Sigma \rightarrow \{V, F\})$$

$$\llbracket _ \rrbracket^{comm} \in \langle comm \rangle \rightarrow (\Sigma \rightarrow \Sigma')$$

Recordemos que $\Sigma' = (\Sigma \cup (\{\text{abort}\} \times \Sigma))_{\perp}$.

3

Semántica Operacional. Ideas.

- En la semántica denotacional no aparecen los estados intermedios.
- La semántica **operacional** explicita esos estados intermedios.
- La ejecución de un programa es una secuencia de **configuraciones**:

$$\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \dots$$

- Una configuración, γ , puede ser **terminal**: un estado o un estado fallido;
- o puede ser **no-terminal**: un comando (lo que falta de ejecutar) y un estado.

4

Semántica Operacional. Ideas.

- Una ejecución será una secuencia *maximal* de configuraciones donde cada par sucesivo está relacionado por la relación $\rightarrow$.
- La secuencia puede ser finita, entonces termina en un estado (correcto o fallido),
- o puede ser una secuencia infinita: el programa no termina.

5

Semántica Operacional. Formalmente.

Relación de un paso (o small-step)

$\Gamma_T = \Sigma \cup (\{\text{abort}\} \times \Sigma)$	configuración terminal
$\Gamma_{NT} = \langle \text{comm} \rangle \times \Sigma$	configuración no-terminal
$\Gamma = \Gamma_T \cup \Gamma_{NT}$	configuración
$\rightarrow \subseteq \Gamma_{NT} \times \Gamma$	relación

A continuación definimos la relación $\rightarrow$ inductivamente.

6

La relación $\rightarrow$. Comandos Atómicos

$$\text{(skip)} \frac{}{\langle \text{skip}, \sigma \rangle \rightarrow \sigma}$$

$$\text{(assign)} \frac{}{\langle v := e, \sigma \rangle \rightarrow [\sigma \mid v : \llbracket e \rrbracket \sigma]}$$

$$\text{fail} \frac{}{\langle \text{fail}, \sigma \rangle \rightarrow \langle \text{abort}, \sigma \rangle}$$

7

La relación $\rightarrow$. Composición secuencial.

$$\text{(seq-ok)} \frac{\langle c_0, \sigma \rangle \rightarrow \sigma'}{\langle c_0 ; c_1, \sigma \rangle \rightarrow \langle c_1, \sigma' \rangle}$$

$$\text{(seq-ab)} \frac{\langle c_0, \sigma \rangle \rightarrow \langle \text{abort}, \sigma' \rangle}{\langle c_0 ; c_1, \sigma \rangle \rightarrow \langle \text{abort}, \sigma' \rangle}$$

$$\text{(seq-nt)} \frac{\langle c_0, \sigma \rangle \rightarrow \langle c'_0, \sigma' \rangle}{\langle c_0 ; c_1, \sigma \rangle \rightarrow \langle c'_0 ; c_1, \sigma' \rangle}$$

8

La relación $\rightarrow$. Condicional.

(if-true) $\frac{}{\langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle \rightarrow \langle c_0, \sigma \rangle}$ si $\llbracket b \rrbracket \sigma = T$

(if-false) $\frac{}{\langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle \rightarrow \langle c_1, \sigma \rangle}$ si $\llbracket b \rrbracket \sigma = F$

9

La relación $\rightarrow$. Ciclo.

(while-false) $\frac{}{\langle \text{while } b \text{ do } c, \sigma \rangle \rightarrow \sigma}$ si $\llbracket b \rrbracket \sigma = F$

(while-true) $\frac{}{\langle \text{while } b \text{ do } c, \sigma \rangle \rightarrow \langle c; \text{while } b \text{ do } c, \sigma \rangle}$ si $\llbracket b \rrbracket \sigma = T$

10

La relación $\rightarrow$. Variables locales.

$$(\text{loc-ok}) \frac{\langle c, [\sigma | v : \llbracket e \rrbracket \sigma] \rangle \rightarrow \sigma'}{\langle \text{newvar } v := e \text{ in } c, \sigma \rangle \rightarrow [\sigma' | v : \sigma v]}$$

$$(\text{loc-abort}) \frac{\langle c, [\sigma | v : \llbracket e \rrbracket \sigma] \rangle \rightarrow \langle \text{abort}, \sigma' \rangle}{\langle \text{newvar } v := e \text{ in } c, \sigma \rangle \rightarrow \langle \text{abort}, [\sigma' | v : \sigma v] \rangle}$$

$$(\text{loc-nt}) \frac{\langle c, [\sigma | v : \llbracket e \rrbracket \sigma] \rangle \rightarrow \langle c', \sigma' \rangle}{\langle \text{newvar } v := e \text{ in } c, \sigma \rangle \rightarrow \langle \text{newvar } v := \sigma' v \text{ in } c', [\sigma' | v : \sigma v] \rangle}$$

11

La relación $\rightarrow$. Captura de Fallas.

$$(\text{catch-ok}) \frac{\langle c, \sigma \rangle \rightarrow \sigma'}{\langle \text{catchin } c \text{ with } \hat{c}, \sigma \rangle \rightarrow \sigma'}$$

$$(\text{catch-abort}) \frac{\langle c, \sigma \rangle \rightarrow \langle \text{abort}, \sigma' \rangle}{\langle \text{catchin } c \text{ with } \hat{c}, \sigma \rangle \rightarrow \langle \hat{c}, \sigma' \rangle}$$

$$(\text{catch-nt}) \frac{\langle c, \sigma \rangle \rightarrow \langle c', \sigma' \rangle}{\langle \text{catchin } c \text{ with } \hat{c}, \sigma \rangle \rightarrow \langle \text{catchin } c' \text{ with } \hat{c}, \sigma' \rangle}$$

12

Ejecuciones. Propiedades.

Progreso

Para todo $\langle c, \sigma \rangle$ existe γ tal que $\langle c, \sigma \rangle \rightarrow \gamma$.

Determinismo

Si $\langle c, \sigma \rangle \rightarrow \gamma$ y $\langle c, \sigma \rangle \rightarrow \gamma'$, entonces $\gamma = \gamma'$.

Función

La relación $\rightarrow$ define una función de Γ_{NT} a Γ .

13

Ejecuciones. Formalmente.

Definimos $\rightarrow^*$ como la clausura reflexiva y transitiva de $\rightarrow$:

$$\text{(refl)} \frac{}{\gamma \rightarrow^* \gamma}$$

$$\text{(paso)} \frac{\gamma \rightarrow \gamma'}{\gamma \rightarrow^* \gamma'}$$

$$\text{(trans)} \frac{\gamma_1 \rightarrow^* \gamma_2 \quad \gamma_2 \rightarrow^* \gamma_3}{\gamma_1 \rightarrow^* \gamma_3}$$

14

Ejecución. Formalmente.

Una ejecución es una secuencia maximal de transiciones.

Es decir, $\rho : \gamma_0 \rightarrow^* \gamma_n$ es maximal si γ_n es una configuración terminal o si ρ es infinita.

Si ρ es una secuencia infinita decimos que diverge y escribimos $\gamma \uparrow$.

Definimos una nueva semántica usando la ejecución:

$$\llbracket _ \rrbracket : \langle comm \rangle \rightarrow \Sigma \rightarrow \Sigma'$$
$$\llbracket c \rrbracket \sigma = \begin{cases} \perp & \text{si } \langle c, \sigma \rangle \uparrow \\ \gamma & \text{si } \langle c, \sigma \rangle \rightarrow^* \gamma \in \Gamma_T \end{cases}$$

15

Teorema de Corrección

Teorema de Corrección

Para todo comando c , $\llbracket c \rrbracket = \llbracket c \rrbracket$.

Prueba: práctico.



07_imperat
ivo-io

Lenguaje Imperativo con Output e Input

Miguel Pagano

10 de mayo de 2023

Repaso

Un Lenguaje Imperativo (no tan) Simple

Comandos

$\langle comm \rangle ::= \text{skip}$
| $\langle var \rangle := \langle intexp \rangle$
| $\langle comm \rangle ; \langle comm \rangle$
| **if** $\langle boolexp \rangle$ **then** $\langle comm \rangle$ **else** $\langle comm \rangle$
| **newvar** $\langle var \rangle := \langle intexp \rangle$ **in** $\langle comm \rangle$
| **while** $\langle boolexp \rangle$ **do** $\langle comm \rangle$
| **fail**
| **catchin** $\langle comm \rangle$ **with** $\langle comm \rangle$
| **!** $\langle intexp \rangle$

1

Un Lenguaje Imperativo (no tan) Simple

Expresiones

$\langle natconst \rangle ::= 0 \mid 1 \mid 2 \mid \dots$ $\langle boolconst \rangle ::= \text{true} \mid \text{false}$
 $\langle intexp \rangle ::= \langle natconst \rangle$ $\langle boolexp \rangle ::= \langle boolconst \rangle$
| $\langle var \rangle$ | $\neg \langle boolexp \rangle$
| $-\langle intexp \rangle$ | $\langle intexp \rangle \ominus \langle intexp \rangle$
| $\langle intexp \rangle \oplus \langle intexp \rangle$ | $\langle boolexp \rangle \oslash \langle boolexp \rangle$
 $\oplus \in \{+, -, *, /, \%, \text{rem}\}$ $\ominus \in \{<, \leq, =, \neq, \geq, >\}$
 $\oslash \in \{\wedge, \vee, \Rightarrow, \Leftrightarrow\}$

1

Output

Significado de los comandos de LIS

Posibles comportamientos

1. Cantidad finita de output y un estado final (exitoso o no):
 $\langle 4, 3, 2, 1, [\sigma \mid x : 0] \rangle$ ó $\langle 1, 2, \langle \text{abort}, \sigma \rangle \rangle$.
2. Cantidad finita de output: $\langle 2 \rangle$. $\rightarrow$ se clava.
3. Cantidad infinita de output: $\langle -1, -2, -3, -4, \dots \rangle$.

de qué programas?

1. `!4; !3; !2; !1` (ó `!4; !3; !2; !1; fail`)
2. `!x; (while  $x = 2$  do skip); !x,`
3. `!x; while  $x \neq 0$  do !x;  $x := x - 1$ .`

! output

Dominio de salidas

Ω

$$\begin{aligned}\Omega &= \{\langle n_0, \dots, n_k, \sigma' \rangle \mid n_i \in \mathbb{Z}, \sigma' \in \hat{\Sigma}\} \text{ termino o fallo} \\ &\cup \{\langle n_0, \dots, n_k \rangle \mid n_i \in \mathbb{Z}\} \text{ imprimio y se clausa} \\ &\cup \{\langle n_0, n_1, n_2, n_3, \dots, n_k, \dots \rangle \mid n_i \in \mathbb{Z}\} \text{ imprimio infinito} \\ &\text{donde } \hat{\Sigma} = \Sigma \cup (\{\text{abort}\} \times \Sigma)\end{aligned}$$

Concatenación de tuplas

Podemos “concatenar” ciertos elementos de Ω y obtener elementos de Ω :

$$\langle n_0, \dots, n_k \rangle ++ \langle m_0, \dots, s \rangle = \langle n_0, \dots, n_k, m_0, \dots, s \rangle$$

Ahí s puede ser un entero o un estado (fallido o no).

3

Dominio de salidas

Estructura de dominio

1. El orden está dado por ser prefijo: $\langle 4, 3, 2, 1 \rangle \leq \langle 4, 3, 2, 1, \sigma \rangle$.

4

Dominio de salidas

Estructura de dominio

1. El orden está dado por ser prefijo: $\langle 4, 3, 2, 1 \rangle \leq \langle 4, 3, 2, 1, \sigma \rangle$.
2. El supremo de una cadena no interesante es el último elemento de la cadena.

4

Dominio de salidas

Estructura de dominio

1. El orden está dado por ser prefijo: $\langle 4, 3, 2, 1 \rangle \leq \langle 4, 3, 2, 1, \sigma \rangle$.
2. El supremo de una cadena no interesante es el último elemento de la cadena.
3. El supremo de una cadena interesante es *la* tupla que coincide en todas las posiciones de cada tupla de la cadena.

4

Dominio de salidas

Estructura de dominio

1. El orden está dado por ser prefijo: $\langle 4, 3, 2, 1 \rangle \leq \langle 4, 3, 2, 1, \sigma \rangle$.
2. El supremo de una cadena no interesante es el último elemento de la cadena.
3. El supremo de una cadena interesante es **la** tupla que coincide en todas las posiciones de cada tupla de la cadena.

4

Dominio de salidas

Estructura de dominio

1. El orden está dado por ser prefijo: $\langle 4, 3, 2, 1 \rangle \leq \langle 4, 3, 2, 1, \sigma \rangle$.
2. El supremo de una cadena no interesante es el último elemento de la cadena.
3. El supremo de una cadena interesante es **la** tupla que coincide en todas las posiciones de cada tupla de la cadena.

Observaciones

1. El menor elemento está dado por $\langle \rangle$.

4

Dominio de salidas

Estructura de dominio

1. El orden está dado por ser prefijo: $\langle 4, 3, 2, 1 \rangle \leq \langle 4, 3, 2, 1, \sigma \rangle$.
2. El supremo de una cadena no interesante es el último elemento de la cadena.
3. El supremo de una cadena interesante es **la** tupla que coincide en todas las posiciones de cada tupla de la cadena.

Observaciones

1. El menor elemento está dado por $\langle \rangle$.
2. En una cadena interesante de tuplas hay tuplas de cada longitud,

4

Dominio de salidas

Estructura de dominio

1. El **orden** está dado por **ser prefijo**: $\langle 4, 3, 2, 1 \rangle \leq \langle 4, 3, 2, 1, \sigma \rangle$.
2. El **supremo de una cadena no interesante es el último elemento de la cadena.**
3. El supremo de una cadena interesante es **la tupla** que coincide en todas las posiciones de cada tupla de la cadena.

Observaciones

1. El menor elemento está dado por $\langle \rangle$.
2. En una cadena interesante de tuplas **hay tuplas de cada longitud,**
3. por lo tanto el **supremo es efectivamente único.**

4

Extensión de lenguaje y de semántica

Semántica

Nueva semántica de programas:

$$\llbracket _ \rrbracket : \langle comm \rangle \rightarrow (\Sigma \rightarrow \Omega)$$

5

Extensión de lenguaje y de semántica

Semántica

Nueva **semántica** de programas:

$$\begin{aligned} \llbracket _ \rrbracket &: \langle comm \rangle \rightarrow (\Sigma \rightarrow \Omega) \\ \llbracket !e \rrbracket \sigma &= \langle \llbracket e \rrbracket \sigma, \sigma \rangle \end{aligned}$$

5

Ecuaciones Semánticas de Comandos (sin output)

Comandos sencillos

$$\llbracket \text{skip} \rrbracket \sigma = \langle \sigma \rangle$$

6

Ecuaciones Semánticas de Comandos (sin output)

Comandos sencillos

$$\llbracket \text{skip} \rrbracket \sigma = \langle \sigma \rangle$$

$$\llbracket v := e \rrbracket \sigma = \langle [\sigma \mid v : \llbracket e \rrbracket \sigma] \rangle$$

6

Ecuaciones Semánticas de Comandos (sin output)

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \langle \sigma \rangle \\ \llbracket v := e \rrbracket \sigma &= \langle [\sigma \mid v : \llbracket e \rrbracket \sigma] \rangle \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases}\end{aligned}$$

6

Ecuaciones Semánticas de Comandos (sin output)

Comandos sencillos

$$\begin{aligned}\llbracket \text{skip} \rrbracket \sigma &= \langle \sigma \rangle \\ \llbracket v := e \rrbracket \sigma &= \langle [\sigma \mid v : \llbracket e \rrbracket \sigma] \rangle \\ \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases} \\ \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket_* (\llbracket c_0 \rrbracket \sigma)\end{aligned}$$

6

Ecuaciones Semánticas de Comandos (sin output)

Comandos sencillos

$$\begin{aligned}
 \llbracket \text{skip} \rrbracket \sigma &= \langle \sigma \rangle \\
 \llbracket v := e \rrbracket \sigma &= \langle [\sigma \mid v : \llbracket e \rrbracket \sigma] \rangle \\
 \llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma &= \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases} \\
 \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket_* (\llbracket c_0 \rrbracket \sigma) \\
 \llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma &= (\text{rest}_{v, \sigma}) \uparrow (\llbracket c \rrbracket [\sigma \mid v : \llbracket e \rrbracket \sigma])
 \end{aligned}$$

6

Ecuaciones Semánticas de Comandos (sin output)

La ecuación

Sea $b \in \langle \text{boolexp} \rangle$ y $c \in \langle \text{comm} \rangle$.

$$\begin{aligned}
 F_{b,c} &: (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma \rightarrow \Sigma') \\
 F_{b,c}(f)(\sigma) &= \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f_* (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}
 \end{aligned}$$

7

Ecuaciones Semánticas de Comandos (sin output)

La ecuación

Sea $b \in \langle \text{boolexp} \rangle$ y $c \in \langle \text{comm} \rangle$.

$$F_{b,c} : (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma \rightarrow \Sigma')$$

$$F_{b,c}(f)(\sigma) = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f_* (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \left(\bigsqcup_{i \in \mathbb{N}} F_{b,c}^i \perp \right) \sigma$$

No hay que cambiar nada, sólo redefinir los operadores.

7

Funciones de propagación de errores

Propagación de errores

$$(_)_* : (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma' \rightarrow \Sigma')$$

$$f_*(\perp) = \perp$$

$$f_*(\langle \text{abort}, \sigma \rangle) = \langle \text{abort}, \sigma \rangle$$

$$f_*(\sigma) = f(\sigma)$$

Como map para listas

$$(_)_{\dagger} : (\Sigma \rightarrow \Sigma) \rightarrow (\Sigma' \rightarrow \Sigma')$$

$$f_{\dagger}(\perp) = \perp$$

$$f_{\dagger}(\langle \text{abort}, \sigma \rangle) = \langle \text{abort}, f(\sigma) \rangle$$

$$f_{\dagger}(\sigma) = f(\sigma)$$

8

Revisando Funciones de propagación de errores

Propagación de errores

$$(_)_* : (\Sigma \rightarrow \Omega) \rightarrow (\Omega \rightarrow \Omega)$$

$$f_*(\langle n_0, \dots, n_{k-1} \rangle) = \langle n_0, \dots, n_{k-1} \rangle$$

$$f_*(\langle n_0, \dots, n_{k-1}, \sigma \rangle) = \langle n_0, \dots, n_{k-1} \rangle \uparrow\uparrow f(\sigma)$$

$$f_*(\langle n_0, \dots, n_{k-1}, \langle \text{abort}, \sigma \rangle \rangle) = \langle n_0, \dots, n_{k-1}, \langle \text{abort}, \sigma \rangle \rangle$$

$$f_*(\langle n_0, \dots, n_{k-1}, \dots \rangle) = \langle n_0, \dots, n_{k-1}, \dots \rangle$$

$$f_* (\langle n_0, \dots, n_{k-1}, \langle \text{abort}, \sigma \rangle \rangle) = \langle n_0, \dots, n_{k-1}, \langle \text{abort}, \sigma \rangle \rangle$$

$$f_* (\langle n_0, \dots, n_{k-1}, \dots \rangle) = \langle n_0, \dots, n_{k-1}, \dots \rangle$$

9

Revisando Funciones de propagación de errores

Como *map* para listas

$$(_)_{\dagger} : (\Sigma \rightarrow \Sigma) \rightarrow (\Omega \rightarrow \Omega)$$

$$f_{\dagger} (\langle n_0, \dots, n_{k-1} \rangle) = \langle n_0, \dots, n_{k-1} \rangle$$

$$f_{\dagger} (\langle n_0, \dots, n_{k-1}, \sigma \rangle) = \langle n_0, \dots, n_{k-1}, f(\sigma) \rangle$$

$$f_{\dagger} (\langle n_0, \dots, n_{k-1}, \langle \text{abort}, \sigma \rangle \rangle) = \langle n_0, \dots, n_{k-1}, \langle \text{abort}, f(\sigma) \rangle \rangle$$

$$f_{\dagger} (\langle n_0, \dots, n_{k-1}, \dots \rangle) = \langle n_0, \dots, n_{k-1}, \dots \rangle$$

10

Una presentación más estructural para Ω

Consideremos las siguientes funciones:

$$\iota_{\perp} : \{\langle \rangle\} \rightarrow \Omega$$

$$\iota_{\perp}(\langle \rangle) = \langle \rangle$$

$$\iota_{\text{abort}} : \Sigma \rightarrow \Omega$$

$$\iota_{\text{abort}}(\sigma) = \langle \sigma \rangle$$

$$\iota_{\text{term}} : \Sigma \rightarrow \Omega$$

$$\iota_{\text{term}}(\sigma) = \langle \sigma \rangle$$

$$\iota_{\text{out}} : \mathbb{Z} \times \Omega \rightarrow \Omega$$

$$\iota_{\text{out}}(\langle n, \omega \rangle) = \langle n \rangle ++ \omega$$

11

Revisando Funciones de propagación de errores

Propagación de errores

$$\begin{aligned}(_)_* &: (\Sigma \rightarrow \Omega) \rightarrow (\Omega \rightarrow \Omega) \\ f_*(\perp) &= \perp \\ f_*(\iota_{term}(\sigma)) &= f(\sigma) \\ f_*(\iota_{abort}(\sigma)) &= \iota_{abort}(\sigma) \\ f_*(\iota_{out}(\langle n, \omega \rangle)) &= \iota_{out}(\langle n, f_*(\omega) \rangle)\end{aligned}$$

Observación: Es fundamental que f_* sea continua.

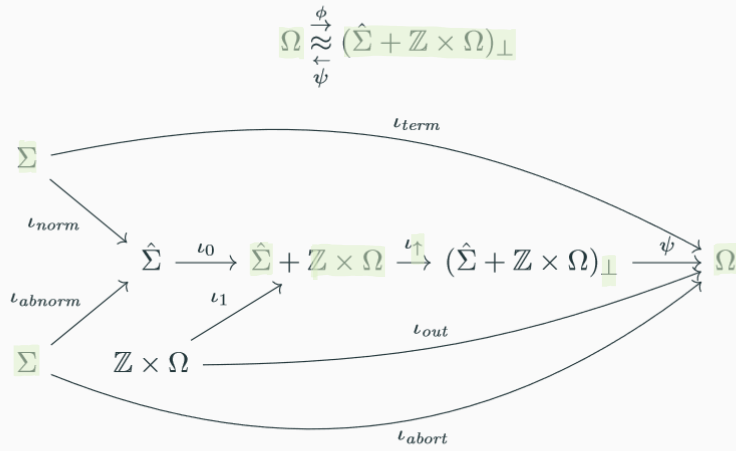
12

Una nueva definición de Omega

$$\Omega \overset{\phi}{\underset{\psi}{\approx}} (\hat{\Sigma} + \mathbb{Z} \times \Omega)_{\perp}$$

13

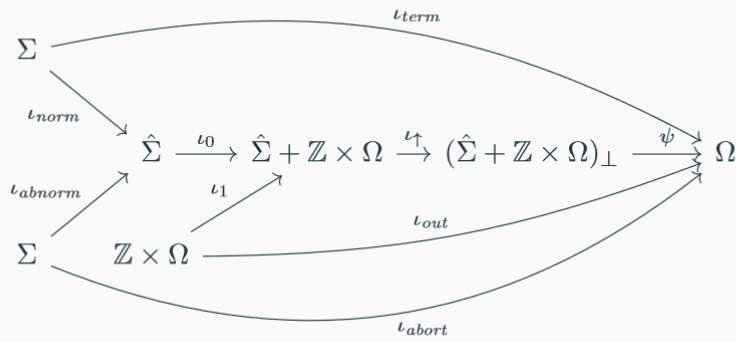
Una nueva definición de Omega



Observación: El orden de Ω lo podemos pensar como el orden $(\hat{\Sigma} + \mathbb{Z} \times \Omega)_{\perp}$.

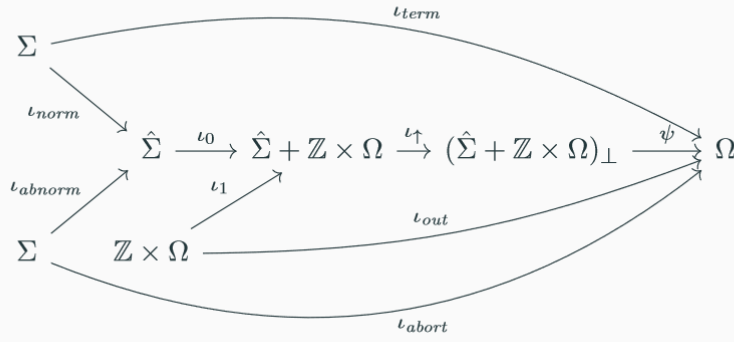
13

Una nueva definición de Omega



14

Una nueva definición de Omega



$$\iota_{term} = \psi \circ \iota_{\uparrow} \circ \iota_0 \circ \iota_{norm}$$

$$\iota_{abort} = \psi \circ \iota_{\uparrow} \circ \iota_0 \circ \iota_{abnorm}$$

$$\iota_{out} = \psi \circ \iota_{\uparrow} \circ \iota_1$$

Observación: Podemos extender Ω con nuevas construcciones.

14

Propagación de comportamientos y map

$$(_)_{*}: (\Sigma \rightarrow \Omega) \rightarrow (\Omega \rightarrow \Omega)$$

$$f_{*}(\perp) = \perp$$

$$f_{*}(\iota_{term}(\sigma)) = f(\sigma)$$

$$f_{*}(\iota_{abort}(\sigma)) = \iota_{abort}(\sigma)$$

$$f_{*}(\iota_{out}(n, \omega)) = \iota_{out}(n, f_{*}(\omega))$$

$$(_)_{\dagger}: (\Sigma \rightarrow \Sigma) \rightarrow (\Omega \rightarrow \Omega)$$

$$f_{\dagger}(\perp) = \perp$$

$$f_{\dagger}(\iota_{term}(\sigma)) = \iota_{term}(f(\sigma))$$

$$f_{\dagger}(\iota_{abort}(\sigma)) = \iota_{abort}(f(\sigma))$$

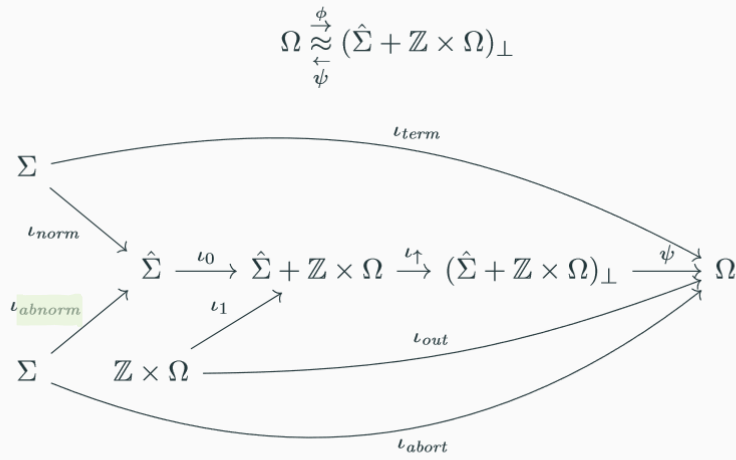
$$f_{\dagger}(\iota_{out}(\langle n, \omega \rangle)) = \iota_{out}(\langle n, f_{\dagger}(\omega) \rangle)$$

15

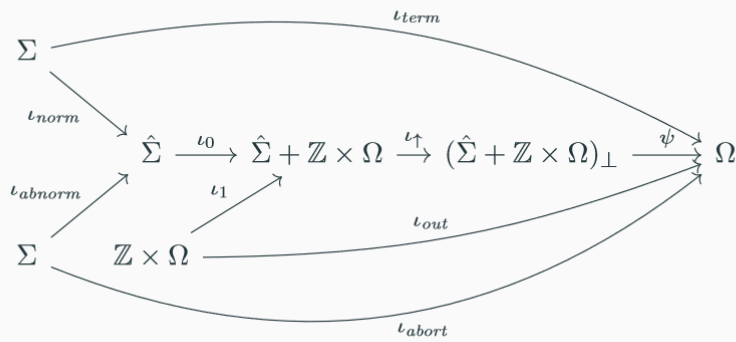
Una nueva definición de Omega

$$\Omega \xrightarrow[\psi]{\phi} (\hat{\Sigma} + \mathbb{Z} \times \Omega)_{\perp}$$

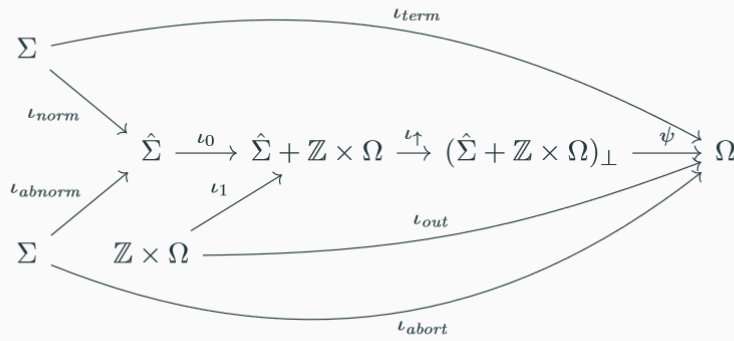
Una nueva definición de Omega



Una nueva definición de Omega



Una nueva definición de Omega



$$l_{\text{term}} = \psi \circ l_{\uparrow} \circ l_0 \circ l_{\text{norm}}$$

$$l_{\text{abort}} = \psi \circ l_{\uparrow} \circ l_0 \circ l_{\text{abnorm}}$$

$$l_{\text{out}} = \psi \circ l_{\uparrow} \circ l_1$$

Observación: Podemos extender Ω con nuevas construcciones.

17

Un Lenguaje Imperativo con Output e Input

Comandos

```

<comm> ::= skip
| <var> := <intexp>
| <comm> ; <comm>
| if <boolexp> then <comm> else <comm>
| newvar <var> := <intexp> in <comm>
| while <boolexp> do <comm>
| fail
| catchin <comm> with <comm>
| ! <intexp>
| ? <var> input.
    
```

18

Un dominio con comportamientos para Input

$$\Omega \overset{\phi}{\underset{\psi}{\approx}} (\hat{\Sigma} + \mathbb{Z} \times \Omega + \mathbb{Z} \rightarrow \Omega)_{\perp}$$

19

Un dominio con comportamientos para Input

$$\Omega \overset{\phi}{\underset{\psi}{\approx}} (\hat{\Sigma} + \overbrace{\mathbb{Z} \times \Omega}^{\text{out}} + \overbrace{\mathbb{Z}}^{\text{input}} \rightarrow \Omega)_{\perp}$$

$$\iota_{term} = \psi \circ \iota_{\uparrow} \circ \iota_0 \circ \iota_{norm}$$

$$\iota_{abort} = \psi \circ \iota_{\uparrow} \circ \iota_0 \circ \iota_{abnorm}$$

$$\iota_{out} = \psi \circ \iota_{\uparrow} \circ \iota_1$$

$$\iota_{in} = \psi \circ \iota_{\uparrow} \circ \iota_2 : (\mathbb{Z} \rightarrow \Omega) \rightarrow \Omega$$

Observación: Podemos extender Ω con nuevas construcciones.

19

Propagación de comportamientos y map

$$\begin{aligned}(_)_{*}: (\Sigma \rightarrow \Omega) &\rightarrow (\Omega \rightarrow \Omega) \\ f_{*}(\perp) &= \perp \\ f_{*}(\iota_{term}(\sigma)) &= f(\sigma) \\ f_{*}(\iota_{abort}(\sigma)) &= \iota_{abort}(\sigma) \\ f_{*}(\iota_{out}(\langle n, \omega \rangle)) &= \iota_{out}(\langle n, f_{*}(\omega) \rangle) \\ f_{*}(\iota_{in}(g)) &= \iota_{in}(\lambda n \in \mathbb{Z}. f_{*}(g(n)))\end{aligned}$$

20

Propagación de “restauración” (map)

$$\begin{aligned}(_)_{\dagger}: (\Sigma \rightarrow \Sigma) &\rightarrow (\Omega \rightarrow \Omega) \\ f_{\dagger}(\perp) &= \perp \\ f_{\dagger}(\iota_{term}(\sigma)) &= \iota_{term}(f(\sigma)) \\ f_{\dagger}(\iota_{abort}(\sigma)) &= \iota_{abort}(f(\sigma)) \\ f_{\dagger}(\iota_{out}(n, \omega)) &= \iota_{out}(n, f_{\dagger}(\omega)) \\ f_{\dagger}(\iota_{in}(g)) &= \iota_{in}(\lambda n \in \mathbb{Z}. f_{\dagger}(g(n)))\end{aligned}$$

21

hmbola

Monday, September 22, 2025

5:26 PM



09_calculo-l
ambda_h...

Lenguajes y Compiladores

Miguel Pagano

3 de mayo de 2024

Estructura de la materia a grandes rasgos:

Primera Parte Lenguaje imperativo.

Segunda Parte Lenguaje aplicativo puro, y lenguaje aplicativo con referencias y asignación

Cálculo Lambda

2

Cálculo Lambda

El Cálculo Lambda

- Church (1930), continuado por Kleene, Rosser (1930s)
Motivación original: problemas de fundamentación de la matemática
- **Como notación:** se usa para generar una expresión que denote una función, sin necesidad de dar nombre
Por ejemplo: $f(x) = x + y$ se puede escribir

$$f = \lambda x. x + y,$$

con lo cual uno se puede referir a la función f mediante la expresión $\lambda x. x + y$, sin necesidad de ponerle nombre.

- Todas las funciones recursivas son definibles en el Cálculo Lambda.

3

Sintaxis abstracta del CL

$\langle expr \rangle ::=$	expresiones o términos
$\langle var \rangle$	variables
$ \langle expr \rangle \langle expr \rangle$	aplicación
$ \lambda \langle var \rangle. \langle expr \rangle$	abstracción o expresión lambda

Convención: La aplicación asocia a izquierda. Por ejemplo,

$\lambda x. (\lambda y. xy)x$ es lo mismo que

$\lambda x. (\lambda y. (xy)x)$

4

Variables libres

En la abstracción

$$\lambda x. x + 2$$

se determina que estamos construyendo una función que varía de acuerdo a qué valor toma x . Es decir, la abstracción es un *ligador*.

$$\begin{aligned} FV(v) &= \{v\} \\ FV(ee') &= FV(e) \cup FV(e') \\ FV(\lambda v. e) &= FV(e) - \{v\} \end{aligned}$$

5

Operador de sustitución

Conjunto de sustituciones: $\Delta = \langle var \rangle \rightarrow \langle expr \rangle$

Operador de sustitución: $_/_ \in \langle expr \rangle \times \Delta \rightarrow \langle expr \rangle$

$$\begin{aligned} v/\delta &= \delta v \\ (ee')/\delta &= (e/\delta)(e'/\delta) \\ (\lambda v. e)/\delta &= \lambda v_{new}. e/[\delta|v : v_{new}] \\ &\text{donde } v_{new} \notin \bigcup_{w \in FV(e) - \{v\}} FV(\delta w) \end{aligned}$$

Notaciones

$I: \Delta$	Sustitución identidad
$I(x) = x$	
$v \mapsto e: \Delta$	Sustitución en un punto
$v \mapsto e = [I \mid v : e]$	

6

Conversión α

Renombre: Cambio en $\lambda v.e$ de la variable ligada v (y todas sus ocurrencias) por una variable v' que no ocurra libre en e :

$$\lambda v'. e/v \mapsto v'$$

donde $v' \notin FV(e)$.

α -conversión: Si e_1 se obtiene a partir de e_0 por cero o más renombres de ocurrencias de subfrases. También se dice que e_0 α -convierte a e_1 .

Notación para expresiones α -convertibles: $e_0 \equiv e_1$

Podemos definir $\equiv$ mediante un conjunto de reglas.

7

Ejecución: Contracción β

Redex: Es una expresión de la forma $(\lambda v.e)e'$

Contracción β : Reemplaza en e_0 una ocurrencia de un redex $(\lambda v.e)e'$ por su contracción $(e/v \mapsto e')$, y luego efectúa cero o más renombres de cualquier subexpresión.

Notación: Si e_1 es el resultado de una contracción β de e_0 , entonces escribimos

$$e_0 \rightarrow e_1$$

8

Ejecución: Formas normales

Forma normal: expresión sin rédices.

Forma Normal

Las formas normales representan configuraciones terminales.

Por eso la semántica operacional del cálculo lambda consiste en efectuar contracciones β hasta obtener formas normales.

Ejemplo:

$$\begin{aligned}(\lambda x. \lambda y. (\lambda x. xy)(zx))(yz) &\rightarrow (\lambda x. \lambda y. (zx)y)(yz) \\ &\equiv (\lambda x. \lambda t. (zx)t)(yz) \\ &\rightarrow \lambda t. z(yz)t\end{aligned}$$

9

Ejecución: hay expresiones divergentes

Sea

$$\Delta = \lambda x. xx$$

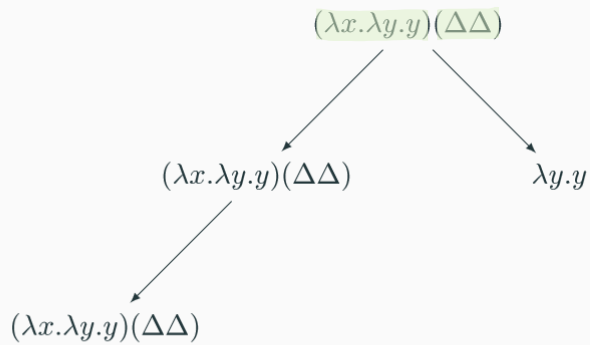
entonces $\Delta\Delta$ no tiene forma normal:

$$\begin{aligned}(\lambda x. xx)(\lambda x. xx) &\rightarrow (\lambda x. xx)(\lambda x. xx) \\ (\lambda x. xx)(\lambda x. xx) &\rightarrow (\lambda x. xx)(\lambda x. xx) \\ &\rightarrow (\lambda x. xx)(\lambda x. xx) \\ &\vdots\end{aligned}$$

10

Ejecución: hay más de una forma de reducción

En otros casos, la forma normal se puede encontrar si elegimos correctamente qué redex reducir en cada caso:



Es decir la reducción no es determinística.

11

Ejecución (formalmente)

$\rightarrow^*$ denota la clausura transitiva y reflexiva de $\rightarrow$

(o sea, aplicar $\rightarrow$ cero o más veces)

Por ejemplo, no existe forma normal N tal que $\Delta\Delta \rightarrow^* N$

Formalmente:

$e \rightarrow^* e'$ si y sólo si existen $e_0, \dots, e_n$ (con $n \geq 0$) tales que

$$e = e_0 \rightarrow e_1 \rightarrow \dots \rightarrow e_n = e'$$

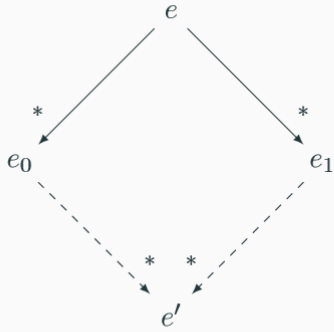
Notar que si $n = 0$ entonces $e = e'$

12

Si existe forma normal, es única

Es consecuencia inmediata de:

Teorema de Church-Rosser Si $e \rightarrow^* e_0$ y $e \rightarrow^* e_1$, entonces existe e' tal que $e_0 \rightarrow^* e'$ y $e_1 \rightarrow^* e'$.



13

Regla η

Un η -redex es una expresión de la forma $\lambda v. ev$, donde $v \notin FV e$

$$\frac{}{\lambda v. ev \rightarrow e} \text{ si } v \notin FV e \quad (\eta)$$

14

Evaluación

→* no representa adecuadamente la ejecución de los lenguajes aplicativos. En los lenguajes aplicativos la evaluación

1. sólo está definida para expresiones cerradas,
2. es determinística,
3. no busca formas normales sino formas canónicas.

Vamos a estudiar:

Evaluación (en orden) normal: lenguajes funcionales lazy (Haskell)

Evaluación eager o estricta: lenguajes estrictos (ML).

15

Formas canónicas

La evaluación busca una forma canónica, las mismas juegan el rol de ser "valores" de expresiones.

La noción de forma canónica depende de la definición de evaluación. Se define una noción de forma canónica para la evaluación normal, y otra para la evaluación eager.

En el caso del cálculo lambda coinciden: son las abstracciones

16

Formas canónicas vs. formas normales

Propiedad: Una aplicación cerrada no puede ser forma normal.

Corolario: una expresión cerrada que es forma normal es también forma canónica.

El recíproco no vale.

17

Formas canónicas

¿Por qué conformarse con una forma canónica en vez de continuar ejecutando hasta obtener una forma normal?

Sólo tiene sentido evaluar expresiones cerradas.

Una vez que se alcanzó una abstracción $\lambda v.e$, continuar evaluando implicaría evaluar e que puede no ser una expresión cerrada, puede contener a la variable v .

18

Semántica natural o big-step

En este tipo de semántica, uno no describe un paso de ejecución, sino directamente una relación entre los términos y sus valores (que también son términos, son formas canónicas).

Llamaremos $\Rightarrow$ a esta relación.

19

Evaluación Normal

Reglas para $\Rightarrow_N$

Regla para las formas canónicas

$$\overline{\lambda v.e \Rightarrow_N \lambda v.e}$$

Regla para la aplicación

$$\frac{e \Rightarrow_N \lambda v.e_0 \quad (e_0/v \mapsto e') \Rightarrow_N z}{ee' \Rightarrow_N z}$$

20

Reglas para $\Rightarrow_E$

Regla para las formas canónicas

$$\overline{\lambda v.e \Rightarrow_E \lambda v.e}$$

Regla para la aplicación

$$\frac{e \Rightarrow_E \lambda v.e_0 \quad e' \Rightarrow_E z' \quad (e_0/v \mapsto z') \Rightarrow_E z}{ee' \Rightarrow_E z}$$



10_calculo-l
ambda_h...

Lenguajes y Compiladores

Miguel Pagano

10 de mayo de 2024

Cálculo Lambda

El Cálculo Lambda

- Church (1930), continuado por Kleene, Rosser (1930s)
Motivación original: problemas de fundamentación de la matemática
- **Como notación:** se usa para generar una expresión que denote una función, sin necesidad de dar nombre
Por ejemplo: $f(x) = x + y$ se puede escribir

$$f = \lambda x. x + y,$$

con lo cual uno se puede referir a la función f mediante la expresión $\lambda x. x + y$, sin necesidad de ponerle nombre.

- Todas las funciones recursivas son definibles en el Cálculo Lambda.

1

Sintaxis abstracta del CL

$\langle expr \rangle ::=$	expresiones o términos
$\langle var \rangle$	variables
$ \langle expr \rangle \langle expr \rangle$	aplicación
$ \lambda \langle var \rangle. \langle expr \rangle$	abstracción o expresión lambda

Convención: La aplicación asocia a izquierda. Por ejemplo,

$\lambda x. (\lambda y. xy)x$ es lo mismo que

$\lambda x. (\lambda y. (xy)x)$

2

Variables libres

En la abstracción

$$\lambda x. x + 2$$

se determina que estamos construyendo una función que varía de acuerdo a qué valor toma x . Es decir, la abstracción es un **ligador**.

$$\begin{aligned} FV(v) &= \{v\} \\ FV(ee') &= FV(e) \cup FV(e') \\ FV(\lambda v. e) &= FV(e) - \{v\} \end{aligned}$$

3

Operador de sustitución

Conjunto de sustituciones: $\Delta = \langle var \rangle \rightarrow \langle expr \rangle$

Operador de sustitución: $_/_ \in \langle expr \rangle \times \Delta \rightarrow \langle expr \rangle$

$$v/\delta = \delta v$$

$$(ee')/\delta = (e/\delta)(e'/\delta)$$

$$(\lambda v.e)/\delta = \lambda v_{new}. e/[\delta|v : v_{new}]$$

donde $v_{new} \notin \bigcup_{w \in FV(e) - \{v\}} FV(\delta w)$

Notaciones

$I : \Delta$ Sustitución identidad

$$I(x) = x$$

$v \mapsto e : \Delta$ Sustitución en un punto

$$v \mapsto e = [I | v : e]$$

4

Conversión α

Renombre: Cambio en $\lambda v.e$ de la variable ligada v (y todas sus ocurrencias) por una variable v' que no ocurra libre en e :

$$\lambda v'. e/v \mapsto v'$$

donde $v' \notin FV(e)$.

α -conversión: Si e_1 se obtiene a partir de e_0 por cero o más renombres de ocurrencias de subfrases. También se dice que e_0 α -convierte a e_1 .

Notación para expresiones α -convertibles: $e_0 \equiv e_1$

Podemos definir $\equiv$ mediante un conjunto de reglas.

5

Ejecución: Contracción β

Redex: Es una expresión de la forma $(\lambda v.e)e'$

Contracción β : Reemplaza en e_0 una ocurrencia de un redex $(\lambda v.e)e'$ por su contracción $(e/v \mapsto e')$, y luego efectúa cero o más renombres de cualquier subexpresión.

Notación: Si e_1 es el resultado de una contracción β de e_0 , entonces escribimos

$$e_0 \rightarrow e_1$$

6

Ejecución: Formas normales

Forma normal: expresión sin redices.

Las formas normales representan configuraciones terminales.

Por eso la semántica operacional del cálculo lambda consiste en efectuar contracciones β hasta obtener formas normales.

7

Ejecución (formalmente)

$\rightarrow^*$ denota la clausura transitiva y reflexiva de $\rightarrow$

(o sea, aplicar $\rightarrow$ cero o más veces)

Formalmente:

$e \rightarrow^* e'$ si y sólo si existen $e_0, \dots, e_n$ (con $n \geq 0$) tales que

$$e = e_0 \rightarrow e_1 \rightarrow \dots \rightarrow e_n = e'$$

Notar que si $n = 0$ entonces $e = e'$

8

Sintaxis y semántica

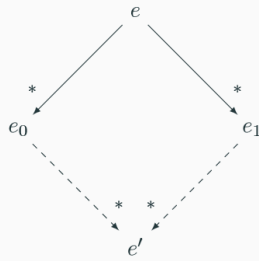
1. ¿Cómo damos un modelo semántico del cálculo lambda?
2. ¿Qué propiedades queremos de un “modelo” del cálculo lambda?

9

Si existe forma normal, es única

Es consecuencia inmediata de:

Teorema de Church-Rosser Si $e \rightarrow^* e_0$ y $e \rightarrow^* e_1$, entonces existe e' tal que $e_0 \rightarrow^* e'$ y $e_1 \rightarrow^* e'$.



10

Regla η

Un η -redex es una expresión de la forma $\lambda v. ev$, donde $v \notin FV e$

$$\frac{}{\lambda v. ev \rightarrow e} \text{ si } v \notin FV e \quad (\eta)$$

11

Evaluación Normal

Reglas para $\Rightarrow_N$

Regla para las formas canónicas

$$\frac{}{\lambda v. e \Rightarrow_N \lambda v. e}$$

Regla para la aplicación

$$\frac{e \Rightarrow_N \lambda v. e_0 \quad (e_0/v \mapsto e') \Rightarrow_N z}{ee' \Rightarrow_N z}$$

12

Evaluación Eager

Reglas para $\Rightarrow_E$

Regla para las formas canónicas

$$\overline{\lambda v.e \Rightarrow_E \lambda v.e}$$

Regla para la aplicación

$$\frac{e \Rightarrow_E \lambda v.e_0 \quad e' \Rightarrow_E z' \quad (e_0/v \mapsto z') \Rightarrow_E z}{ee' \Rightarrow_E z}$$

13

Sintaxis y semántica

1. ¿Cómo damos un modelo semántico del cálculo lambda?
2. ¿Qué propiedades queremos de un “modelo” del cálculo lambda?

14

Semántica Denotacional del Cálculo Lambda

Necesitamos poder modelar la aplicación y la abstracción:

$$\text{abs}: (C \rightarrow C) \rightarrow C$$

$$\text{app}: (C \rightarrow C) \times C \rightarrow C$$

tales que $\text{app}(\langle \text{abs}(f), x \rangle) = f(x)$.

Pero además permitimos la auto-aplicación, así que queremos:

$$C \overset{\phi}{\cong} C \rightarrow C$$

Sea $f: C \rightarrow C$ cualquier función, y definamos $p_f(x) = f(x(x))$.

Entonces, paradoja, toda función tiene punto fijo.

15

Semántica Denotacional del Cálculo Lambda

La función que mapea toda función a su punto fijo:

$$Y: (C \rightarrow C) \rightarrow C$$
$$Y(f) = p_f(p_f)$$

$$Y(f) = p_f(p_f) = f(p_f(p_f)) = p_f$$

¿Lo podemos hacer sintácticamente? Fijemos f :

$$p_f \doteq \lambda x. f(x x)$$

Entonces

$$p_f p_f \doteq (\lambda x. f(x x))(\lambda x. f(x x))$$

Ahora abstraigamos f :

$$Y \doteq \lambda f. (\lambda x. f(x x))(\lambda x. f(x x))$$

16

Semántica Denotacional del Cálculo Lambda

Dana Scott en 1969 mostró que podemos construir un dominio D_∞ :

$$\phi: D_\infty \rightarrow [D_\infty \rightarrow D_\infty]$$
$$\psi: [D_\infty \rightarrow D_\infty] \rightarrow D_\infty$$

tales que

$$\phi \circ \psi = Id_{[D_\infty \rightarrow D_\infty]}$$
$$\psi \circ \phi = Id_{D_\infty}$$

17

Semántica Denotacional del Cálculo Lambda

Ambientes (Entornos): $Env = \langle var \rangle \rightarrow D_\infty$

Función semántica: $\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D_\infty$

Ecuaciones semánticas:

$$\llbracket v \rrbracket \eta = \eta v$$
$$\llbracket e_0 e_1 \rrbracket \eta = \phi(\llbracket e_0 \rrbracket \eta) \llbracket e_1 \rrbracket \eta$$
$$\llbracket \lambda v. e \rrbracket \eta = \psi(\lambda d \in D_\infty. \llbracket e \rrbracket [\eta | v : d])$$

18

Propiedades de la semántica del CL

Teorema de Coincidencia:

Si $\eta w = \eta' w$ para todo $w \in FV e$, entonces $\llbracket e \rrbracket \eta = \llbracket e \rrbracket \eta'$.

Teorema de Renombre

Si $v_{new} \notin FV e - \{v\}$, entonces $\llbracket \lambda v_{new}.(e/v \mapsto v_{new}) \rrbracket = \llbracket \lambda v.e \rrbracket$.

Sustituciones: $\Delta = \langle var \rangle \rightarrow \langle exp \rangle$

Teorema de Sustitución

Si $\llbracket \delta w \rrbracket \eta = \eta' w$ para todo $w \in FV e$, entonces $\llbracket e/\delta \rrbracket \eta = \llbracket e \rrbracket \eta'$.

Asumimos sin demostrar $\llbracket \Delta \Delta \rrbracket \eta = \perp$.

19

Corrección de reglas β y η

$$\llbracket (\lambda v.e) e' \rrbracket \eta = \llbracket e/v \mapsto e' \rrbracket \eta$$

$$\llbracket \lambda v.ev \rrbracket \eta = \llbracket e \rrbracket \eta, \text{ si } v \notin FV e$$

20

$$\psi(\lambda \alpha. \llbracket ev \rrbracket [h/v:\alpha])$$

$$\psi(\lambda \alpha. [\phi(\llbracket e \rrbracket h) \alpha])$$

$$\psi(\lambda \alpha. [\underbrace{\phi(\llbracket e \rrbracket h)}_{\llbracket e \rrbracket h} \alpha])$$

$$\psi(\phi(\llbracket e \rrbracket h))$$

$$\llbracket e \rrbracket h$$

$$\phi \llbracket e \rrbracket h$$

$$e \in D_{\infty} \rightarrow D_{\infty}$$

$$x \in \alpha \neq \alpha$$

$$e \in Abs. \Rightarrow \boxed{\lambda x. \alpha}$$

Es D_{∞} modelo de la evaluación?

Cualquier abstracción $\lambda x.e$ es un valor (forma canónica) en ambos órdenes de evaluación.

En particular $\lambda x.\Delta\Delta$ es un valor, pero:

$$\llbracket \lambda x.\Delta\Delta \rrbracket = \perp$$

Necesitamos distinguir la denotación de valores de la denotación de términos que no tienen forma canónica.

21

Modelos para evaluación

Consideremos las reglas de evaluación de la aplicación:

$$\frac{e \Rightarrow \lambda v. e'' \quad \overset{\text{valor}}{e' \Rightarrow z'}}{ee' \Rightarrow z} \quad (e''/v \rightarrow z') \Rightarrow z \quad (\text{Eager})$$

$$\frac{e \Rightarrow \lambda v. e'' \quad (e''/v \rightarrow \overset{\text{expresión}}{e'}) \Rightarrow z}{ee' \Rightarrow z} \quad (\text{Normal})$$

Notar que en un caso reemplazamos valores y en el otro expresiones (eventualmente divergentes).

22

Modelos para evaluación

En ambos casos queremos un dominio para dar sentido a las expresiones que tienen forma canónica y otro para las expresiones divergentes.

	Valores (V)	Términos (D)
Normal	$[D \rightarrow D]$	$V_{\perp}$
Eager	$[V \rightarrow D]$	$V_{\perp}$

Notemos que distinguimos la función constantemente $\perp$ de $\perp_D$.

23

Semántica Denotacional Normal

Queremos las siguientes ecuaciones de dominios:

$$D^N = V_{\perp}^N$$

$$V^N \cong [D^N \rightarrow D^N]$$

usaremos el siguiente isomorfismos y funciones:

$$\phi: V^N \rightarrow [D^N \rightarrow D^N]$$

$$\psi: [D^N \rightarrow D^N] \rightarrow V^N$$

con

$$\phi \circ \psi = Id_{[D^N \rightarrow D^N]}$$

$$\psi \circ \phi = Id_{V^N}$$

$$\iota_{\perp}: V^N \rightarrow D^N$$

24

Semántica Denotacional del Cálculo Lambda Normal

$$D^N = V_{\perp}^N$$
$$V^N \cong [D^N \rightarrow D^N]$$

Ambientes: $Env = (\langle var \rangle \rightarrow D^N)$

Función semántica:

$$\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D^N$$
$$\llbracket v \rrbracket \eta = \eta v$$
$$\llbracket e_0 e_1 \rrbracket \eta = \phi_{\perp}(\llbracket e_0 \rrbracket \eta)(\llbracket e_1 \rrbracket \eta)$$
$$\llbracket \lambda v. e \rrbracket \eta = \iota_{\perp} \circ \psi(\lambda d \in D. \llbracket e \rrbracket [\eta | v : d])$$

25

Semántica Denotacional del Cálculo Lambda Normal

Pregunta

1. ¿Si $\llbracket e_1 \rrbracket \eta = \perp$, sucede que $\llbracket e_0 e_1 \rrbracket \eta = \perp$?
2. ¿Valen los teoremas de coincidencia, renombre y sustitución?

Corrección de evaluación

Si $e \Rightarrow^N z$, entonces $\llbracket e \rrbracket \eta = \iota_{\perp} v$ y además $\llbracket e \rrbracket \eta = \llbracket z \rrbracket \eta$.

Resulta crucial la validez de la regla β .

Invalidez de la regla η

¿Por qué?

26

Semántica Denotacional Eager

Consideremos $(\lambda x. \lambda y. y)(\Delta \Delta)$.

Mientras que en la evaluación normal tiene un valor, no lo tiene en la evaluación eager.

Consecuentemente necesitamos otro modelo para esta última.

27

Semántica Denotacional Eager

Queremos las siguientes ecuaciones de dominios:

$$D^E = V_{\perp}^E$$

$$V^E \cong [V^E \rightarrow D^E]$$

usaremos el siguiente isomorfismos y funciones:

$$\phi: V^E \rightarrow [V^E \rightarrow D^E]$$

$$\psi: [V^E \rightarrow D^E] \rightarrow V^E$$

con

$$\phi \circ \psi = Id_{[V^E \rightarrow D^E]}$$

$$\psi \circ \phi = Id_{V^E}$$

$$\iota_{\perp}: V^E \rightarrow D^E$$

28

Semántica Denotacional del Cálculo Lambda Eager

$$D^E = V_{\perp}^E$$

$$V^E \cong [V^E \rightarrow D^E]$$

Ambientes: $Env = \langle var \rangle \rightarrow V^E$

Función semántica:

$$\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D^E$$

$$\llbracket v \rrbracket \eta = \iota_{\perp}(\eta v)$$

$$\llbracket e_0 e_1 \rrbracket \eta = (\phi_{\perp}(\llbracket e_0 \rrbracket \eta)) \cdot \llbracket e_1 \rrbracket \eta$$

$$\llbracket \lambda x. e \rrbracket \eta = \iota_{\perp} \circ \psi (\lambda v \in V^E. \llbracket e \rrbracket [\eta | x : v])$$

29

Semántica Denotacional del Cálculo Lambda Eager

Preguntas

- ¿Si $\llbracket e_1 \rrbracket \eta = \perp$, sucede que $\llbracket e_0 e_1 \rrbracket \eta = \perp$?
- ¿Valen los teoremas de coincidencia, renombre y sustitución?

Corrección de evaluación

Si $e \Rightarrow^E z$, entonces $\llbracket e \rrbracket \eta = \iota_{\perp} v$ y además $\llbracket e \rrbracket \eta = \llbracket z \rrbracket \eta$.

Invalidez de la regla β

¿Por qué?

Invalidez de la regla η

¿Por qué?

30



12_aplicat
ivo_con_r...

Lenguajes y Compiladores

Miguel Pagano

2024

1

Estructura de la materia a grandes rasgos:

Primera Parte: Lenguaje imperativo

Segunda Parte: Lenguaje aplicativo puro, y lenguaje aplicativo con referencias y asignación

2

Lenguajes Aplicativos puros

3

Lenguajes Aplicativos puros

Lenguajes Aplicativos

El lenguaje.

$$\begin{aligned}\langle exp \rangle ::= & \langle var \rangle \mid \langle exp \rangle \langle exp \rangle \mid \lambda \langle var \rangle . \langle exp \rangle \\ & \mid \langle natconst \rangle \mid \langle boolconst \rangle \\ & \mid \neg \langle exp \rangle \mid \langle exp \rangle + \langle exp \rangle \mid \langle exp \rangle * \langle exp \rangle \mid \dots \\ & \mid \langle exp \rangle \geq \langle exp \rangle \mid \langle exp \rangle \leq \langle exp \rangle \mid \langle exp \rangle < \langle exp \rangle \mid \dots \\ & \mid \langle exp \rangle \wedge \langle exp \rangle \mid \langle exp \rangle \vee \langle exp \rangle \mid \neg \langle exp \rangle \\ & \mid \text{if } \langle exp \rangle \text{ then } \langle exp \rangle \text{ else } \langle exp \rangle\end{aligned}$$
$$\langle natconst \rangle ::= 0 \mid 1 \mid 2 \mid \dots$$
$$\langle boolconst \rangle ::= \text{true} \mid \text{false}$$

4

Evaluación Eager

Formas canónicas

$$\langle cnf \rangle ::= \langle intcnf \rangle \mid \langle boolcnf \rangle \mid \langle funcnf \rangle$$
$$\langle intcnf \rangle ::= \dots \mid -2 \mid -1 \mid 0 \mid 1 \mid 2 \mid \dots$$
$$\langle boolcnf \rangle ::= \langle boolconst \rangle$$
$$\langle funcnf \rangle ::= \lambda \langle var \rangle . \langle exp \rangle$$

5

Reglas para $\Rightarrow_E$

Notación

- z como una metavariable para $\langle cnf \rangle$,
- c para $\langle intcnf \rangle$ ó para $\langle boolcnf \rangle$,
- i para $\langle intcnf \rangle$.

Regla para las formas canónicas

$$\frac{}{z \Rightarrow_E z}$$

Regla para la aplicación

$$\frac{e \Rightarrow_E \lambda v. e_0 \quad e' \Rightarrow_E z' \quad (e_0/v \mapsto z') \Rightarrow_E z}{ee' \Rightarrow_E z}$$

6

Reglas para $\Rightarrow_E$

Regla para las operaciones enteras y booleanas:

$$\frac{e \Rightarrow_E [i] \quad e' \Rightarrow_E [i']}{e + e' \Rightarrow_E [i + i']}$$

Generalizando:

$$\frac{e \Rightarrow_E [c] \quad e' \Rightarrow_E [c']}{e \oplus e' \Rightarrow_E [c \oplus c']}$$

donde $\oplus \in \{+, -, *, =, \neq, \leq, \geq, \dots\}$ pero $\oplus \notin \{/, \div\}$.

¿Por qué excluimos a la división?

7

Reglas para $\Rightarrow_E$

Regla para las operaciones enteras y booleanas:

$$\frac{e \Rightarrow_E [c]}{\sim e \Rightarrow_E [\sim c]} \text{ con } \sim \in \{-, \neg\}$$

$$\frac{e \Rightarrow_E [i] \quad e' \Rightarrow_E [i']}{e \oplus e' \Rightarrow_E [i \oplus i']} \quad i' \neq 0, \text{ con } \oplus \in \{/, \div\}$$

8

Reglas para $\Rightarrow_E$

$$\frac{e \Rightarrow_E \text{true} \quad e_0 \Rightarrow_E z}{\text{if } e \text{ then } e_0 \text{ else } e_1 \Rightarrow_E z}$$

$$\frac{e \Rightarrow_E \text{false} \quad e_1 \Rightarrow_E z}{\text{if } e \text{ then } e_0 \text{ else } e_1 \Rightarrow_E z}$$

9

Evaluación Normal

Formas Canónicas: Las mismas (por ahora).

Regla para las formas canónicas

$$\frac{}{z \Rightarrow_N z}$$

Regla para la aplicación

$$\frac{e \Rightarrow_N \lambda v. e_0 \quad (e_0/v \mapsto e') \Rightarrow_N z}{ee' \Rightarrow_N z}$$

10

Más reglas para $\Rightarrow_N$

Regla “lazy” para las operaciones booleanas:

$$\frac{e \Rightarrow_N \text{false}}{e \wedge e' \Rightarrow_N \text{false}}$$

Alternativa: uso de abreviaturas

$$\begin{aligned} e \wedge e' &=_{def} \text{if } e \text{ then } e' \text{ else false} \\ e \vee e' &=_{def} \text{if } e \text{ then true else } e' \\ &\vdots \end{aligned}$$

11

Tuplas

$$\langle exp \rangle ::= \dots \mid \langle \langle exp \rangle, \dots, \langle exp \rangle \rangle \mid \langle exp \rangle. \langle tagexp \rangle$$

$$\langle tagexp \rangle ::= \langle natconst \rangle$$

Los componentes de la tupla pueden ser de distintos tipos:

Ejemplo si $M = \langle 2, \lambda n. 1, \lambda f x. f (f x) \rangle$

entonces

$$\lambda M \Omega \ M \ 2 \ M \ 1 \backslash \Rightarrow_{\tau} \lambda x \ (\lambda n \ 1) \ ((\lambda n \ 1) \ x) \backslash$$

$$\langle M.0, M.2 \ M.1 \rangle \Rightarrow_E \langle 2, \lambda x.(\lambda n.1) ((\lambda n.1) x) \rangle$$

$$\langle M.0, M.2 \ M.1 \rangle \Rightarrow_N \langle 2, (\lambda f x.f (f x))(\lambda n.1) \rangle$$

12

Evaluación de las Tuplas

Formas canónicas:

$$\langle cnf \rangle ::= \langle intcnf \rangle \mid \langle boolcnf \rangle \mid \langle funcnf \rangle \mid \langle tuplecnf \rangle$$

Formas canónicas de las tuplas para la evaluación Eager:

$$\langle tuplecnf \rangle ::= \langle \langle cnf \rangle, \dots, \langle cnf \rangle \rangle$$

Formas canónicas de tuplas para la evaluación Normal:

$$\langle tuplecnf \rangle ::= \langle \langle exp \rangle, \dots, \langle exp \rangle \rangle$$

13

Evaluación de las Tuplas (Reglas para Eager)

$$\frac{e_0 \Rightarrow_E z_0 \quad \dots \quad e_{n-1} \Rightarrow_E z_{n-1}}{\langle e_0, \dots, e_{n-1} \rangle \Rightarrow_E \langle z_0, \dots, z_{n-1} \rangle}$$

$$\frac{e \Rightarrow_E \langle z_0, \dots, z_{n-1} \rangle}{e.[k] \Rightarrow_E z_k} \quad k < n$$

14

Evaluación de las Tuplas (Reglas para Normal)

No hay regla para la tupla $\langle e_0, \dots, e_{n-1} \rangle$ porque es una forma canónica.

Dicho de otro modo, la regla es $z \Rightarrow_N z$.

$$\frac{e \Rightarrow_N \langle e_0, \dots, e_{n-1} \rangle \quad e_k \Rightarrow_N z}{e.[k] \Rightarrow_N z} \quad k < n$$

15

Semántica Denotacional Eager

Para el Cálculo Lambda Eager teníamos:

$$D = V_{\perp}$$

$$V \simeq V_{fun} \quad \text{con } V_{fun} = [V \rightarrow D]$$

16

Semántica Denotacional Eager: Dominios

Para el Lenguaje Aplicativo Eager necesitamos más valores:

$$V \stackrel{\phi}{\cong} V_{int} + V_{bool} + V_{fun} \quad \text{y} \quad D = (V + \{\mathbf{error}\} + \{\mathbf{typeerror}\})_{\perp}$$

Los pre-dominios para valores son:

$$V_{int} = \mathbb{Z}$$

$$V_{bool} = \{T, F\}$$

$$V_{fun} = [V \rightarrow D]$$

Notar que V es un predominio: no hay un elemento mínimo!

¿Cómo son las cadenas en V ?

17

Funciones auxiliares

$\iota_{int} \in V_{int} \rightarrow V$ se define como $V_{int} \xrightarrow{\iota_0} V_i + V_b + V_f \xrightarrow{\psi} V$

$$\iota_{int} = \psi \circ \iota_0$$

donde

$$\iota_0 \in V_{int} \rightarrow V_{int} + V_{bool} + V_{fun}$$

Lo mismo para booleanos y funciones:

$$\psi \circ \iota_0 = \iota_{int} \in V_{int} \rightarrow V$$

$$\psi \circ \iota_1 = \iota_{bool} \in V_{bool} \rightarrow V$$

$$\psi \circ \iota_2 = \iota_{fun} \in V_{fun} \rightarrow V$$

18

“Constructores” de D

$$D = (V + \{\text{error}\} + \{\text{typeerror}\})_{\perp}$$

$$\text{err}, \text{tyerr} \in D$$

$$\text{err} = \iota_{\perp}(\iota_1(\text{error}))$$

$$\text{tyerr} = \iota_{\perp}(\iota_2(\text{typeerror}))$$

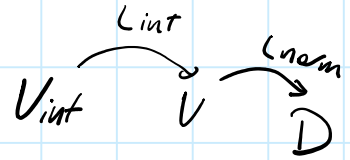
$$\iota_{\text{norm}} \in V \rightarrow D$$

$$\iota_{\text{norm}} = \iota_{\perp} \circ \iota_0$$

Uso habitual: $\iota_{\text{norm}} \circ \iota_{\text{int}} \in V_{\text{int}} \rightarrow D$

Abreviatura: $\iota_{\text{int}} = \iota_{\text{norm}} \circ \iota_{\text{int}} \in V_{\text{int}} \rightarrow D$

En general: $\iota_{\theta} = \iota_{\text{norm}} \circ \iota_{\theta} \in V_{\theta} \rightarrow D$ con $\theta \in \{\text{int}, \text{bool}, \text{fun}\}$.



19

Más funciones auxiliares

Si $f: V \rightarrow D$, entonces $f_*: D \rightarrow D$ se define:

$$f_*(\iota_{\text{norm}} z) = f z$$

$$f_*(\text{err}) = \text{err}$$

$$f_*(\text{tyerr}) = \text{tyerr}$$

$$f_*(\perp) = \perp$$

20

Más funciones auxiliares

Si $f: V_{\text{int}} \rightarrow D$ entonces $f_{\text{int}}: V \rightarrow D$ se define:

$$f_{\text{int}}(\iota_{\text{int}} i) = f i$$

$$f_{\text{int}}(\iota_{\text{bool}} b) = \text{tyerr}$$

$$f_{\text{int}}(\iota_{\text{fun}} f) = \text{tyerr}$$

Si $f: V_{\text{int}} \rightarrow D$, podemos componer las dos transformaciones

$$(f_{\text{int}})_* : D \rightarrow D$$

undo

Si $f: V_{int} \rightarrow D$, podemos componer las dos transformaciones

$$(f_{int})_*: D \rightarrow D$$

que lo escribimos

$$f_{int*}: D \rightarrow D$$

** por que
septe algo normalizado*

21

Más funciones auxiliares

Si $f \in V_\theta \rightarrow D$ entonces $f_\theta \in V \rightarrow D$ se define:

$$\begin{aligned} f_\theta(\iota_\theta x) &= f x \\ f_\theta(\iota_{\theta'} y) &= \text{tyerr} \quad \text{si } \theta \neq \theta' \end{aligned}$$

En general: si $f \in V_\theta \rightarrow D$, entonces

$$\begin{aligned} f_{\theta*} &\in D \rightarrow D \quad \text{satisfaciendo} \\ f_{\theta*}(\iota_\theta x) &= f x \\ f_{\theta*}(\iota_{\theta'} y) &= \text{tyerr} \quad \text{si } \theta \neq \theta' \\ f_{\theta*}(\text{err}) &= \text{err} \\ f_{\theta*}(\text{tyerr}) &= \text{tyerr} \\ f_{\theta*}(\perp) &= \perp \end{aligned}$$

22

Ecuaciones semánticas

$$\begin{aligned} Env &= \langle \text{var} \rangle \rightarrow V \\ \llbracket _ \rrbracket &\in \langle \text{exp} \rangle \rightarrow Env \rightarrow D \end{aligned}$$

$$\begin{aligned} \llbracket 0 \rrbracket \eta &= \iota_{int} 0 \\ \llbracket \text{true} \rrbracket \eta &= \iota_{bool} T \\ \llbracket -e \rrbracket \eta &= (\lambda i \in V_{int}. \iota_{int}(-i))_{int*}(\llbracket e \rrbracket \eta) \\ \llbracket e + e' \rrbracket \eta &= (\lambda i \in V_{int}. (\lambda i' \in V_{int}. \iota_{int}(i + i'))_{int*}(\llbracket e' \rrbracket \eta))_{int*}(\llbracket e \rrbracket \eta) \\ &\vdots \\ \llbracket e / e' \rrbracket \eta &= (\lambda i \in V_{int}. (\lambda i' \in V_{int}. \\ &\quad \text{if } i' = 0 \text{ then err else } \iota_{int}(i / i'))_{int*}(\llbracket e' \rrbracket \eta))_{int*}(\llbracket e \rrbracket \eta) \end{aligned}$$

23

Ecuaciones semánticas

$$Env = \langle \text{var} \rangle \rightarrow V$$

$$Env = \langle var \rangle \rightarrow V$$

$$\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D$$

$$\llbracket v \rrbracket \eta = \iota_{norm}(\eta v)$$

$$\begin{aligned} \llbracket ee' \rrbracket \eta &= (\lambda f \in V_{fun}. (\lambda z \in V. f z)_*(\llbracket e' \rrbracket \eta))_{fun*}(\llbracket e \rrbracket \eta) \\ &= (\lambda f \in V_{fun}. f_*(\llbracket e' \rrbracket \eta))_{fun*}(\llbracket e \rrbracket \eta) \end{aligned}$$

$$\llbracket \lambda v. e \rrbracket \eta = \iota_{fun}(\lambda z \in V. \llbracket e \rrbracket [\eta|v : z])$$

24

Semántica Denotacional Normal

Para el Cálculo Lambda Normal:

$$D = V_{\perp}$$

$$V \simeq V_{fun} \quad \text{con } V_{fun} = [D \rightarrow D]$$

Para el Lenguaje Aplicativo Normal (lo mismo que el eager):

$$D = (V + \{\mathbf{error}\} + \{\mathbf{typeerror}\})_{\perp}$$

$$V \simeq V_{int} + V_{bool} + V_{fun}$$

25

Semántica Denotacional Normal: Valores

$$V \simeq V_{int} + V_{bool} + V_{fun} \quad D = (V + \{\mathbf{error}\} + \{\mathbf{typeerror}\})_{\perp}$$

$$V_{int} = \mathbb{Z}$$

$$V_{bool} = \{T, F\}$$

$$V_{fun} = [D \rightarrow D]$$

$$\phi \in V \rightarrow V_{int} + V_{bool} + V_{fun}$$

$$err, tyerr \in D$$

$$\psi \in V_{int} + V_{bool} + V_{fun} \rightarrow V$$

$$\iota_{\theta} \in V_{\theta} \rightarrow V$$

$$f_{\theta} \in V \rightarrow D$$

$$\iota_{norm} \in V \rightarrow D$$

$$f_* \in D \rightarrow D$$

26

Ecuaciones semánticas

Entornos: $Env = \langle var \rangle \rightarrow D$

Función semántica: $\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D$

La mayoría de las ecuaciones son las mismas. Vamos a señalar aquellas que presentan algún cambio.

Evaluación lazy de las expresiones booleanas:

¿Cuál es el problema con :

$$\llbracket e \vee e' \rrbracket \eta =$$

$$(\lambda b \in V_{bool}. \text{if } b \text{ then } \iota_{bool} T$$

$$\text{else } (\lambda b' \in V_{bool}. \iota_{bool} b')_{bool*} (\llbracket e' \rrbracket \eta)$$

$$)_{bool*} (\llbracket e \rrbracket \eta) ?$$

27

Ecuaciones semánticas

Cálculo lambda:

$$Env = \langle var \rangle \rightarrow V$$
$$\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D$$

$$\begin{aligned}\llbracket v \rrbracket \eta &= \eta v \\ \llbracket ee' \rrbracket \eta &= (\lambda f \in V_{fun}. f \llbracket e' \rrbracket \eta)_{fun*}(\llbracket e \rrbracket \eta) \\ \llbracket \lambda v.e \rrbracket \eta &= \iota_{fun}(\lambda d \in D. \llbracket e \rrbracket [\eta | v : d])\end{aligned}$$

28

Semántica denotacional de las tuplas

$$V \simeq V_{int} + V_{bool} + V_{fun} + V_{tuple}$$

Semántica denotacional Eager:

$$\begin{aligned}V_{tuple} &= V^* = \bigcup_{n \in \mathbb{N}} V^n \\ &= \{ \langle \rangle \} \cup \{ \langle v \rangle | v \in V \} \cup \dots \{ \langle v_1, \dots, v_n \rangle | v_i \in V \}\end{aligned}$$

Notar que $\langle \iota_{tuple} \langle \iota_{int} 2, \iota_{bool} T \rangle \rangle \in V_{tuple}$

Semántica denotacional Normal:

$$V_{tuple} = D^*$$

29

Ecuaciones semánticas Eager

$$\begin{aligned} \llbracket \langle e_0, \dots, e_{n-1} \rangle \rrbracket \eta = & \\ (\lambda z_0 \in V. & \\ (\lambda z_1 \in V. & \\ \dots (\lambda z_{n-1} \in V. & \iota_{tuple} \langle z_0, \dots, z_{n-1} \rangle)_* (\llbracket e_{n-1} \rrbracket) \dots \\)_* (\llbracket e_1 \rrbracket) & \\)_* (\llbracket e_0 \rrbracket) & \eta) \end{aligned}$$

30

Ecuaciones semánticas Eager

$$\begin{aligned} \llbracket e.[k] \rrbracket \eta = & \\ (\lambda t \in V_{tuple}. & \\ \text{if } k \geq |t| \text{ then } & tyerr \text{ else } \iota_{norm} t_k \\)_{tuple_*} (\llbracket e \rrbracket \eta) & \end{aligned}$$

31

Ecuaciones semánticas Normales

$$\llbracket \langle e_0, \dots, e_{n-1} \rangle \rrbracket \eta = \iota_{tuple} \langle \llbracket e_0 \rrbracket \eta, \dots, \llbracket e_{n-1} \rrbracket \eta \rangle$$

$$\llbracket e.[k] \rrbracket \eta =$$

$$(\lambda t \in V_{tuple}.$$

$$\text{if } k \geq |t| \text{ then } tyerr \text{ else } t_k$$

$$)_{tuple*}(\llbracket e \rrbracket \eta)$$

32

Definiciones locales y patrones

$$\langle exp \rangle ::= \text{let } \langle pat \rangle \equiv \langle exp \rangle, \dots, \langle pat \rangle \equiv \langle exp \rangle \text{ in } \langle exp \rangle$$

$$| \lambda \langle pat \rangle. \langle exp \rangle$$

$$\langle pat \rangle ::= \langle var \rangle | \langle \langle pat \rangle, \dots, \langle pat \rangle \rangle$$

Por ejemplo, podemos escribir

$$\text{let } \langle m, \langle f, G \rangle \rangle \equiv \langle 2, \langle \lambda n.1, \lambda fx.f(fx) \rangle \rangle \text{ in } G f$$

en vez de

$$(\lambda t.(t.1.1) (t.1.0)) \langle 2, \langle \lambda n.1, \lambda fx.f(fx) \rangle \rangle$$

33

Definiciones locales y patrones (sin azúcar)

Se definen como abreviaturas (azucar sintactico):

$$\lambda \langle p_1, \dots, p_n \rangle . e \stackrel{def}{=} \lambda v. \mathbf{let} \ p_1 \equiv v.0, \dots, p_n \equiv v.[n-1] \mathbf{in} \ e$$

donde v es una variable nueva (no ocurre libre en e ni en ninguno de los patrones).

$$\mathbf{let} \ p_1 \equiv e_1, \dots, p_n \equiv e_n \mathbf{in} \ e \stackrel{def}{=} (\lambda p_1 \dots \lambda p_n . e) e_1 \dots e_n$$

34

Recursión

$\langle exp \rangle ::= \dots \mid \mathbf{letrec} \ \langle var \rangle \equiv \lambda \langle var \rangle . \langle exp \rangle \mathbf{in} \ \langle exp \rangle$ (Eval. Eager)

$\langle exp \rangle ::= \dots \mid \mathbf{rec} \ \langle exp \rangle$ (Eval. Normal)

El letrec permite hacer definiciones recursivas como

$\mathbf{letrec} \ \mathbf{fact} \equiv \lambda n. \mathbf{if} \ n = 0 \mathbf{then} \ 1 \mathbf{else} \ n * \mathbf{fact}(n - 1) \mathbf{in} \ \mathbf{fact} \ 2$

35

Regla para la Evaluación Eager

$$\frac{(e/f \mapsto \lambda u. e_0^*) \Rightarrow_E z}{\text{letrec } f \equiv \lambda u. e_0 \text{ in } e \Rightarrow_E z}$$

donde $e_0^* = \text{letrec } f \equiv \lambda u. e_0 \text{ in } e_0$

36

Regla para la Evaluación Normal

$$\frac{e (\text{rec } e) \Rightarrow_N z}{\text{rec } e \Rightarrow_N z}$$

37

Semántica Denotacional Eager (Recursión)

$$\llbracket \text{letrec } v \equiv \lambda u. e_0 \text{ in } e \rrbracket \eta = \llbracket e \rrbracket [\eta | v : \iota_{fun} g]$$

donde g es el menor punto fijo de

wtf!!

$$F f z = \llbracket e \rrbracket [\eta | v : \iota_{fun} f, u : z]$$

o sea

$$g = \mathbf{Y}_{V_{fun}} F \quad \mathbf{Y}_{V_{fun}} F = \sqcup_i F^i \perp$$

$$g = \mathbf{Y}_{V_{fun}} (\lambda f \in V_{fun}. \lambda z \in V. \llbracket e \rrbracket [\eta | v : \iota_{fun} f, u : z])$$

38

Semántica Denotacional Normal (Recursión)

$$\llbracket \text{rec } e \rrbracket \eta = (\lambda f \in V_{fun}. \mathbf{Y}_D f)_{fun*} (\llbracket e \rrbracket \eta)$$

donde $\mathbf{Y}_D$ es el operador de menor punto fijo

$$\mathbf{Y}_D f = \sqcup_i f^i \perp$$

39



13_iswim

Lenguajes y Compiladores

Lenguajes Iswim-like

2023

1

Estructura de la materia a grandes rasgos:

Primera Parte: Lenguaje imperativo

Segunda Parte: Lenguaje aplicativo puro, y lenguaje aplicativo con referencias y asignación

2

Ejes de contenidos de la segunda parte

Cálculo Lambda

Lenguajes Aplicativos puros

Un Lenguaje Aplicativo con referencias y asignación

3

Cálculo Lambda

Lenguajes Aplicativos puros

Lenguajes Aplicativos

$\langle exp \rangle ::= \langle var \rangle \mid \langle exp \rangle \langle exp \rangle \mid \lambda \langle var \rangle . \langle exp \rangle$
| **letrec** $\langle var \rangle \equiv \lambda \langle var \rangle . \langle exp \rangle$ **in** $\langle exp \rangle$
| $\langle natconst \rangle \mid \langle boolconst \rangle$
| $\neg \langle exp \rangle \mid \langle exp \rangle + \langle exp \rangle \mid \langle exp \rangle * \langle exp \rangle \mid \dots$
| $\langle exp \rangle \geq \langle exp \rangle \mid \langle exp \rangle \leq \langle exp \rangle \mid \langle exp \rangle < \langle exp \rangle \mid \dots$
| $\langle exp \rangle \wedge \langle exp \rangle \mid \langle exp \rangle \vee \langle exp \rangle \mid \neg \langle exp \rangle$
| **if** $\langle exp \rangle$ **then** $\langle exp \rangle$ **else** $\langle exp \rangle$

$\langle natconst \rangle ::= 0 \mid 1 \mid 2 \mid \dots$

$\langle boolconst \rangle ::= \text{true} \mid \text{false}$

Un Lenguaje Aplicativo con referencias y asignación

Lenguaje(s) Iswim-like

En los '60 Peter Landin utilizaba el cálculo lambda para dar la semántica de distintos lenguajes. Además definió la máquina abstracta SECD para la evaluación de expresiones (y explicó como programar la máquina abstracta).

Introdujo la noción de estado para explicar la asignación. Casi textualmente: “El significado de una IAE tiene dos aspectos, el descriptivo tiene que ver con el valor de la expresión; el imperativo con el cambio del estado de la máquina al ejecutar la IAE”.

El lenguaje Iswim propone incorporar una componente imperativa a un lenguaje aplicativo eager mediante **las referencias** como un tipo más de valores, y que por lo tanto pueden ser devueltos por una función, o pasados como valor que recibe una función.

Lenguaje Iswim

El lenguaje Iswim extiende al lenguaje aplicativo eager mediante las siguientes construcciones:

```
 $\langle exp \rangle ::= \dots$   
| ref  $\langle exp \rangle$   
| val  $\langle exp \rangle$   
|  $\langle exp \rangle := \langle exp \rangle$   
|  $\langle exp \rangle =_{ref} \langle exp \rangle$ 
```

Mientras que para las formas canónicas agregamos *referencias*:

```
 $\langle cnf \rangle ::= \dots \mid V_{ref}$ 
```

La evaluación será una combinación de evaluación de aplicativo con semántica de transiciones de imperativo.

6

Intuición de nuevas expresiones. Crear referencia.

La expresión `ref e` extiende el estado generando una locación nueva que aloja el valor producido por `e` (no hay restricciones para el valor que puede adquirir `e`)

7

Intuición de nuevas expresiones. Dereferencia.

La expresión `val e` estará definida cuando `e` produzca un valor de tipo `referencia`, y la expresión completa devolverá lo alojado en esa referencia.

8

Intuición de nuevas expresiones. Asignación.

La expresión `e := e'` produce la modificación en el estado producto de la asignación (`e` debe producir un valor de tipo `referencia`), devolviendo un valor especial que llamaremos `unit` (esto es arbitrario).

9

Intuición de nuevas expresiones. Comprobación de igualdad.

La expresión $e =_{ref} e'$ comprueba que si ambas expresiones producen (evalúan a) referencias r y r' , respectivamente, ambas sean la misma; es decir es una forma de chequear igualdad de punteros.

10

Fragmento imperativo: skip

Podemos **definir** construcciones del lenguaje imperativo como la iteración y la declaración de variables locales sin extender la sintaxis abstracta. El orden de evaluación eager permite obtenerlas como azucar sintáctico.

$skip =_{def} \langle \rangle$

11

Fragmento imperativo: secuencia

Dado que una frase de tipo $\langle exp \rangle$ tiene el potencial de simultáneamente producir un efecto en el estado y a su vez un valor, podemos modelar la secuencia de comandos a través de un `let` en el cual el valor producido por la expresión se descarte:

$$e; e' =_{def} \text{let } v = e \text{ in } e' \quad (v \notin FV e')$$

Note que aquí es fundamental el orden de evaluación eager para que la secuencia de ejecución de los comandos involucrados se respete.

12

Fragmento imperativo: newvar

Ampliar el ambiente con una nueva variable que denote una referencia también puede ser expresado mediante un `let`:

$$\text{newvar } v := e \text{ in } e' =_{def} \text{let } v = \text{ref } e \text{ in } e'$$

Agotado el alcance de la declaración local, la referencia al lugar de memoria eventualmente se pierde (pero el lugar de memoria sigue definido). La restauración de v está dada por la semántica de la secuencia.

13

Fragmento imperativo: while

La iteración es un tipo espacial de declaración de función:

$\text{while } e \text{ do } e' =_{def} \text{letrec } f = \lambda v. \text{if } e \text{ then } e'; f v \text{ else skip in } f \langle \rangle$

Aquí las variables f y v no deben ocurrir en e ni e' .

14

Noción de Estado en Iswim

Suponemos la existencia de un conjunto infinito de locaciones, llamado V_{ref} , de memoria que siempre alojan valores.

El conjunto de estados Σ está formado por funciones parciales definidas en un subconjunto finito de V_{ref} .

$$\Sigma = \bigcup_{F \subset_{fin} V_{ref}} F \rightarrow \langle cnf \rangle$$

Para $F \subset_{fin} V_{ref}$, $new(F) \in V_{ref}$ nos provee de una referencia con la propiedad $new(F) \notin F$. Por simplicidad sea $new(\sigma) = new(dom(\sigma))$.

15

Semántica operacional de Iswim

Formas Canónicas:

$$\langle \text{cnf} \rangle ::= \dots \mid V_{\text{ref}}$$

Semántica big-step (evaluación):

$$\sigma, e \Rightarrow z, \sigma'$$

Se define a través de reglas (axiomáticamente)

16

Evaluación del fragmento aplicativo

Todas las reglas aplicativas del lenguaje eager se incorporan con la indicación explícita de cómo se transforma el estado.

Por ejemplo la regla

$$\frac{e \Rightarrow_E \lambda v. e_0 \quad e' \Rightarrow_E z' \quad (e_0/v \mapsto z') \Rightarrow_E z}{ee' \Rightarrow_E z}$$

se transforma en:

$$\frac{\sigma, e \Rightarrow_E \lambda v. e_0, \sigma' \quad \sigma', e' \Rightarrow_E z', \sigma'' \quad \sigma'', (e_0/v \mapsto z') \Rightarrow_E z, \hat{\sigma}}{\sigma, e e' \Rightarrow_E z, \hat{\sigma}}$$

17

Reglas para el fragmento imperativo

$$\frac{\sigma, e \Rightarrow r, \sigma' \quad \sigma', e' \Rightarrow z', \sigma''}{\sigma, e := e' \Rightarrow z', [\sigma'' \mid r : z']}$$

$$\frac{\sigma, e \Rightarrow z, \sigma'}{\sigma, \text{ref } e \Rightarrow r, [\sigma' \mid r : z]} \quad r = \text{new}(\sigma')$$

$$\frac{\sigma, e \Rightarrow r, \sigma'}{\sigma, \text{val } e \Rightarrow \sigma' r, \sigma'} \quad r \in \text{dom}(\sigma')$$

$$\frac{\sigma, e \Rightarrow r, \sigma'}{\sigma, \text{val } e \Rightarrow \sigma' r, \sigma'} \quad r \in \text{dom}(\sigma')$$

$$\frac{\sigma, e \Rightarrow r, \sigma' \quad \sigma', e' \Rightarrow r', \sigma''}{\sigma, e =_{\text{ref}} e' \Rightarrow [r = r'], \sigma''}$$

18

Dominios para Semántica denotacional

Para la semántica denotacional tenemos estados “denotacionales”:

$$\Sigma = \bigcup_{F \subset_{\text{fin}} V_{\text{ref}}} F \rightarrow V$$

$$D = (\Sigma \times V + \{\text{error}, \text{typeerror}\})_{\perp} \quad \text{donde}$$

$$V = V_{\text{int}} + V_{\text{bool}} + V_{\text{fun}} + V_{\text{tuple}} + V_{\text{ref}}$$

Las funciones ahora deben reflejar la posibilidad de cambio del estado, es decir, una función no sólo toma el valor sino además el estado producido por la evaluación del operando. Por ello:

$$V_{\text{fun}} = \Sigma \times V \rightarrow D$$

Como antes, los valores $\text{err}, \text{tyerr} \in D$ constituyen la denotación de los errores.

19

Funciones auxiliares

Si $f \in \Sigma \times V \rightarrow D$, entonces $f_* \in D \rightarrow D$ se define:

$$f_*(\iota_{\text{norm}} \langle \sigma, z \rangle) = f(\langle \sigma, z \rangle)$$

$$f_*(\text{err}) = \text{err}$$

$$f_*(\text{tyerr}) = \text{tyerr}$$

$$f_*(\perp) = \perp$$

20

Funciones auxiliares

Por otro lado, si $f \in \Sigma \times V_{int} \rightarrow D$, entonces $f_{int} \in \Sigma \times V \rightarrow D$ se define:

$$\begin{aligned} f_{int}(\langle \sigma, \iota_{int} k \rangle) &= f(\langle \sigma, k \rangle) \\ f_{int}(\langle \sigma, \iota_{\theta} z \rangle) &= tyerr \quad \text{si } \theta \neq int \end{aligned}$$

De manera similar se definen los operadores $(_)_{bool}$, $(_)_{fun}$, etc.

21

Función semántica

La función semántica será de tipo:

$$\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow \Sigma \rightarrow D$$

22

Ecuaciones semánticas

La semántica de las construcciones típicamente imperativas es la siguiente:

$$\llbracket \text{val } e \rrbracket_{\eta\sigma} = (\lambda \langle \sigma', r \rangle. \text{if } r \in \text{dom}(\sigma') \\ \text{then } \iota_{\text{norm}} \langle \sigma', \sigma' r \rangle \text{else } \text{err} \rangle_{\text{ref}*} (\llbracket e \rrbracket_{\eta\sigma}))$$

$$\llbracket \text{ref } e \rrbracket_{\eta\sigma} = (\lambda \langle \sigma', z \rangle. \iota_{\text{norm}} \langle [\sigma' \mid r_{\sigma'} : z], \iota_{\text{ref}} r_{\sigma'} \rangle \rangle_* (\llbracket e \rrbracket_{\eta\sigma})) \\ \text{donde } (r_{\sigma'} = \text{new}(\sigma'))$$

23

Ecuaciones semánticas

$$\llbracket e := e' \rrbracket_{\eta\sigma} = (\lambda \langle \sigma', r \rangle. (\lambda \langle \sigma'', z \rangle. \iota_{\text{norm}} \langle [\sigma'' \mid r : z], z \rangle \rangle_* (\llbracket e' \rrbracket_{\eta\sigma'}))_{\text{ref}*} \\ (\llbracket e \rrbracket_{\eta\sigma}))$$

$$\llbracket e =_{\text{ref}} e' \rrbracket_{\eta\sigma} = (\lambda \langle \sigma', r \rangle. (\lambda \langle \sigma'', z \rangle. \iota_{\text{norm}} \langle \sigma'', \iota_{\text{bool}} r = r' \rangle \rangle_{\text{ref}*} \\ (\llbracket e' \rrbracket_{\eta\sigma'}))_{\text{ref}*} (\llbracket e \rrbracket_{\eta\sigma}))$$

24

Ecuaciones semánticas: fragmento puro

$$\llbracket 0 \rrbracket_{\eta\sigma} = \iota_{\text{norm}} \langle \sigma, \iota_{\text{int}} 0 \rangle$$

$$\llbracket \text{true} \rrbracket_{\eta\sigma} = \iota_{\text{norm}} \langle \sigma, \iota_{\text{bool}} T \rangle$$

$$\begin{aligned} \llbracket -e \rrbracket_{\eta\sigma} &= (\lambda \langle \sigma', i \rangle. \iota_{\text{norm}} \langle \sigma', \iota_{\text{int}} -i \rangle)_{\text{int}*} (\llbracket e \rrbracket_{\eta\sigma}) \\ \llbracket e + e' \rrbracket_{\eta\sigma} &= (\lambda \langle \sigma', i \rangle \\ &\quad (\lambda \langle \sigma'', j \rangle. \iota_{\text{norm}} \langle \sigma'', \iota_{\text{int}} i + j \rangle)_{\text{int}*} (\llbracket e' \rrbracket_{\eta\sigma'}))_{\text{int}*} (\llbracket e \rrbracket_{\eta\sigma}) \end{aligned}$$

25

Ecuaciones semánticas: operadores del cálculo lambda

$$\llbracket v \rrbracket_{\eta\sigma} = \iota_{\text{norm}} \langle \sigma, \eta v \rangle$$

$$\begin{aligned} \llbracket e e' \rrbracket_{\eta\sigma} &= (\lambda \langle \sigma', f \rangle. f_* (\llbracket e' \rrbracket_{\eta\sigma'}))_{\text{fun}*} (\llbracket e \rrbracket_{\eta\sigma}) \\ \llbracket \lambda v. e \rrbracket_{\eta\sigma} &= \iota_{\text{norm}} \langle \sigma, \iota_{\text{fun}} (\lambda \langle \sigma', z \rangle. \llbracket e \rrbracket_{\eta [v : z] \sigma'}) \rangle \end{aligned}$$

26

$$\llbracket \text{letrec } w = \lambda v. e \text{ in } e' \rrbracket \eta \sigma = \llbracket e' \rrbracket [\eta \mid w : \iota_{fun} g] \sigma$$

donde

$$g = \mathbf{Y}_{V_{fun}} F$$

$$F f \langle \sigma', z \rangle = \llbracket e \rrbracket [\eta \mid w : \iota_{fun} f \mid v : z] \sigma'$$

27

Algunas propiedades del fragmento imperativo

Dado que una frase de tipo $\langle exp \rangle$ tiene el potencial de simultaneamente producir un efecto en el estado y a su vez un valor, podemos modelar la secuencia de comandos a través de un let en el cual el valor producido por la expresión se descarte:

$$e; e' =_{def} \text{let } v = e \text{ in } e' \quad (v \notin FV e')$$

La semántica operacional nos permite verificar el significado esperado:

$$\frac{\sigma, e \Rightarrow z, \sigma' \quad \sigma', e' \Rightarrow z', \sigma''}{\sigma, e; e' \Rightarrow z', \sigma''}$$

Note que la regla pone en evidencia que el valor z producido al evaluar e es descartado.

28

Lema Si $\sigma, e \Rightarrow z, \sigma'$ y $\sigma', e' \Rightarrow z', \sigma''$, entonces

$$\sigma, e; e' \Rightarrow z', \sigma''$$

Lema $\llbracket e; e' \rrbracket \eta \sigma = (\lambda \langle \sigma', z \rangle. \llbracket e' \rrbracket \eta \sigma')_* (\llbracket e \rrbracket \eta \sigma)$

Composicion

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket \perp (\llbracket c_0 \rrbracket \sigma)$$

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (\text{rest}_{v,\sigma}) \perp (\llbracket c \rrbracket [\sigma|v : \llbracket e \rrbracket \sigma])$$

Con notación lambda:

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (\lambda \sigma' \in \Sigma. [\sigma' | v : \sigma v]) \perp (\llbracket c \rrbracket [\sigma|v : \llbracket e \rrbracket \sigma])$$

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ \llbracket \text{while } b \text{ do } c \rrbracket \perp (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

y encontrar su menor punto fijo.

$$F_{b,c} f \sigma = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f \perp (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

Definición de sustitución

$$\begin{aligned} \text{skip} / \delta &= \text{skip} \\ v := e / \delta &= \delta v := e / \delta \\ (c_0; c_1) / \delta &= c_0 / \delta; c_1 / \delta \\ (\text{if } b \text{ then } c_0 \text{ else } c_1) / \delta &= \text{if } b / \delta \text{ then } c_0 / \delta \text{ else } c_1 / \delta \\ (\text{while } b \text{ do } c) / \delta &= \text{while } (b / \delta) \text{ do } (c_0 / \delta) \\ (\text{newvar } v := e \text{ in } c) / \delta &= \text{newvar } v_{\text{new}} := e / \delta \text{ in } c / [\delta | v : v_{\text{new}}] \end{aligned}$$

donde $v_{\text{new}} \notin \{\delta w \mid w \in FV(c) - \{v\}\}$

Fallas

$$\llbracket \text{skip} \rrbracket \sigma = \sigma$$

$$\llbracket v := e \rrbracket \sigma = [\sigma | v : \llbracket e \rrbracket \sigma]$$

$$\llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma = \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases}$$

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket \ast (\llbracket c_0 \rrbracket \sigma)$$

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (\text{rest}_{v,\sigma}) \ast (\llbracket c \rrbracket [\sigma|v : \llbracket e \rrbracket \sigma])$$

$$F_{b,c} : (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma \rightarrow \Sigma')$$

$$F_{b,c}(f)(\sigma) = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f \ast (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \left(\bigcup_{i \in \mathbb{N}} F^i \perp \right) \sigma$$

Sintaxis

$\langle \text{comm} \rangle ::= \dots \mid \text{fail} \mid \text{catchin } \langle \text{comm} \rangle \text{ with } \langle \text{comm} \rangle$

Semántica

$$\llbracket _ \rrbracket : \langle \text{comm} \rangle \rightarrow (\Sigma \rightarrow \Sigma')$$

$$\llbracket \text{catchin } c_0 \text{ with } c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket \ast (\llbracket c_0 \rrbracket \sigma)$$

donde $\ast$ está definido por

$$(_)\ast : (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma' \rightarrow \Sigma')$$

$$f\ast(\perp) = \perp$$

$$f\ast(\llbracket \text{abort}, \sigma \rrbracket) = f(\sigma)$$

$$f\ast(\sigma) = \sigma$$

La semántica denotacional daba el significado de los comandos como una función de estados en "comportamientos":

Semántica Operacional

- La semántica **operacional** explicita esos estados intermedios.
- La ejecución de un programa es una secuencia de configuraciones:

$$\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \dots$$

$$(\text{skip}) \frac{}{\llbracket \text{skip}, \sigma \rrbracket \rightarrow \sigma}$$

$$(\text{assign}) \frac{}{\llbracket x := e, \sigma \rrbracket \rightarrow \llbracket x := \llbracket e \rrbracket \sigma, \sigma \rrbracket}$$

$$(\text{seq-ok}) \frac{\langle c_0, \sigma \rangle \rightarrow \sigma'}{\langle c_0; c_1, \sigma \rangle \rightarrow \langle c_1, \sigma' \rangle}$$

$$(\text{seq-ab}) \frac{\langle c_0, \sigma \rangle \rightarrow \langle \text{abort}, \sigma' \rangle}{\langle c_0; c_1, \sigma \rangle \rightarrow \langle \text{abort}, \sigma' \rangle}$$

• La semántica **operacional** explicita esos estados intermedios.

• La ejecución de un programa es una secuencia de configuraciones:

$$\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \dots$$

- Una configuración, γ , puede ser **terminal**: un estado o un estado fallido;
- o puede ser **no-terminal**: un comando (lo que falta de ejecutar) y un estado.

$$(\text{refl}) \frac{}{\gamma \rightarrow^* \gamma}$$

$$(\text{paso}) \frac{\gamma \rightarrow \gamma'}{\gamma \rightarrow^* \gamma'}$$

$$(\text{trans}) \frac{\gamma_1 \rightarrow^* \gamma_2 \quad \gamma_2 \rightarrow^* \gamma_3}{\gamma_1 \rightarrow^* \gamma_3}$$

$$\llbracket _ \rrbracket : \langle \text{comm} \rangle \rightarrow \Sigma \rightarrow \Sigma'$$

$$\llbracket c \rrbracket \sigma = \begin{cases} \perp & \text{si } \langle c, \sigma \rangle \uparrow \\ \gamma & \text{si } \langle c, \sigma \rangle \rightarrow^* \gamma \in \Gamma_T \end{cases}$$

Teorema de Corrección

Para todo comando c , $\llbracket c \rrbracket = \llbracket c \rrbracket$.

$$\llbracket _ \rrbracket : \langle \text{comm} \rangle \rightarrow (\Sigma \rightarrow \Omega)$$

$$\llbracket e \rrbracket \sigma = \langle \llbracket e \rrbracket \sigma, \sigma \rangle$$

$$\llbracket \text{skip} \rrbracket \sigma = \langle \sigma \rangle$$

$$\llbracket v := e \rrbracket \sigma = \langle \sigma \mid v : \llbracket e \rrbracket \sigma \rangle$$

$$\llbracket \text{if } b \text{ then } c \text{ else } c' \rrbracket \sigma = \begin{cases} \llbracket c \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c' \rrbracket \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases}$$

$$\llbracket c_0; c_1 \rrbracket \sigma = \llbracket c_1 \rrbracket \bullet (\llbracket c_0 \rrbracket \sigma)$$

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (\text{rest}_{v, \sigma}) \dagger (\llbracket c \rrbracket [\sigma \mid v : \llbracket e \rrbracket \sigma])$$

$$F_{b,c} : (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma \rightarrow \Sigma')$$

$$F_{b,c}(f)(\sigma) = \begin{cases} \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \\ f_{\star}(\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \end{cases}$$

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \left(\bigsqcup_{i \in \mathbb{N}} F^i \perp \right) \sigma$$

Propagación de errores

$$(_)\star : (\Sigma \rightarrow \Sigma') \rightarrow (\Sigma' \rightarrow \Sigma')$$

$$f_{\star}(\perp) = \perp$$

$$f_{\star}(\langle \text{abort}, \sigma \rangle) = \langle \text{abort}, \sigma \rangle$$

$$f_{\star}(\sigma) = f(\sigma)$$

Como map para listas

$$(_)\dagger : (\Sigma \rightarrow \Sigma) \rightarrow (\Sigma' \rightarrow \Sigma')$$

$$f_{\dagger}(\perp) = \perp$$

$$f_{\dagger}(\langle \text{abort}, \sigma \rangle) = \langle \text{abort}, f(\sigma) \rangle$$

$$f_{\dagger}(\sigma) = f(\sigma)$$

$$(\text{skip}) \frac{}{\langle \text{skip}, \sigma \rangle \rightarrow \sigma}$$

$$(\text{assign}) \frac{}{\langle v := e, \sigma \rangle \rightarrow \langle \sigma \mid v : \llbracket e \rrbracket \sigma \rangle}$$

$$\text{fail} \frac{}{\langle \text{fail}, \sigma \rangle \rightarrow \langle \text{abort}, \sigma \rangle}$$

$$(\text{if-true}) \frac{}{\langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle \rightarrow \langle c_0, \sigma \rangle} \text{ si } \llbracket b \rrbracket \sigma = T$$

$$(\text{if-false}) \frac{}{\langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle \rightarrow \langle c_1, \sigma \rangle} \text{ si } \llbracket b \rrbracket \sigma = F$$

$$(\text{while-false}) \frac{}{\langle \text{while } b \text{ do } c, \sigma \rangle \rightarrow \sigma} \text{ si } \llbracket b \rrbracket \sigma = F$$

$$(\text{while-true}) \frac{}{\langle \text{while } b \text{ do } c, \sigma \rangle \rightarrow \langle c; \text{while } b \text{ do } c, \sigma \rangle} \text{ si } \llbracket b \rrbracket \sigma = T$$

$$(\text{loc-ok}) \frac{}{\langle c, [\sigma \mid v : \llbracket e \rrbracket \sigma] \rangle \rightarrow \sigma'} \frac{}{\langle \text{newvar } v := e \text{ in } c, \sigma \rangle \rightarrow \langle \sigma' \mid v : \sigma v \rangle}$$

$$(\text{loc-abort}) \frac{}{\langle c, [\sigma \mid v : \llbracket e \rrbracket \sigma] \rangle \rightarrow \langle \text{abort}, \sigma' \rangle} \frac{}{\langle \text{newvar } v := e \text{ in } c, \sigma \rangle \rightarrow \langle \text{abort}, [\sigma' \mid v : \sigma v] \rangle}$$

$$(\text{loc-nt}) \frac{}{\langle c, [\sigma \mid v : \llbracket e \rrbracket \sigma] \rangle \rightarrow \langle c', \sigma' \rangle} \frac{}{\langle \text{newvar } v := e \text{ in } c, \sigma \rangle \rightarrow \langle \text{newvar } v := \sigma' v \text{ in } c', [\sigma' \mid v : \sigma v] \rangle}$$

$$(\text{catch-ok}) \frac{}{\langle c, \sigma \rangle \rightarrow \sigma'} \frac{}{\langle \text{catchin } c \text{ with } \hat{c}, \sigma \rangle \rightarrow \sigma'}$$

$$(\text{catch-abort}) \frac{}{\langle c, \sigma \rangle \rightarrow \langle \text{abort}, \sigma' \rangle} \frac{}{\langle \text{catchin } c \text{ with } \hat{c}, \sigma \rangle \rightarrow \langle \hat{c}, \sigma' \rangle}$$

$$(\text{catch-nt}) \frac{}{\langle c, \sigma \rangle \rightarrow \langle c', \sigma' \rangle} \frac{}{\langle \text{catchin } c \text{ with } \hat{c}, \sigma \rangle \rightarrow \langle \text{catchin } c' \text{ with } \hat{c}, \sigma' \rangle}$$

$$(_)\star : (\Sigma \rightarrow \Omega) \rightarrow (\Omega \rightarrow \Omega)$$

$$f_{\star}(\langle n_0, \dots, n_{k-1} \rangle) = \langle n_0, \dots, n_{k-1} \rangle$$

$$f_{\star}(\langle n_0, \dots, n_{k-1}, \sigma \rangle) = \langle n_0, \dots, n_{k-1} \rangle \uparrow\uparrow f(\sigma)$$

$$f_{\star}(\langle n_0, \dots, n_{k-1}, \langle \text{abort}, \sigma \rangle \rangle) = \langle n_0, \dots, n_{k-1}, \langle \text{abort}, \sigma \rangle \rangle$$

$$f_{\star}(\langle n_0, \dots, n_{k-1}, \dots \rangle) = \langle n_0, \dots, n_{k-1}, \dots \rangle$$

$$(_)\dagger : (\Sigma \rightarrow \Omega) \rightarrow (\Omega \rightarrow \Omega)$$

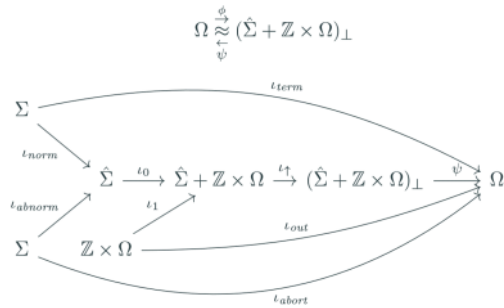
$$f_{\dagger}(\langle n_0, \dots, n_{k-1} \rangle) = \langle n_0, \dots, n_{k-1} \rangle$$

$$f_{\dagger}(\langle n_0, \dots, n_{k-1}, \sigma \rangle) = \langle n_0, \dots, n_{k-1}, f(\sigma) \rangle$$

$$f_{\dagger}(\langle n_0, \dots, n_{k-1}, \langle \text{abort}, \sigma \rangle \rangle) = \langle n_0, \dots, n_{k-1}, \langle \text{abort}, f(\sigma) \rangle \rangle$$

$$f_{\dagger}(\langle n_0, \dots, n_{k-1}, \dots \rangle) = \langle n_0, \dots, n_{k-1}, \dots \rangle$$

$$\begin{aligned} \iota_{\perp} &: \{\langle \rangle\} \rightarrow \Omega \\ \iota_{\perp}(\langle \rangle) &= \{\} \\ \iota_{\text{abort}} &: \Sigma \rightarrow \Omega \\ \iota_{\text{abort}}(\sigma) &= \{\sigma\} \\ \iota_{\text{term}} &: \Sigma \rightarrow \Omega \\ \iota_{\text{term}}(\bar{\sigma}) &= \langle \bar{\sigma} \rangle \\ \iota_{\text{out}} &: \mathbb{Z} \times \Omega \rightarrow \Omega \\ \iota_{\text{out}}(\langle n, \omega \rangle) &= \langle n \rangle \uparrow \omega \end{aligned}$$



$$\begin{aligned} (_)* &: (\Sigma \rightarrow \Omega) \rightarrow (\Omega \rightarrow \Omega) \\ f_*(\perp) &= \perp \\ f_*(\iota_{\text{term}}(\sigma)) &= f(\sigma) \\ f_*(\iota_{\text{abort}}(\sigma)) &= \iota_{\text{abort}}(\sigma) \\ f_*(\iota_{\text{out}}(\langle n, \omega \rangle)) &= \iota_{\text{out}}(\langle n, f_*(\omega) \rangle) \end{aligned}$$

$$\begin{aligned} (_)* &: (\Sigma \rightarrow \Omega) \rightarrow (\Omega \rightarrow \Omega) \\ f_*(\perp) &= \perp \\ f_*(\iota_{\text{term}}(\sigma)) &= f(\sigma) \\ f_*(\iota_{\text{abort}}(\sigma)) &= \iota_{\text{abort}}(\sigma) \\ f_*(\iota_{\text{out}}(\langle n, \omega \rangle)) &= \iota_{\text{out}}(\langle n, f_*(\omega) \rangle) \end{aligned}$$

$$\begin{aligned} (_)_{\dagger} &: (\Sigma \rightarrow \Sigma) \rightarrow (\Omega \rightarrow \Omega) \\ f_{\dagger}(\perp) &= \perp \\ f_{\dagger}(\iota_{\text{term}}(\sigma)) &= \iota_{\text{term}}(f(\sigma)) \\ f_{\dagger}(\iota_{\text{abort}}(\sigma)) &= \iota_{\text{abort}}(f(\sigma)) \\ f_{\dagger}(\iota_{\text{out}}(\langle n, \omega \rangle)) &= \iota_{\text{out}}(\langle n, f_{\dagger}(\omega) \rangle) \end{aligned}$$

$$\begin{aligned} \iota_{\text{term}} &= \psi \circ \iota_{\uparrow} \circ \iota_0 \circ \iota_{\text{norm}} \\ \iota_{\text{abort}} &= \psi \circ \iota_{\uparrow} \circ \iota_0 \circ \iota_{\text{abnorm}} \\ \iota_{\text{out}} &= \psi \circ \iota_{\uparrow} \circ \iota_1 \end{aligned}$$

$$\Omega \xrightarrow[\psi]{\psi} (\hat{\Sigma} + \mathbb{Z} \times \Omega + \mathbb{Z} \rightarrow \Omega)_{\perp}$$

$$\begin{aligned} \iota_{\text{term}} &= \psi \circ \iota_{\uparrow} \circ \iota_0 \circ \iota_{\text{norm}} \\ \iota_{\text{abort}} &= \psi \circ \iota_{\uparrow} \circ \iota_0 \circ \iota_{\text{abnorm}} \\ \iota_{\text{out}} &= \psi \circ \iota_{\uparrow} \circ \iota_1 \\ \iota_{\text{in}} &= \psi \circ \iota_{\uparrow} \circ \iota_2 : (\mathbb{Z} \rightarrow \Omega) \rightarrow \Omega \end{aligned}$$

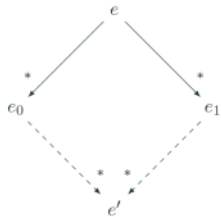
$$\begin{aligned} (_)* &: (\Sigma \rightarrow \Omega) \rightarrow (\Omega \rightarrow \Omega) \\ f_*(\perp) &= \perp \\ f_*(\iota_{\text{term}}(\sigma)) &= f(\sigma) \\ f_*(\iota_{\text{abort}}(\sigma)) &= \iota_{\text{abort}}(\sigma) \\ f_*(\iota_{\text{out}}(\langle n, \omega \rangle)) &= \iota_{\text{out}}(\langle n, f_*(\omega) \rangle) \\ f_*(\iota_{\text{in}}(g)) &= \iota_{\text{in}}(\lambda n \in \mathbb{Z}. f_*(g(n))) \end{aligned}$$

$$\begin{aligned} (_)_{\dagger} &: (\Sigma \rightarrow \Sigma) \rightarrow (\Omega \rightarrow \Omega) \\ f_{\dagger}(\perp) &= \perp \\ f_{\dagger}(\iota_{\text{term}}(\sigma)) &= \iota_{\text{term}}(f(\sigma)) \\ f_{\dagger}(\iota_{\text{abort}}(\sigma)) &= \iota_{\text{abort}}(f(\sigma)) \\ f_{\dagger}(\iota_{\text{out}}(\langle n, \omega \rangle)) &= \iota_{\text{out}}(\langle n, f_{\dagger}(\omega) \rangle) \\ f_{\dagger}(\iota_{\text{in}}(g)) &= \iota_{\text{in}}(\lambda n \in \mathbb{Z}. f_{\dagger}(g(n))) \end{aligned}$$

Forma normal: expresión sin r dices.

$$\Delta = \lambda x. xx$$

Teorema de Church-Rosser Si $e \rightarrow^* e_0$ y $e \rightarrow^* e_1$, entonces existe e' tal que $e_0 \rightarrow^* e'$ y $e_1 \rightarrow^* e'$.



$$\lambda v. e \Rightarrow_N \lambda v. e$$

$$\lambda v. e \Rightarrow_E \lambda v. e$$

Regla para la aplicaci n

$$\frac{e \Rightarrow_N \lambda v. e_0 \quad (e_0/v \mapsto e') \Rightarrow_N z}{ee' \Rightarrow_N z}$$

Regla para la aplicaci n

$$\frac{e \Rightarrow_E \lambda v. e_0 \quad e' \Rightarrow_E z' \quad (e_0/v \mapsto z') \Rightarrow_E z}{ee' \Rightarrow_E z}$$

$$C \xrightarrow[\psi]{\psi} C \rightarrow C$$

La función que mapea toda función a su punto fijo:

$$Y: (C \rightarrow C) \rightarrow C$$

$$Y(f) = p_f(p_f)$$

$$Y(f) = p_f(p_f) = f(p_f(p_f)) = p_f$$

¿Lo podemos hacer sintácticamente? Fijemos f :

$$p_f \doteq \lambda x. f(x x)$$

Entonces

$$p_f p_f \doteq (\lambda x. f(x x))(\lambda x. f(x x))$$

Ahora abstraigamos f :

$$Y \doteq \lambda f. (\lambda x. f(x x))(\lambda x. f(x x))$$

$$\phi: D_\infty \rightarrow [D_\infty \rightarrow D_\infty]$$

$$\psi: [D_\infty \rightarrow D_\infty] \rightarrow D_\infty$$

$$\phi \circ \psi = Id_{[D_\infty \rightarrow D_\infty]}$$

$$\psi \circ \phi = Id_{D_\infty}$$

Ambientes (Entornos): $Env = \langle var \rangle \rightarrow D_\infty$

Función semántica: $\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D_\infty$

Ecuaciones semánticas:

$$\llbracket v \rrbracket \eta = \eta v$$

$$\llbracket e_0 e_1 \rrbracket \eta = \phi(\llbracket e_0 \rrbracket \eta) (\llbracket e_1 \rrbracket \eta)$$

$$\llbracket \lambda v. e \rrbracket \eta = \psi(\lambda d \in D_\infty. \llbracket e \rrbracket [\eta | v : d])$$

Teorema de Coincidencia:

Si $\eta w = \eta' w$ para todo $w \in FV e$, entonces $\llbracket e \rrbracket \eta = \llbracket e \rrbracket \eta'$.

Teorema de Renombre

Si $v_{new} \notin FV e - \{v\}$, entonces $\llbracket \lambda v_{new}. (e/v \mapsto v_{new}) \rrbracket = \llbracket \lambda v. e \rrbracket$.

Sustituciones: $\Delta = \langle var \rangle \rightarrow \langle exp \rangle$

Teorema de Sustitución

Si $\llbracket \delta w \rrbracket \eta = \eta' w$ para todo $w \in FV e$, entonces $\llbracket e/\delta \rrbracket \eta = \llbracket e \rrbracket \eta'$.

Asumimos sin demostrar $\llbracket \Delta \Delta \rrbracket \eta = \perp$.

	Valores (V)	Términos (D)
Normal	$[D \rightarrow D]$	$V_\perp$
Eager	$[V \rightarrow D]$	$V_\perp$

Normal.

$$D^N = V_\perp^N$$

$$V^N \cong [D^N \rightarrow D^N]$$

isomorfismos y funciones:

$$\phi: V^N \rightarrow [D^N \rightarrow D^N]$$

$$\psi: [D^N \rightarrow D^N] \rightarrow V^N$$

Ambientes: $Env = (\langle var \rangle \rightarrow D^N)$

Función semántica:

$$\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D^N$$

$$\llbracket v \rrbracket \eta = \eta v$$

$$\llbracket e_0 e_1 \rrbracket \eta = \phi_\perp(\llbracket e_0 \rrbracket \eta)(\llbracket e_1 \rrbracket \eta)$$

$$\llbracket \lambda v. e \rrbracket \eta = \iota_\perp \circ \psi(\lambda d \in D. \llbracket e \rrbracket [\eta | v : d])$$

Eager

$$D^E = V_\perp^E$$

$$V^E \cong [V^E \rightarrow D^E]$$

isomorfismos y funciones:

$$\phi: V^E \rightarrow [V^E \rightarrow D^E]$$

$$\psi: [V^E \rightarrow D^E] \rightarrow V^E$$

Ambientes: $Env = \langle var \rangle \rightarrow V^E$

Función semántica:

$$\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D^E$$

$$\llbracket v \rrbracket \eta = \iota_\perp(\eta v)$$

$$\llbracket e_0 e_1 \rrbracket \eta = (\phi_\perp(\llbracket e_0 \rrbracket \eta))_\perp(\llbracket e_1 \rrbracket \eta)$$

$$\llbracket \lambda x. e \rrbracket \eta = \iota_\perp \circ \psi(\lambda v \in V^E. \llbracket e \rrbracket [\eta | x : v])$$

Generalizando:

$$\frac{e \Rightarrow_E [c] \quad e' \Rightarrow_E [c']}{e \oplus e' \Rightarrow_E [c \oplus c']}$$

donde $\oplus \in \{+, -, *, =, \neq, \leq, \geq, \dots\}$ pero $\oplus \notin \{/, \div\}$.

$$\frac{e \Rightarrow_E [c]}{\sim e \Rightarrow_E [\sim c]} \text{ con } \sim \in \{-, \neg\}$$

$$\frac{e \Rightarrow_E [i] \quad e' \Rightarrow_E [i']}{e \oplus e' \Rightarrow_E [i \oplus i']} \quad i' \neq 0, \text{ con } \oplus \in \{/, \div\}$$

$$\frac{e \Rightarrow_E \text{true} \quad e_0 \Rightarrow_E z}{\text{if } e \text{ then } e_0 \text{ else } e_1 \Rightarrow_E z}$$

$$\frac{e \Rightarrow_E \text{false} \quad e_1 \Rightarrow_E z}{\text{if } e \text{ then } e_0 \text{ else } e_1 \Rightarrow_E z}$$

$$\frac{e \Rightarrow_N \text{false}}{e \wedge e' \Rightarrow_N \text{false}}$$

Tuplas

$\langle exp \rangle ::= \dots \mid \langle \langle exp \rangle, \dots, \langle exp \rangle \rangle$
 $\mid \langle exp \rangle . \langle tagexp \rangle$
 $\langle tagexp \rangle ::= \langle natconst \rangle$

$\langle cnf \rangle ::= \langle intcnf \rangle \mid \langle boolcnf \rangle \mid \langle funcnf \rangle \mid \langle tuplecnf \rangle$

Formas canónicas de las tuplas para la evaluación Eager:

$\langle tuplecnf \rangle ::= \langle \langle cnf \rangle, \dots, \langle cnf \rangle \rangle$

Formas canónicas de tuplas para la evaluación Normal:

$\langle tuplecnf \rangle ::= \langle \langle exp \rangle, \dots, \langle exp \rangle \rangle$

No hay regla para la tupla $\langle e_0, \dots, e_{n-1} \rangle$ porque es una forma canónica.

Dicho de otro modo, la regla es $z \Rightarrow_N z$.

$$\frac{e \Rightarrow_N \langle e_0, \dots, e_{n-1} \rangle \quad e_k \Rightarrow_N z \quad k < n}{e.[k] \Rightarrow_N z}$$

$$\frac{e_0 \Rightarrow_E z_0 \quad \dots \quad e_{n-1} \Rightarrow_E z_{n-1}}{\langle e_0, \dots, e_{n-1} \rangle \Rightarrow_E \langle z_0, \dots, z_{n-1} \rangle}$$

$$\frac{e \Rightarrow_E \langle z_0, \dots, z_{n-1} \rangle}{e.[k] \Rightarrow_E z_k} \quad k < n$$

$$V \xrightarrow[\psi]{\uparrow \mathbb{N} \rightarrow \Phi} V_{int} + V_{bool} + V_{fun} \quad \text{y} \quad D = (V + \{\mathbf{error}\} + \{\mathbf{typeerror}\})_{\perp}$$

Los pre-dominios para valores son:

$$V_{int} = \mathbb{Z}$$

$$V_{bool} = \{T, F\}$$

$$V_{fun} = [V \rightarrow D]$$

$\iota_{int} \in V_{int} \rightarrow V$ se define como $V_{int} \xrightarrow{\iota_0} V_{int} + V_{bool} + V_{fun} \xrightarrow{\psi} V$

$$\iota_{int} = \psi \circ \iota_0$$

donde

$$\iota_0 \in V_{int} \rightarrow V_{int} + V_{bool} + V_{fun}$$

$$D = (V + \{\mathbf{error}\} + \{\mathbf{typeerror}\})_{\perp}$$

$$err, tyerr \in D$$

$$err = \iota_{\perp}(\iota_1(\mathbf{error}))$$

$$tyerr = \iota_{\perp}(\iota_2(\mathbf{typeerror}))$$

$$\iota_{norm} \in V \rightarrow D$$

$$\iota_{norm} = \iota_{\perp} \circ \iota_0$$

Uso habitual: $\iota_{norm} \circ \iota_{int} \in V_{int} \rightarrow D$

Abreviatura: $\iota_{int} = \iota_{norm} \circ \iota_{int} \in V_{int} \rightarrow D$

En general: $\iota_{\theta} = \iota_{norm} \circ \iota_{\theta} \in V_{\theta} \rightarrow D$ con $\theta \in \{int, bool, fun\}$.

Si $f: V_{int} \rightarrow D$ entonces $f_{int}: V \rightarrow D$ se define:

$$f_{int}(\iota_{int} i) = f i$$

$$f_{int}(\iota_{bool} b) = tyerr$$

$$f_{int}(\iota_{fun} f) = tyerr$$

$$Env = \langle var \rangle \rightarrow V$$

$$\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D$$

$$\begin{aligned} \llbracket 0 \rrbracket \eta &= \iota_{int} 0 \\ \llbracket true \rrbracket \eta &= \iota_{bool} T \\ \llbracket -e \rrbracket \eta &= (\lambda i \in V_{int}. \iota_{int}(-i))_{int*}(\llbracket e \rrbracket \eta) \\ \llbracket e + e' \rrbracket \eta &= (\lambda i \in V_{int}. (\lambda i' \in V_{int}. \iota_{int}(i + i'))_{int*}(\llbracket e \rrbracket \eta))_{int*}(\llbracket e' \rrbracket \eta) \\ &\vdots \\ \llbracket e/e' \rrbracket \eta &= (\lambda i \in V_{int}. (\lambda i' \in V_{int}. \\ &\quad \text{if } i' = 0 \text{ then err else } \iota_{int}(i/i'))_{int*}(\llbracket e' \rrbracket \eta))_{int*}(\llbracket e \rrbracket \eta) \end{aligned}$$

eu normal

Entornos: $Env = \langle var \rangle \rightarrow D$

$$\begin{aligned} V_{int} &= \mathbb{Z} \\ V_{bool} &= \{T, F\} \\ V_{fun} &= [D \rightarrow D] \end{aligned}$$

$$\begin{aligned} Env &= \langle var \rangle \rightarrow V \\ \llbracket _ \rrbracket &\in \langle exp \rangle \rightarrow Env \rightarrow D \end{aligned}$$

$$\begin{aligned} \llbracket v \rrbracket \eta &= \eta v \\ \llbracket ee' \rrbracket \eta &= (\lambda f \in V_{fun}. f \llbracket e' \rrbracket \eta)_{fun*}(\llbracket e \rrbracket \eta) \\ \llbracket \lambda v. e \rrbracket \eta &= \iota_{fun}(\lambda d \in D. \llbracket e \rrbracket [\eta | v : d]) \end{aligned}$$

$$V \simeq V_{int} + V_{bool} + V_{fun} + V_{tuple}$$

Semántica denotacional Eager:

$$\begin{aligned} V_{tuple} &= V^* = \bigcup_{n \in \mathbb{N}} V^n \\ &= \{\langle \rangle\} \cup \{\langle v \rangle | v \in V\} \cup \dots \{\langle v_1, \dots, v_n \rangle | v_i \in V\} \end{aligned}$$

Notar que $\langle \iota_{tuple}(\iota_{int} 2, \iota_{bool} T) \rangle \in V_{tuple}$

Semántica denotacional Normal:

$$V_{tuple} = D^*$$

$$Env = \langle var \rangle \rightarrow V$$

$$\llbracket _ \rrbracket \in \langle exp \rangle \rightarrow Env \rightarrow D$$

$$\begin{aligned} \llbracket v \rrbracket \eta &= \iota_{norm}(\eta v) \\ \llbracket ee' \rrbracket \eta &= (\lambda f \in V_{fun}. (\lambda z \in V. f z) * (\llbracket e' \rrbracket \eta))_{fun*}(\llbracket e \rrbracket \eta) \\ &= (\lambda f \in V_{fun}. f * (\llbracket e' \rrbracket \eta))_{fun*}(\llbracket e \rrbracket \eta) \\ \llbracket \lambda v. e \rrbracket \eta &= \iota_{fun}(\lambda z \in V. \llbracket e \rrbracket [\eta | v : z]) \end{aligned}$$

$$\begin{aligned} \llbracket \langle e_0, \dots, e_{n-1} \rangle \rrbracket \eta &= \\ &(\lambda z_0 \in V. \\ &\quad (\lambda z_1 \in V. \\ &\quad \dots (\lambda z_{n-1} \in V. \iota_{tuple}(\langle z_0, \dots, z_{n-1} \rangle) * (\llbracket e_{n-1} \rrbracket \eta)) \dots \\ &\quad) * (\llbracket e_1 \rrbracket \eta) \\ &\quad) * (\llbracket e_0 \rrbracket \eta) \end{aligned}$$

$$\begin{aligned} \llbracket e.[k] \rrbracket \eta &= \\ &(\lambda t \in V_{tuple}. \\ &\quad \text{if } k \geq |t| \text{ then } tyerr \text{ else } \iota_{norm} t_k \\ &\quad)_{tuple*}(\llbracket e \rrbracket \eta) \end{aligned}$$

Normal

$$\llbracket \langle e_0, \dots, e_{n-1} \rangle \rrbracket \eta = \iota_{tuple}(\llbracket e_0 \rrbracket \eta, \dots, \llbracket e_{n-1} \rrbracket \eta)$$

$$\llbracket e.[k] \rrbracket \eta =$$

$$(\lambda t \in V_{tuple}.$$

$$\text{if } k \geq |t| \text{ then } tyerr \text{ else } t_k$$

$$)_{tuple*}(\llbracket e \rrbracket \eta)$$

$$\begin{aligned} \langle exp \rangle &::= \text{let } \langle pat \rangle \equiv \langle exp \rangle, \dots, \langle pat \rangle \equiv \langle exp \rangle \text{ in } \langle exp \rangle \\ &\quad | \lambda \langle pat \rangle. \langle exp \rangle \\ \langle pat \rangle &::= \langle var \rangle | \langle \langle pat \rangle, \dots, \langle pat \rangle \rangle \end{aligned}$$

Se definen como abreviaturas (azucar sintactico):

$$\lambda \langle p_1, \dots, p_n \rangle. e \stackrel{def}{=} \lambda v. \text{let } p_1 \equiv v.0, \dots, p_n \equiv v.[n-1] \text{ in } e$$

donde v es una variable nueva (no ocurre libre en e ni en ninguno de los patrones).

$$\text{let } p_1 \equiv e_1, \dots, p_n \equiv e_n \text{ in } e \stackrel{def}{=} (\lambda p_1 \dots \lambda p_n. e) e_1 \dots e_n$$

$$\langle exp \rangle ::= \dots | \text{letrec } \langle var \rangle \equiv \lambda \langle var \rangle. \langle exp \rangle \text{ in } \langle exp \rangle \quad (\text{Eval. Eager})$$

$$\langle exp \rangle ::= \dots | \text{rec } \langle exp \rangle \quad (\text{Eval. Normal})$$

$$\text{letrec } fact \equiv \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * fact(n-1) \text{ in } fact 2$$

$$\frac{(e/f \mapsto \lambda u. e_0^*) \Rightarrow_E z}{\text{letrec } f \equiv \lambda u. e_0 \text{ in } e \Rightarrow_E z}$$

donde $e_0^* = \text{letrec } f \equiv \lambda u. e_0 \text{ in } e_0$

$$\llbracket \text{letrec } v \equiv \lambda u. e_0 \text{ in } e \rrbracket \eta = \llbracket e \rrbracket [\eta | v : \iota_{fun} g]$$

donde g es el menor punto fijo de

$$F f z = \llbracket e \rrbracket [\eta | v : \iota_{fun} f, u : z]$$

o sea

$$g = \mathbf{Y}_{V_{fun}} F \quad \mathbf{Y}_{V_{fun}} F = \sqcup_i F^i \perp$$

$$g = \mathbf{Y}_{V_{fun}} (\lambda f \in V_{fun}. \lambda z \in V. \llbracket e \rrbracket [\eta | v : \iota_{fun} f, u : z])$$

$$\frac{e (\text{rec } e) \Rightarrow_N z}{\text{rec } e \Rightarrow_N z}$$

$$\llbracket \text{rec } e \rrbracket \eta = (\lambda f \in V_{fun}. \mathbf{Y}_D f)_{fun} (\llbracket e \rrbracket \eta)$$

donde $\mathbf{Y}_D$ es el operador de menor punto fijo

$$\mathbf{Y}_D f = \sqcup_i f^i \perp$$

$$\llbracket \text{letrec } v \equiv \lambda u. e_0 \text{ in } e \rrbracket \eta = \llbracket e \rrbracket [\eta | v : \iota_{fun} g]$$

$$F f z = \llbracket e \rrbracket [\eta | v : \iota_{fun} f, u : z]$$

$$g = \mathbf{Y}_{V_{fun}} F$$

$$g = \mathbf{Y}_{V_{fun}} (\lambda f. \lambda z. \llbracket e \rrbracket [\eta | v : \iota_{fun} f, u : z])$$

El lenguaje Iswim extiende al lenguaje aplicativo eager mediante las siguientes construcciones:

$\langle exp \rangle ::= \dots$
 | **ref** $\langle exp \rangle$
 | **val** $\langle exp \rangle$
 | $\langle exp \rangle := \langle exp \rangle$
 | $\langle exp \rangle =_{ref} \langle exp \rangle$

Mientras que para las formas canónicas agregamos **referencias**:

$\langle cnf \rangle ::= \dots \mid V_{ref}$

skip $=_{def} \langle \rangle$

$e; e' =_{def} \text{let } v = e \text{ in } e' \quad (v \notin FV e')$

newvar $v := e \text{ in } e' =_{def} \text{let } v = \text{ref } e \text{ in } e'$

while $e \text{ do } e' =_{def} \text{letrec } f = \lambda v. \text{if } e \text{ then } e'; f v \text{ else skip in } f \langle \rangle$

Aquí las variables f y v no deben ocurrir en e ni e' .

Formas Canónicas:

$\langle cnf \rangle ::= \dots \mid V_{ref}$

Por ejemplo la regla

$$\frac{e \Rightarrow_E \lambda v. e_0 \quad e' \Rightarrow_E z' \quad (e_0/v \mapsto z') \Rightarrow_E z}{ee' \Rightarrow_E z}$$

Semántica big-step (evaluación):

se transforma en:

$$\sigma, e \Rightarrow z, \sigma'$$

$$\frac{\sigma, e \Rightarrow r, \sigma' \quad \sigma', e' \Rightarrow z', \sigma''}{\sigma, e := e' \Rightarrow z', [\sigma'' | r : z']}$$

$$\frac{\sigma, e \Rightarrow_E \lambda v. e_0, \sigma' \quad \sigma', e' \Rightarrow_E z', \sigma'' \quad \sigma'', (e_0/v \mapsto z') \Rightarrow_E z, \hat{\sigma}}{\sigma, e e' \Rightarrow_E z, \hat{\sigma}}$$

$$\frac{\sigma, e \Rightarrow z, \sigma'}{\sigma, \text{ref } e \Rightarrow r, [\sigma' | r : z]} \quad r = \text{new}(\sigma')$$

$$\frac{\sigma, e \Rightarrow r, \sigma' \quad \sigma', e' \Rightarrow z', \sigma''}{\sigma, e := e' \Rightarrow z', [\sigma' \mid r : z']}$$

$$\frac{\sigma, e \rightarrow_E \wedge e, \sigma \quad \sigma, e \rightarrow_E z, \sigma \quad \sigma, (e_0 / e \mapsto z) \rightarrow_E z, \sigma}{\sigma, e e' \Rightarrow_E z, \hat{\sigma}}$$

$$\frac{\sigma, e \Rightarrow z, \sigma'}{\sigma, \mathbf{ref} \, e \Rightarrow r, [\sigma' \mid r : z]} \quad r = \mathbf{new}(\sigma')$$

$$\frac{\sigma, e \Rightarrow r, \sigma'}{\sigma, \mathbf{val} \, e \Rightarrow \sigma' r, \sigma'} \quad r \in \mathbf{dom}(\sigma')$$

$$\frac{\sigma, e \Rightarrow r, \sigma' \quad \sigma', e' \Rightarrow r', \sigma''}{\sigma, e =_{\mathbf{ref}} e' \Rightarrow [r = r'], \sigma''}$$

Para la semántica denotacional tenemos estados "denotacionales":

$$\Sigma = \bigcup_{F \subset_{\text{fin}} V_{\text{ref}}} F \rightarrow V$$

$$D = (\Sigma \times V + \{\mathbf{error}, \mathbf{typeerror}\})_{\perp} \quad \text{donde} \\ V = V_{\text{int}} + V_{\text{bool}} + V_{\text{fun}} + V_{\text{tuple}} + V_{\text{ref}}$$

Las funciones ahora deben reflejar la posibilidad de cambio del estado, es decir, una función no sólo toma el valor sino además el estado producido por la evaluación del operando. Por ello:

$$V_{\text{fun}} = \Sigma \times V \rightarrow D$$

$$\llbracket \mathbf{val} \, e \rrbracket \eta \sigma = (\lambda \langle \sigma', r \rangle. \text{if } r \in \mathbf{dom}(\sigma') \\ \text{then } \iota_{\text{norm}} \langle \sigma', \sigma' r \rangle \text{else } \mathbf{err})_{\text{ref}*} (\llbracket e \rrbracket \eta \sigma)$$

$$\llbracket \mathbf{ref} \, e \rrbracket \eta \sigma = (\lambda \langle \sigma', z \rangle. \iota_{\text{norm}} \langle [\sigma' \mid r_{\sigma'} : z], \iota_{\text{ref}} r_{\sigma'} \rangle)_* (\llbracket e \rrbracket \eta \sigma) \\ \text{donde } (r_{\sigma'} = \mathbf{new}(\sigma'))$$

$$\llbracket e := e' \rrbracket \eta \sigma = (\lambda \langle \sigma', r \rangle. (\lambda \langle \sigma'', z \rangle. \iota_{\text{norm}} \langle [\sigma'' \mid r : z], z \rangle)_* (\llbracket e' \rrbracket \eta \sigma'))_{\text{ref}*} (\llbracket e \rrbracket \eta \sigma)$$

$$\llbracket e =_{\text{ref}} e' \rrbracket \eta \sigma = (\lambda \langle \sigma', r \rangle. (\lambda \langle \sigma'', z \rangle. \iota_{\text{norm}} \langle \sigma'', \iota_{\text{bool}} r = r' \rangle)_{\text{ref}*} (\llbracket e' \rrbracket \eta \sigma'))_{\text{ref}*} (\llbracket e \rrbracket \eta \sigma)$$

$$\llbracket 0 \rrbracket \eta \sigma = \iota_{\text{norm}} \langle \sigma, \iota_{\text{int}} 0 \rangle$$

$$\llbracket \mathbf{true} \rrbracket \eta \sigma = \iota_{\text{norm}} \langle \sigma, \iota_{\text{bool}} T \rangle$$

$$\llbracket -e \rrbracket \eta \sigma = (\lambda \langle \sigma', i \rangle. \iota_{\text{norm}} \langle \sigma', \iota_{\text{int}} -i \rangle)_{\text{int}*} (\llbracket e \rrbracket \eta \sigma) \\ \llbracket e + e' \rrbracket \eta \sigma = (\lambda \langle \sigma', i \rangle. (\lambda \langle \sigma'', j \rangle. \iota_{\text{norm}} \langle \sigma'', \iota_{\text{int}} i + j \rangle)_{\text{int}*} (\llbracket e' \rrbracket \eta \sigma'))_{\text{int}*} (\llbracket e \rrbracket \eta \sigma)$$

$$\llbracket v \rrbracket \eta \sigma = \iota_{\text{norm}} \langle \sigma, \eta v \rangle$$

$$\llbracket e \, e' \rrbracket \eta \sigma = (\lambda \langle \sigma', f \rangle. f_* (\llbracket e' \rrbracket \eta \sigma'))_{\text{fun}*} (\llbracket e \rrbracket \eta \sigma) \\ \llbracket \lambda v. e \rrbracket \eta \sigma = \iota_{\text{norm}} \langle \sigma, \iota_{\text{fun}} (\lambda \langle \sigma', z \rangle. \llbracket e \rrbracket [\eta \mid v : z] \sigma') \rangle$$

$$\llbracket !x \rrbracket \sigma = \iota_{\text{out}} (\llbracket x \rrbracket \sigma, \iota_{\text{tem}} \sigma)$$

$$\llbracket ?y \rrbracket \sigma = \iota_{\text{in}} (\lambda \kappa \in \mathbb{Z}. \iota_{\text{tem}} [\sigma \mid y : \kappa])$$

$$\llbracket \text{letrec } w = \lambda v. e \text{ in } e' \rrbracket \eta \sigma = \llbracket e' \rrbracket [\eta \mid w : \iota_{fun} g] \sigma$$

donde

$$g = \mathbf{Y}_{V_{fun}} F$$

$$F f \langle \sigma', z \rangle = \llbracket e \rrbracket [\eta \mid w : \iota_{fun} f \mid v : z] \sigma'$$

Lema Si $\sigma, e \Rightarrow z, \sigma' \text{ y } \sigma', e' \Rightarrow z', \sigma''$, entonces

$$\sigma, e; e' \Rightarrow z', \sigma''$$

Lema $\llbracket e; e' \rrbracket \eta \sigma = (\lambda \langle \sigma', z \rangle. \llbracket e' \rrbracket \eta \sigma')_*(\llbracket e \rrbracket \eta \sigma)$